UNIVERSIDAD TÉCNICA DE ORURO
FACULTAD NACIONAL DE INGENIERÍA
CARRERA INGENIERÍA INFORMÁTICA



Proyecto de Grado para optar al título de:

Licenciado en Ingeniería Informática

GESTIÓN INTELIGENTE DE PARQUEADEROS BASADA EN VISIÓN ARTIFICIAL Y APRENDIZAJE PROFUNDO

Postulante: Univ. Nina Nuñez Marco

Oruro, 4 de noviembre de 2026

RESUMEN

Se desarrolló un sistema de gestión de acceso vehicular para un parqueadero de Oruro con el fin de sustituir el registro manual de ingresos y salidas por un proceso automatizado de visión artificial y aprendizaje profundo. El sistema se apoyó en las cámaras existentes del recinto e integró un conjunto de datos propio, modelos de detección y clasificación, un reconocedor de matrículas y una plataforma web con base de datos, cola de tareas y notificación en tiempo real. Se verificaron 640 matrículas reales y se generaron 360,000 muestras sintéticas. La detección de vehículos alcanzó un mAP@50 de 0.901; la de placas, una exhaustividad de 0.9714; la clasificación de tipo, un F1 de 0.8887; y la de color, un F1 macro de 0.8124. El reconocimiento de matrículas obtuvo 0.851 de exactitud por carácter y 52.6 % de coincidencia exacta por recorte, ambos limitados por la resolución de captura. Dos criterios quedaron por debajo del umbral fijado (el F1 de color y la exactitud por carácter), la determinación de eventos quedó inconclusa por tamaño de muestra y la latencia por evento no llegó a medirse; por ello se rechaza la hipótesis de investigación y se acepta la hipótesis nula conforme al criterio definido de antemano. La estimación económica arrojó 62 puntos de función y un costo total de 33,170 bolivianos. El sistema está construido e integrado por componentes, pero no satisface el conjunto de criterios de desempeño en su estado actual.

Palabras clave: visión artificial, aprendizaje profundo, reconocimiento de matrículas, gestión de acceso, parqueadero.

ABSTRACT

A vehicle access management system was developed for a parking facility in Oruro, Bolivia, to replace the manual registration of entries and exits with an automated process based on computer vision and deep learning. The system relied on the facility's existing cameras and integrated a dedicated dataset, detection and classification models, a license plate recognizer, and a web platform with a database, a task queue, and real-time notification. A total of 640 real license plates were verified and 360,000 synthetic samples were generated. Vehicle detection reached 0.901 mAP@50; plate detection, 0.9714 recall; vehicle type classification, 0.8887 F1; and color classification, 0.8124 macro F1. License plate recognition obtained 0.851 character-level accuracy and 52.6 % per-crop exact match, both limited by capture resolution. Two criteria fell below the target threshold (color F1 and character accuracy), event determination remained inconclusive because of sample size, and per-event latency was not measured; the research hypothesis is therefore rejected and the null hypothesis is accepted under the predefined criterion. The cost estimate yielded 62 function points and a total cost of 33,170 bolivianos. The system is built and integrated by components, but it does not meet the set of performance criteria in its current state.

Keywords: computer vision, deep learning, license plate recognition, access management, parking.

Índice general

RESUMEN II

ABSTRACT III

LISTA DE ABREVIATURAS XXII

INTRODUCCIÓN 1

ANTECEDENTES 1

TRABAJOS DESARROLLADOS 2

INFORMACIÓN DOCUMENTAL Y ESTADÍSTICA 3

SITUACIÓN PROBLEMÁTICA 5

PLANTEAMIENTO DEL PROBLEMA 6

OBJETO DE ESTUDIO 7

OBJETIVOS 7

OBJETIVO GENERAL 7

OBJETIVOS ESPECÍFICOS 7

CAMPO DE ACCIÓN 8

HIPÓTESIS 9

CRITERIO DE EVALUACIÓN DE LA HIPÓTESIS 10

JUSTIFICACIÓN 12

JUSTIFICACIÓN TÉCNICA 12

JUSTIFICACIÓN ECONÓMICA 13

JUSTIFICACIÓN SOCIAL 13

ALCANCES Y LIMITACIONES 13

ALCANCES 13

LIMITACIONES 14

APORTES 16

APORTE TEÓRICO 16

APORTE PRÁCTICO 16

APORTE ACADÉMICO 16

INGENIERÍA DEL PROYECTO 16

ORGANIZACIÓN DEL TRABAJO 19

I. Marco Teórico **20**

1.1. Introducción 20

1.2. Visión Artificial y Procesamiento de Imágenes 21

1.2.1. Modelo de la imagen digital y espacios de color (RGB, HSV) aplicados al entorno 21

1.2.2. Filtrado espacial lineal y suavizado Gaussiano 22

1.2.3. Morfología matemática (Top-Hat, Apertura) en el preprocesamiento de matrículas 22

1.2.4. Limitaciones del procesamiento clásico ante perturbaciones ambientales (sombras, oclusión) 23

1.3. Fundamentos de Aprendizaje Automático y Aprendizaje Profundo 24

1.3.1. El Paradigma del Aprendizaje Supervisado y Partición de Datos . . . 25

1.3.2. Arquitectura Base: Neuronas, Propagación y Activaciones no Lineales 25

1.3.3. Optimización de Modelos: Descenso de Gradiente y Funciones de Pérdida 26

1.3.4. Generalización: Sesgo-Varianza, Sobreajuste y Regularización 26

1.3.5. Redes Neuronales Convolucionales (CNN) en la Percepción Visual . . 27

1.3.6. Aprendizaje por Transferencia (Transfer Learning), Fine-Tuning y Aumento de Datos 28

1.4. Detección de Objetos mediante Aprendizaje Profundo en Entornos Urbanos . 30

1.4.1. Formulación de la detección: Cajas delimitadoras y Funciones de Pérdida 30

1.4.2. Taxonomía de detectores: Modelos de una etapa para inferencia en tiempo real 31

1.4.3. Evolución arquitectónica de la familia YOLO 31

1.4.4. Cuellos de botella en el Borde (Edge AI): NMS y DFL en oclusión vehicular 32

1.4.5. Arquitecturas modernas (NMS-Free) y Detección de Objetos Pequeños 33

1.5. Percepción Vehicular y Reconocimiento de Matrículas 34

1.5.1. Clasificación de atributos visuales en espacios de características discriminativos 34

1.5.2. Localización de objetos pequeños con geometría específica: detección de matrículas 35

1.5.3. Del OCR clásico por segmentación al modelado de secuencias 35

1.5.4. Modelos de secuencias (RNN, LSTM) y alineación mediante CTC . . 36

1.5.5. Dominio de aplicación: estructura y gramática formal de las matrículas en Bolivia 37

1.5.6. Factores de degradación visual en el reconocimiento de matrículas . . 38

1.6. Seguimiento Vehicular y Determinación de Eventos 39

1.6.1. Formulación matemática del seguimiento de objetos múltiples (MOT) 40

1.6.2. Modelado de incertidumbre y estimación del estado mediante Filtro de Kalman 40

1.6.3. Asociación de datos y optimización combinatoria: Algoritmo Húngaro y SORT 41

1.6.4. Determinación de eventos a partir de trayectorias vehiculares 42

1.7. Evaluación y Optimización de Modelos de Visión Artificial 43

1.7.1. Métricas de clasificación y detección: Precision, Recall, F1-Score, mAP 44

1.7.2. Métricas de reconocimiento de secuencias: Exactitud por carácter y CER 45

1.7.3. Métricas de seguimiento de objetos: MOTA, MOTP e IDF1 46

1.7.4. Compromiso entre exactitud y latencia: Rendimiento en el Borde . . 47

1.7.5. Compresión de modelos: Cuantización de pesos (FP32, FP16, INT8) . 48

1.7.6. Valores de referencia y umbrales de aceptación del proyecto 49

II. Diseño del Sistema **52**

2.1. Introducción 52

2.2. Requerimientos del Sistema 53

2.2.1. Requerimientos funcionales 53

2.2.2. Requerimientos no funcionales 55

2.2.3. Requerimientos de infraestructura y entorno 57

2.3. Descripción de la Arquitectura General 58

2.3.1. Capa de captura 59

2.3.2. Capa de servicios 59

2.3.3. Capa de procesamiento asíncrono 60

2.3.4. Capa de persistencia 60

2.3.5. Capa de presentación 60

2.3.6. Flujo de comunicación 61

2.4. Diseño del Subsistema de Visión Artificial 61

2.4.1. Arquitectura de los pipelines de visión 61

2.4.2. Detección de vehículos 64

2.4.3. Seguimiento vehicular 64

2.4.4. Reconocimiento de matrículas 65

2.4.5. Clasificación de atributos del vehículo 65

2.4.6. Determinación de eventos y contrato de datos 66

2.5. Diseño del Subsistema de Gestión 68

2.5.1. Arquitectura del subsistema de gestión 68

2.5.2. Modelo entidad-relación 68

2.5.3. Modelo de aforo perimetral y gestión de ocupación 73

2.5.4. Especificación de la interfaz de programación 74

2.5.5. Procesamiento asíncrono y notificación en tiempo real 76

2.5.6. Trazabilidad y control de autorización 76

2.6. Diseño de la Interfaz de Usuario 77

2.6.1. Criterios de diseño y estructura de navegación 77

2.6.2. Vistas de operación 78

2.6.3. Vistas de administración y análisis 79

2.6.4. Vistas de incidencia y diagnóstico 80

III Implementación y Pruebas del Sistema **82**

3.1. Introducción 82

3.2. Modelo de Implementación 83

3.2.1. Entorno tecnológico y dependencias 83

3.2.2. Arquitectura de despliegue 85

3.2.3. Estructura del repositorio y organización del código 87

3.3. Construcción del Conjunto de Datos 88

3.3.1. Calibración de zonas de interés y línea virtual 90

3.3.2. Detección de movimiento y segmentación de ventanas 91

3.3.3. Selección de fotogramas representativos 92

3.3.4. Pseudo-etiquetado, generación sintética y etiquetado humano 92

3.3.5. Participación, composición y protocolo de ablación del conjunto de datos 94

3.4. Implementación del Subsistema de Visión Artificial 96

3.4.1. Adquisición del flujo de video 96

3.4.2. Detección de vehículos 97

3.4.3. Seguimiento vehicular 97

3.4.4. Detección de placas de matrícula 98

3.4.5. Reconocimiento óptico de caracteres (CRNN + CTC) 99

3.4.6. Determinación de eventos y trayectorias 102

3.4.7. Clasificación de atributos del vehículo 103

3.5. Implementación del Subsistema de Gestión 103

3.5.1. Modelo físico de datos y migraciones 103

3.5.2. Interfaz de programación 105

3.5.3. Procesamiento asíncrono 109

3.5.4. Notificación en tiempo real 110

3.5.5. Aplicación web 110

3.5.6. Estado de implementación del subsistema 111

3.6. Optimización y Despliegue en el Borde 112

3.6.1. Exportación a formatos intermedios 113

3.6.2. Compilación y ejecución con TensorRT 114

3.6.3. Estrategias de reducción de carga computacional 115

3.7. Pruebas y resultados 116

3.7.1. Condiciones experimentales 116

3.7.2. Detección de vehículos (H1.1): medida 117

3.7.3. Clasificación de tipo de vehículo (H1.2) 119

3.7.4. Clasificación de color predominante (H1.3) 120

3.7.5. Detección de placas de matrícula (H1.4, componente de detección) . . 122

3.7.6. Reconocimiento de placas (H1.4, componente de reconocimiento) . . . 123

3.7.7. Determinación de eventos de ingreso y salida (H1.5) 130

3.7.8. Rendimiento computacional: parcialmente evaluado 132

3.7.9. Capacidad de un trabajador de inferencia 132

3.7.10. Contraste de hipótesis 134

3.7.11. Análisis y limitaciones 135

3.8. Interfaces del Sistema 137

3.8.1. Acceso y autenticación 137

3.8.2. Panel de monitoreo 138

3.8.3. Búsqueda de incidentes y detalle del vehículo 140

3.8.4. Historial de sesiones 140

3.8.5. Telemetría del modelo 142

3.8.6. Vistas de administración 143

3.8.7. Documentación de la interfaz de programación 144

IV.Estimación de Costos **145**

4.1. Introducción 145

4.2. Estimación del Costo de Software 146

4.2.1. Tamaño funcional del sistema 146

4.2.2. Contraste con el tamaño medido del código 148

4.2.3. Esfuerzo y costo de desarrollo 149

4.3. Estimación del Equipamiento 151

4.4. Costo Total Estimado 153

V. Conclusiones y Recomendaciones **154**

5.1. Introducción 154

5.2. Conclusiones 155

5.2.1. Síntesis del sistema construido 155

5.2.2. Contrastación de la hipótesis general 156

5.2.3. Cumplimiento de los objetivos específicos 157

5.2.4. La brecha del reconocimiento de matrículas 160

5.2.5. Alcance del aporte 164

5.2.6. Aportes verificables 165

5.3. Recomendaciones 166

5.3.1. Validación de extremo a extremo y puesta en operación 166

5.3.2. Condiciones de captura 167

5.3.3. Reconocimiento de matrículas 167

5.3.4. Medición pendiente por muestra insuficiente 168

5.3.5. Líneas de continuidad 169

5.3.6. Consideración final 170

Bibliografía **171**

A. Manual de usuario **180**

A.1. Requisitos y acceso 180

A.2. Panel de monitoreo 181

A.3. Búsqueda de incidentes 182

A.4. Detalle del vehículo y cierre manual 183

A.5. Historial de sesiones 183

A.6. Resolución de problemas frecuentes 184

A.7. Límites del manual 185

B. Protocolo de anotación y verificación humana **186**

B.1. Principios 186

B.2. Participación de los conjuntos 187

B.3. Etiquetado del tipo de vehículo 187

B.3.1. Árbol de decisión 188

B.3.2. Las dos confusiones reales 188

B.3.3. La clase fusionada 189

B.4. Etiquetado del color 189

B.5. Localización de placas 190

B.5.1. Propuesta automática 190

B.5.2. Revisión humana 190

B.6. Verificación del texto de matrícula 190

B.7. Registro y trazabilidad 191

B.8. Reproductibilidad del etiquetado 192

C. Reproducibilidad **193**

C.1. Entorno de ejecución 193

C.2. Semillas y determinismo 194

C.3. Comandos por etapa 195

C.3.1. Construcción y verificación de los conjuntos 195

C.3.2. Ablación y entrenamiento del reconocimiento 195

C.3.3. Clasificadores, detector de placas y evaluación 196

C.3.4. Intervalos de confianza 197

C.3.5. Capacidad del trabajador de inferencia 197

C.3.6. Conjunto de detección y evaluación de H1.1 198

C.3.7. Determinación de eventos y evaluación de H1.5 199

C.4. Artefactos 200

C.5. Verificación del sistema 200

C.6. Condiciones que limitan la reproducción 201

Índice de figuras

0.1. Horas de grabación por día de la semana en el recinto de estudio, entre el 16 de marzo y el 16 de mayo de 2026. Fuente: metadatos de las sesiones registradas por las cámaras del recinto. 5

2.1. Arquitectura general en capas del sistema de gestión inteligente de parqueaderos. 59

2.2. Pipeline de operación del subsistema de visión artificial. 62

2.3. Modelo entidad-relación del subsistema de gestión. Cada entidad indica su nombre conceptual y, en tipografía monoespaciada, el nombre físico de la tabla en la base de datos. 69

2.4. Estructura de navegación de la aplicación web. 78

2.5. Maquetas de las vistas de operación. 79

2.6. Maquetas de las vistas de administración y análisis. 80

2.7. Maquetas de las vistas de incidencia y diagnóstico. 81

3.1. Arquitectura de despliegue en contenedores. 85

3.2. Pipeline de construcción del conjunto de datos. 89

3.3. Exactitud por carácter de la etapa 1 sobre su propio dominio (eje izquierdo, línea continua) y sobre el dominio real (eje derecho, línea discontinua). La mejora sintética no se traduce en mejora real: las series están desacopladas y la coincidencia exacta real es nula en todas las épocas. 128

3.4. Pantalla de acceso a la aplicación. 138

3.5. Panel de monitoreo con la ocupación, el estado de las cámaras, las sesiones activas y el registro de eventos en vivo. 139

3.6. Búsqueda de incidentes por ventana temporal y color, con los candidatos y su evidencia visual. 141

3.7. Detalle de un vehículo con su recorte, sus atributos y el historial de sesiones. 142

3.8. Telemetría del modelo: eventos, confianza media y latencia por percentil. . . 143

Índice de cuadros

0.2. Parque automotor registrado en Bolivia y en el departamento de Oruro. Fuente: INE, con base en el RUAT [7]. 4

0.3. Proyección del parque automotor de Oruro bajo dos escenarios de crecimiento. Elaboración propia a partir de la serie del INE [7]. Es una extrapolación mecánica, no un pronóstico. 4

0.4. Ingeniería del Proyecto 17

1.1. Umbrales de aceptación y valores de referencia 49

2.1. Requerimientos funcionales del sistema 53

2.2. Requerimientos no funcionales del sistema 56

2.3. Componentes de infraestructura y entorno 57

2.4. Módulos del pipeline de percepción 63

2.5. Contrato de datos del evento de acceso 66

2.6. Diccionario de datos del subsistema de gestión 69

2.7. Especificación de los recursos de la API 75

3.1. Entorno tecnológico del sistema 83

3.2. Servicios de despliegue y sus dependencias 85

3.3. Organización del código fuente 87

3.4. Parámetros de configuración del extractor 89

3.5. Organización de la salida del extractor y generador 93

3.6. Composición del conjunto de datos de reconocimiento de matrículas 95

3.7. Recursos implementados de la interfaz de programación 105

3.8. Estado de implementación del subsistema de gestión 111

3.9. Huella en disco por formato de modelo 113

3.10. Detección de vehículos sobre 337 fotogramas con 304 cajas de referencia humana. Intervalo de AP por remuestreo de imágenes completas (2000 réplicas); intervalo de recall por Wilson sobre cajas. 118

3.11. Clasificador de tipo: métricas por clase sobre el conjunto de prueba ($n = 633$). 120

3.12. Clasificador de color sobre el conjunto de prueba. Las clases con $n < 40$ se listan solo para dejar constancia de su existencia. 121

3.13. Detector de placas frente al etiquetador automático, sobre el mismo conjunto de referencia humana. 122

3.14. Reconocimiento sobre recortes de placa aislados ($n = 152$, 99 placas, partición por placa). 123

3.15. Exactitud por carácter según la altura del recorte, CRNN. Mismo modelo en todos los tramos. 123

3.16. Sistema completo sobre 99 recortes de vehículo con texto verificado. Detector y lector reciben exactamente la misma entrada. Métricas por recorte; los valores con promedio por placa se reportan en el texto. 124

3.17. Exactitud por carácter del primer epoch de la etapa 1 en ejecuciones idénticas. La diferencia no proviene de la semilla: fija los pesos iniciales, pero no la trayectoria numérica. 126

3.18. Efecto de extender la etapa 1. La etapa 2 es idéntica en ambos casos. 127

3.19. Etapa 1 evaluada sobre su propio dominio y sobre el dominio real, en una misma corrida de cinco épocas. La coincidencia exacta sobre material real nunca abandona el cero. 128

3.20. Determinación de eventos sobre una hora completa de las dos cámaras, con 9 cruces de referencia humana. Intervalo de Wilson [71], [72] sobre la proporción de aciertos. 131

3.21. Costo por fotograma de un trabajador de inferencia y capacidad resultante. Rango sobre tres corridas y dos accesos. 133

3.22. Estado de cada hipótesis específica frente a la evidencia medida. Los intervalos de AP y F1 macro se obtienen por remuestreo; el de exactitud por carácter, por remuestreo de las 99 placas y promedio macro por placa; los de recall, Exact Match y determinación de eventos, con la aproximación de Wilson [71], [72]. El Exact Match se calcula por recorte. Ningún intervalo incorpora la varianza entre corridas. 134

4.1. Recuento de puntos de función sin ajustar 146

4.2. Tamaño del código fuente por área 148

4.3. Parámetros de valoración del costo de software 150

4.4. Equipamiento requerido por el proyecto 151

4.5. Estructura del costo total estimado 153

5.1. Cumplimiento de los objetivos específicos 158

5.2. Qué sostiene la evidencia y qué no, por componente. 164

A.1. Elementos del panel de monitoreo 181

A.2. Situaciones frecuentes y acción recomendada 184

B.1. Clases de tipo de vehículo y criterio de asignación 187

B.2. Composición de los conjuntos construidos 191

C.1. Entorno de entrenamiento y de operación 193

C.2. Artefactos y contenido 200

LISTA DE ABREVIATURAS

Abreviatura	Significado
AP	Average Precision (precisión media)
API	Application Programming Interface
CER	Character Error Rate
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DFL	Distribution Focal Loss
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IDSW	Identity Switch (cambio de identidad)
INE	Instituto Nacional de Estadística
IoU	Intersection over Union
mAP	Mean Average Precision
MOT	Multiple Object Tracking
MOTA	Multiple Object Tracking Accuracy
MOTP	Multiple Object Tracking Precision
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
REST	Representational State Transfer
RF	Requerimiento funcional
RNF	Requerimiento no funcional
RTSP	Real Time Streaming Protocol
RUAT	Registro Único para la Administración Tributaria Municipal
TFA	Tamaño funcional ajustado
UUID	Universally Unique Identifier
YOLO	You Only Look Once

INTRODUCCIÓN

Los parqueaderos de vehículos en zonas urbanas son utilizados por la sociedad desde hace muchos años, la creciente demanda de sistemas eficientes para la gestión de parqueaderos en zonas urbanas ha impulsado el desarrollo de tecnologías avanzadas que optimicen el uso de los espacios y mejoren la seguridad.

Actualmente, los métodos tradicionales, como las barreras automáticas o las cadenas de seguridad, presentan limitaciones en términos de eficiencia, fiabilidad y personalización. Ante esta problemática, surge la necesidad de implementar soluciones innovadoras que integren tecnologías emergentes en los últimos años como la visión artificial y el aprendizaje profundo.

"El procesamiento de imágenes juega un papel vital en el reconocimiento de objetos y sus características [1]".

"El seguimiento de vehículos con múltiples cámaras (MTMC) es esencial para el análisis de tráfico y el mantenimiento de la seguridad [2]", y junto con la inteligencia artificial, permite una toma de decisiones más eficientes para favorecer la gestión del establecimiento. Es decir, la automatización de la aplicación consigue grandes beneficios, como en el ahorro de ciertos recursos económicos, cantidad y calidad de los datos recolectados y permite una toma de decisiones efectiva.

Con este trabajo, se contribuye al desarrollo de tecnologías inteligentes para la gestión de infraestructura urbana, brindando soluciones eficientes, seguras y fáciles de implementar en el control de acceso vehicular.

ANTECEDENTES

El crecimiento acelerado del parque automotor y la urbanización a nivel mundial ha generado un aumento en la demanda de infraestructura idónea para el estacionamiento de vehículos. Este fenómeno ha impulsado a diversas ciudades y países a explorar soluciones tecnológicas [3] que permita gestionar de manera más eficiente los parqueaderos, especialmente en zonas urbanas de mayor crecimiento. En este sentido, los avances en visión artificial y aprendizaje profundo se han posicionado como herramientas clave para automatizar y opti-

mizar estos procesos.

En Bolivia, la gestión de los parqueaderos ha sido un reto en crecimiento, especialmente en la red troncal como La Paz, Cochabamba y Santa Cruz, donde la expansión urbana ha generado una mayor demanda de espacios de estacionamiento. Sin embargo, en comparación con otros países de la región, Bolivia ha experimentado un desarrollo más limitado en cuanto a la implementación de tecnologías avanzadas en este ámbito. A nivel local, los sistemas de acceso y control vehicular siguen siendo mayormente manuales o basados en tecnologías rudimentarios, como la intervención humana.

En los últimos años, algunas empresas en el país han empezado a adoptar tecnologías de reconocimiento de placas y cámaras de seguridad. Sin embargo, se observa una limitada adopción documentada en el uso de aprendizaje profundo para este propósito, lo que subraya la oportunidad de desarrollo e innovación en este campo de la inteligencia artificial.

A nivel global, los sistemas inteligentes de control vehicular basados en visión artificial y aprendizaje profundo han alcanzado un notable grado de sofisticación, particularmente en países de Europa, Asia, y Norteamérica. China y los Estados Unidos lideran la implementación de estas tecnologías, para gestionar grandes volúmenes de tráfico y mejorar la seguridad.

TRABAJOS DESARROLLADOS

ANÁLISIS, DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE CONTROL DE INGRESO DE VEHÍCULOS BASADO EN VISIÓN ARTIFICIAL Y RECONOCIMIENTO DE PLACAS EN EL PARQUEADERO DE LA UNIVERSIDAD POLITÉCNICA SALESIANA - SEDE CUENCA

Desarrollado por Alvares Duran Mayra Alejandra[4], el presente trabajo consiste en un sistema de reconocimiento de placas vehiculares que utiliza algoritmos de Reconocimiento Óptico de Caracteres ROC o OCR, mediante las librerías OpenCV, está desarrollado en lenguaje JAVA y se encuentra basado en Inteligencia Artificial, esta herramienta recibe y procesa las imágenes desde una cámara de video que se encuentra al ingresar al parqueadero de la Universidad Politécnica Salesiana, sede Cuenca.

DISEÑO E IMPLEMENTACIÓN DE UNA METODOLOGÍA BASADA EN INTELIGENCIA ARTIFICIAL CAPAZ DE CONTAR Y CLASIFICAR

VEHÍCULOS EN LOS ACCESO DE AEROPUERTOS Y DE RECONOCER VEHÍCULOS AEROPORTUARIOS

Realizado por Martines Samper Aniceto José [5] en la Universidad Politécnica de Valencia (UPV), este proyecto se basa en realizar un conteo y una clasificación automática de vehículos, capaz de acoplarse a cualquier video de circulación. Particularizando un poco más, este proyecto se va a centrar principalmente en la composición de un algoritmo para el conteo y clasificación de vehículos en las entradas de aeropuertos. Para efectuar este estudio, se ha trabajado con el lenguaje de programación OpenCV. En este proceso, el programa irá evolucionando el programa hasta poder conseguir un resultado óptimo, partiendo de una programación más simple, pasando por métodos básicos de procesamiento de imagen y finalizando en la implementación de redes neuronales (YOLO) para la detección y clasificación de objetos.

SISTEMA INTELIGENTE DE GESTIÓN DE TRÁFICO

González Caballero Javier [6], de la universidad de Valladolid En este proyecto se propone un sistema alternativo capaz de modificar en tiempo real la duración en verde o en rojo de los semáforos según el flujo de vehículos y la presencia o no de peatones. Para ello, se analiza la tecnología de control de los semáforos que se emplea actualmente y las investigaciones realizadas.

INFORMACIÓN DOCUMENTAL Y ESTADÍSTICA

La información documental y estadística que sostiene el problema proviene de dos fuentes verificables. La primera es el registro oficial del parque automotor que publica el Instituto Nacional de Estadística (INE) a partir de los datos del Registro Único para la Administración Tributaria Municipal (RUAT) [7]. La segunda es la campaña de grabación realizada en el recinto de estudio, cuyos metadatos describen la cobertura temporal del material analizado en este trabajo.

Parque automotor. El parque automotor registrado en Bolivia creció de 2,470,622 vehículos en 2023 a 2,583,319 en 2024 y 2,672,176 en 2025, con variaciones anuales de 4.6 % y 3.4 % respectivamente. El departamento de Oruro siguió la misma tendencia: 120,400 vehículos en 2023, 124,008 en 2024 y 127,285 en 2025 (Tabla 0.2). El crecimiento sostenido del parque vehicular aumenta la presión sobre los puntos de acceso de los parqueaderos: un registro ma-

nual de placas atiende un vehículo por vez, de modo que cada unidad adicional se traduce en espera en la barrera y en cola sobre la vía pública.

Cuadro 0.2: Parque automotor registrado en Bolivia y en el departamento de Oruro. Fuente: INE, con base en el RUAT [7].

Año	**Bolivia**	**Oruro**	**Variación anual en Oruro**
2023	2,470,622	120,400	—
2024	2,583,319	124,008	+3.0 %
2025	2,672,176	127,285	+2.6 %

Proyección. Con tres observaciones anuales no se puede estimar un modelo con intervalo de predicción defendible, pero sí una proyección mecánica que acote el orden de magnitud. Aplicando la tasa de crecimiento anual compuesta del departamento entre 2023 y 2025 (2.82 %) se obtienen 146,270 vehículos en 2030; aplicando la tasa nacional del mismo período (4.0 %) se obtienen 154,855. Una extrapolación lineal sobre los mismos datos arroja 144,498, dentro del mismo orden. La Tabla 0.3 resume los dos escenarios.

Cuadro 0.3: Proyección del parque automotor de Oruro bajo dos escenarios de crecimiento. Elaboración propia a partir de la serie del INE [7]. Es una extrapolación mecánica, no un pronóstico.

Año	**Escenario base** (2.82 %)	**Escenario alto** (4.0 %)
2026	130,874	132,375
2027	134,564	137,669
2028	138,358	143,174
2029	142,259	148,900
2030	146,270	154,855

Se trata de una proyección, no de un pronóstico: supone que la tasa reciente se mantiene y no modela saturación del parque, cambios normativos ni crisis económicas. Se incluye porque el orden de magnitud —entre 144,000 y 155,000 vehículos en 2030— es lo que importa para dimensionar un punto de acceso, no porque permita predecir el valor exacto. La cifra observada sigue siendo la que sostiene el problema; la proyección solo muestra hacia dónde crece.

Campaña de grabación. El material del estudio se recolectó en el recinto entre

el 16 de marzo y el 16 de mayo de 2026, con las cámaras de los accesos del recinto. La campaña comprende 276 sesiones de una hora, una por hora grabada, distribuidas en 39 días. La Figura 0.1 muestra las horas registradas por día de la semana: la cobertura es pareja de lunes a sábado (entre 37 y 52 horas) y baja el domingo (8 horas). Esa distribución condiciona el alcance de los resultados: las conclusiones sobre el domingo se limitan a la evidencia disponible para ese día, mientras que el resto de la semana cuenta con material suficiente para las mediciones que se reportan en el Capítulo III. Ese capítulo detalla qué subconjunto de este material se destinó a cada medición y qué horas quedaron fuera de cada conjunto de prueba.

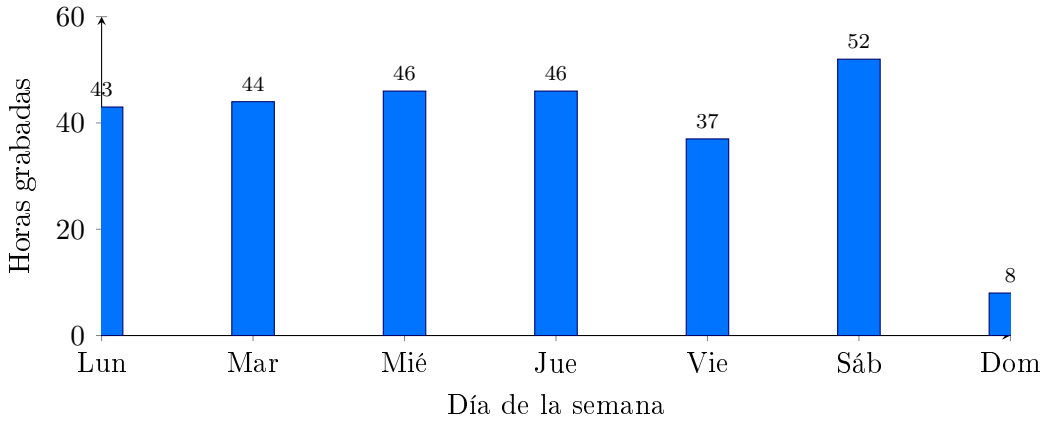


Figura 0.1: Horas de grabación por día de la semana en el recinto de estudio, entre el 16 de marzo y el 16 de mayo de 2026. Fuente: metadatos de las sesiones registradas por las cámaras del recinto.

SITUACIÓN PROBLEMÁTICA

En los parqueaderos urbanos, el control de ingreso y salida de vehículos se realiza mediante procedimientos manuales o semiautomáticos que requieren la intervención de operadores para registrar y verificar la información correspondiente. Esta práctica genera tiempos de atención variables, errores en el registro y dificultades para disponer de información confiable sobre los vehículos que ingresan y salen del recinto.

En el parqueadero, la ausencia de un mecanismo automatizado para identificar las características de los vehículos limita el registro sistemático de información como el tipo de vehículo, el color predominante y la placa de matrícula. Asimismo, la falta de un registro asociado a la fecha, hora, punto de acceso y sentido del evento dificulta la trazabilidad de los movimientos vehiculares y la generación posterior de información estadística.

A lo anterior se suma una limitación en el control de acceso: la identificación de los vehículos y la verificación de su autorización no se realizan de manera automatizada, lo que dificulta la detección oportuna de vehículos no autorizados y la disponibilidad de evidencia digital auditable.

A esto se suma que las condiciones de captura de las cámaras –variaciones de iluminación, distancia, ángulo de observación, oclusiones parciales y visibilidad de las placas– pueden afectar la identificación automática de los vehículos. Estas condiciones reales del entorno deben ser consideradas durante la construcción y evaluación de los modelos de visión artificial.

Ante esta situación, se plantea el desarrollo de un sistema basado en visión artificial y aprendizaje profundo que automatice la identificación de los vehículos en los puntos de acceso, reconozca su tipo, color predominante y placa de matrícula, registre los eventos de ingreso y salida, y almacene la información generada para su consulta y análisis.

PLANTEAMIENTO DEL PROBLEMA

En el control de acceso vehicular de los parqueaderos, los procedimientos manuales o semiautomáticos pueden generar errores de registro, dificultades para ejecutar el seguimiento de los vehículos y limitaciones para disponer de información confiable sobre los eventos de ingreso y salida. La ausencia de mecanismos automatizados para identificar las características de los vehículos también dificulta la revisión de los accesos y la generación de información histórica para el análisis de la operación del recinto.

En el parqueadero, esta problemática se relaciona con la necesidad de identificar automáticamente los vehículos considerando su tipo, color predominante y placa de matrícula, registrar la fecha, hora, cámara y sentido correspondiente al evento, y disponer de esta información para consulta, seguimiento y análisis estadístico.

En este contexto, el desarrollo de técnicas de visión artificial y aprendizaje profundo ofrece la posibilidad de automatizar parte del proceso de identificación vehicular mediante el procesamiento de imágenes y video. Sin embargo, el desempeño del sistema depende de factores como las condiciones de iluminación, el ángulo y distancia de captura, las características de las placas y la calidad de las imágenes obtenidas en el entorno de estudio.

Por tanto, el problema de investigación se centra en determinar en qué medida un sistema basado en visión artificial y aprendizaje profundo puede automatizar la identificación de vehículos y el registro de sus eventos de ingreso y salida, manteniendo un desempeño adecuado bajo las condiciones de captura del parqueadero.

En este contexto, se formula la siguiente pregunta de investigación:

> ¿En qué medida un sistema basado en visión artificial y aprendizaje profundo puede automatizar la identificación de los vehículos, mediante el reconocimiento de su tipo, color predominante y placa de matrícula, así como el registro de sus eventos de ingreso y salida, bajo las condiciones de captura del parqueadero?

OBJETO DE ESTUDIO

El proceso de control de acceso e identificación vehicular en parqueaderos urbanos.

OBJETIVOS

OBJETIVO GENERAL

Desarrollar un sistema inteligente para la gestión automatizada del control de acceso vehicular en un parqueadero, mediante técnicas de visión artificial y aprendizaje profundo, que permita identificar el tipo, color predominante y placas de los vehículos, registrar los eventos de ingreso y salida y generar información para el seguimiento y análisis de la operación del recinto.

OBJETIVOS ESPECÍFICOS

- Analizar las condiciones de operaciones del parqueadero, las características de los puntos de acceso y los requerimientos técnicos y funcionales del sistema, considerando las

condiciones de captura de las cámaras, para establecer los criterios de diseño de la solución propuesta.

- Construir y preparar un conjunto de datos representativos de las condiciones reales del parqueadero, mediante la selección, extracción, organización y anotación de imágenes de vehículos, considerando sus categorías, color predominante y placas, para el entrenamiento y evaluación de los modelos de visión artificial.
- Adaptar y entrenar modelos de aprendizaje profundo para la detección e identificación del tipo y color predominante de los vehículos, así como para la localización de sus placas, utilizando el conjunto de datos construido y evaluando su desempeño mediante métricas apropiadas.
- Desarrollar y entrenar un modelo de reconocimiento óptico de caracteres orientado a la lectura de placas vehiculares, empleando datos correspondientes al formato de las matrículas presentes en el contexto del estudio, y evaluar su desempeño bajo diferentes condiciones de visibilidad.
- Integrar los modelos de visión artificial en un sistema de gestión que permita registrar los vehículos identificados y sus eventos de ingreso y salida, asociando la información obtenida con la fecha, hora y cámara correspondiente, y proporcionando mecanismos para su consulta y visualización.
- Evaluar el desempeño del sistema desarrollado mediante pruebas experimentales en condiciones representativas del entorno de estudio, considerando la precisión de la detección e identificación de vehículos y placas, la correcta determinación de los eventos de ingreso y salida y el rendimiento computacional del modelo antes y después de aplicar técnicas de optimización.

CAMPO DE ACCIÓN

La automatización del proceso de identificación y registro de vehículos en los puntos de acceso de un parqueadero, mediante técnicas de visión computacional y aprendizaje profundo, considerando el reconocimiento de su tipo, color predominante y placa de matrícula, así como el registro de los eventos de ingreso y salida.

HIPÓTESIS

Hipótesis general (H_1):

El sistema de gestión inteligente de parqueaderos basado en visión artificial y aprendizaje profundo alcanza los criterios de desempeño establecidos para la detección e identificación de vehículos, el reconocimiento de placas de matrícula y la determinación de los eventos de ingreso y salida, bajo las condiciones de captura del entorno de estudio.

Hipótesis nula (H_0):

El sistema de gestión inteligente de parqueaderos basado en visión artificial y aprendizaje profundo no alcanza los criterios de desempeño establecidos para la detección e identificación de vehículos, el reconocimiento de placas de matrícula y la determinación de los eventos de ingreso y salida, bajo las condiciones de captura del entorno de estudio.

$H_{1.1}$: El modelo de detección de vehículos alcanza un mAP@50 igual o superior al 80 % bajo las condiciones de captura del entorno de estudio.

$H_{0.1}$: El modelo de detección de vehículos no alcanza un mAP@50 del 80 % bajo las condiciones de captura del entorno de estudio.

$H_{1.2}$: El modelo de identificación del tipo de vehículo alcanza un F1-score macro igual o superior al 85 % bajo las condiciones de captura del entorno de estudio.

$H_{0.2}$: El modelo de identificación del tipo de vehículo no alcanza un F1-score macro del 85 % bajo las condiciones de captura del entorno de estudio.

$H_{1.3}$: El modelo de identificación del color predominante alcanza un F1-score macro igual o superior al 85 % bajo las condiciones de captura del entorno de estudio.

$H_{0.3}$: El modelo de identificación del color predominante no alcanza un F1-score macro del 85 % bajo las condiciones de captura del entorno de estudio.

$H_{1.4}$: El sistema de reconocimiento de placas alcanza simultáneamente un Recall igual o superior al 90 % en la detección de las placas y una exactitud por carácter igual o superior al 90 % en el reconocimiento de las matrículas, bajo las condiciones de visibilidad del entorno

de estudio.

$H_{0.4}$: El sistema de reconocimiento de placas no alcanza simultáneamente los valores mínimos establecidos para la detección y el reconocimiento de las matrículas bajo las condiciones de visibilidad del entorno de estudio.

$H_{1.5}$: El sistema determina correctamente los eventos de ingreso y salida de los vehículos con una exactitud igual o superior al 90 % bajo las condiciones de operación del parqueadero.

$H_{0.5}$: El sistema no alcanza una exactitud del 90 % en la determinación de los eventos de ingreso y salida bajo las condiciones de operación del parqueadero.

CRITERIO DE EVALUACIÓN DE LA HIPÓTESIS

La contrastación de la hipótesis se realizó mediante pruebas experimentales sobre un conjunto de datos de prueba independiente de los datos utilizados para el entrenamiento de los modelos. El conjunto de prueba contiene muestras representativas de las condiciones de captura del parqueadero, incluyendo variaciones de iluminación, visibilidad, distancia, ángulo de captura y oclusiones presentes en el entorno de estudio.

La evaluación se realizó de forma independiente para cada componente del sistema. En todo el documento se emplean *precisión* (*precision*), *exhaustividad* (*recall*) y *exactitud* (*accuracy*); cuando una métrica se cita con su nombre en inglés, corresponde al mismo indicador. Los criterios aplicados son los siguientes:

- **Detección de vehículos:** se utilizó como indicador principal el mAP@50, considerando como criterio mínimo de cumplimiento un valor igual o superior al 80 %. Adicionalmente, se reportan Precisión, Recall y mAP@50–95 para caracterizar el comportamiento del detector.
- **Identificación del tipo de vehículo:** se utilizó como indicador principal el F1-score macro, considerando como criterio mínimo un valor igual o superior al 85 %. Se complementa con Accuracy, Precision, Recall y matriz de confusión por clase.
- **Identificación del color predominante:** se utilizó como indicador principal el F1-score macro, considerando como criterio mínimo un valor igual o superior al 85 %. Se

complementa con Accuracy y matriz de confusión por clase.

- **Detección de placas de matrícula:** se utilizó como indicador principal el Recall, considerando como criterio mínimo un valor igual o superior al 90 %. También se reportan Precision, IoU y mAP para evaluar la calidad de localización de las placas.
- **Reconocimiento óptico de caracteres:** se utilizó como indicador principal la exactitud a nivel de carácter, considerando como criterio mínimo un valor igual o superior al 90 %. Asimismo, se calculó el Character Error Rate (CER), cuyo valor debía ser igual o inferior al 10 %, y la proporción de recortes de placa reconocidos en su totalidad mediante Exact Match, considerando como referencia un valor igual o superior al 80 %.
- **Determinación de eventos de ingreso y salida:** se utilizó como indicador principal la Accuracy, considerando como criterio mínimo un valor igual o superior al 90 %. Se reportan adicionalmente Precision, Recall y F1-score.
- **Rendimiento computacional:** se evaluó mediante la latencia promedio por evento, FPS, tiempo de inferencia y consumo de recursos. Como criterio operacional de referencia se consideró una latencia promedio igual o inferior a un segundo por evento, bajo las condiciones y hardware definidos para las pruebas.

La hipótesis general se considera respaldada cuando el sistema desarrollado cumple simultáneamente los criterios mínimos establecidos para el conjunto de componentes del proceso de identificación y registro vehicular y demuestra que puede ejecutar de forma automatizada las funciones definidas dentro del alcance del proyecto. Por su forma conjuntiva, basta que un solo criterio no se cumpla para que la hipótesis no se respalde.

La hipótesis nula se considera respaldada cuando el sistema no alcanza el conjunto de criterios mínimos establecidos o cuando las deficiencias observadas impiden realizar de manera confiable el proceso automatizado de identificación y registro vehicular bajo las condiciones evaluadas. Al ser la negación de la hipótesis general, no requiere que fallen todos los términos, sino que el conjunto deje de cumplirse.

La regla anterior se operacionaliza como una prueba unilateral de intervalo. Para cada criterio con umbral θ se contrasta $H_0 : \pi \geq \theta$ frente a $H_1 : \pi < \theta$ con un nivel de significancia $\alpha = 0.05$. La hipótesis específica se respalda cuando el límite inferior del intervalo de confianza del 95 % supera θ; se rechaza —y se acepta la hipótesis nula en el sentido de la decisión— cuando el límite superior queda por debajo de θ; y el resultado se declara inconcluso cuando el intervalo contiene θ. La coincidencia exacta $\geq 80\,\%$ se declara en este criterio como referencia

de aceptación y no se incorpora con posterioridad a la medición. Aceptar la hipótesis nula no prueba que el sistema no pueda alcanzar el umbral en toda condición: es la decisión de que, bajo las condiciones evaluadas y con el riesgo declarado, el desempeño queda por debajo [8], [9], [10], [11].

Los resultados se analizan además de manera desagregada por tipo de vehículo, condición de iluminación y nivel de visibilidad, con el propósito de identificar las condiciones en las que el sistema presenta mayores tasas de error.

JUSTIFICACIÓN

JUSTIFICACIÓN TÉCNICA

El desarrollo del sistema se justifica técnicamente por la necesidad de automatizar el proceso de identificación y registro de vehículos mediante técnicas de visión artificial y aprendizaje profundo. La solución desarrollada procesa imágenes obtenidas mediante cámaras ubicadas en los puntos de acceso del parqueadero para detectar los vehículos, identificar su tipo y color predominante y localizar sus placas de matrícula para su posterior reconocimiento.

La utilización de modelos de aprendizaje profundo permitió adaptar la solución a las características visuales del entorno de estudio mediante datos obtenidos del propio parqueadero. Asimismo, el procesamiento de imágenes permitió realizar las etapas necesarias para mejorar la información utilizada por los modelos y por el sistema de reconocimiento de placas.

La integración de los modelos con un sistema de gestión permitió asociar la información visual obtenida con los eventos de ingreso y salida, generando registros estructurados que puedan ser consultados posteriormente. De esta manera, la solución no se limita al reconocimiento de vehículos, sino que integra percepción visual, procesamiento, almacenamiento y gestión de información.

JUSTIFICACIÓN ECONÓMICA

El proyecto presenta una justificación económica relacionada con la automatización de tareas asociadas al registro y control de acceso vehicular, lo que puede reducir la necesidad de intervención manual en determinadas actividades y favorecer un uso más eficiente de la información generada por el parqueadero.

Asimismo, la reutilización de infraestructura de captura existente, cuando sus características técnicas sean adecuadas para el sistema, puede contribuir a disminuir los requerimientos adicionales de equipamiento. La evaluación del sistema permitirá determinar el comportamiento del modelo y sus requerimientos computacionales, proporcionando información para analizar su viabilidad de implementación en equipos con diferentes capacidades de procesamiento.

JUSTIFICACIÓN SOCIAL

El proyecto se justifica socialmente por su contribución al mejoramiento del control y la trazabilidad de los vehículos que ingresan y salen del parqueadero. El registro automatizado de información como la placa de matrícula, el tipo, el color predominante, la fecha, la hora y el punto de acceso permitirá disponer de evidencia digital asociada a los eventos vehiculares.

La disponibilidad de esta información puede facilitar la consulta ante incidentes y aportar evidencia para el control de acceso, en tanto deja registro de cada vehículo identificado con su fecha, hora y punto de acceso. De esta manera, la propuesta busca contribuir a un proceso de control vehicular más organizado, trazable y basado en información.

ALCANCES Y LIMITACIONES

ALCANCES

El sistema automatizado para el parqueadero presenta los siguientes alcances

- **Captura y procesamiento de imágenes:** utilización de las cámaras disponibles en

los puntos de acceso seleccionados para obtener imágenes y video de los vehículos.

- **Detección e identificación vehicular:** detección de vehículos mediante modelos de aprendizaje profundo y reconocimiento de su tipo y color predominante.
- **Escalabilidad del sistema:** el diseño prevé la incorporación de nuevos puntos de acceso sin modificar el núcleo del subsistema de visión. El comportamiento bajo carga concurrente de múltiples cámaras no forma parte de la evaluación experimental de este trabajo.
- **Reconocimiento de placas:** localización de las placas de matrícula y desarrollo de un mecanismo de reconocimiento óptico de caracteres orientado al formato de las matrículas presentes en el contexto de estudio.
- **Determinación de eventos:** registro de los eventos de ingreso y salida asociados a los vehículos, considerando fecha, hora y cámara correspondiente.
- **Gestión de ocupación por aforo perimetral:** cálculo del nivel de ocupación del recinto y disponibilidad global en tiempo real a partir del balance continuo entre los eventos de ingreso y salida registrados en los puntos de acceso, en contraste con la capacidad nominal configurada.
- **Gestión de información:** almacenamiento estructurado de los datos obtenidos y disponibilidad de mecanismos de consulta y visualización mediante una aplicación web.
- **Generación de estadísticas:** elaboración de información estadística basada en los registros históricos, incluyendo afluencia vehicular, distribución temporal de ingresos y salidas y tiempos de permanencia cuando exista información suficiente para su cálculo.
- **Optimización del modelo:** evaluación de técnicas de optimización y cuantización con el propósito de analizar su impacto sobre el rendimiento computacional del sistema.
- **Evaluación experimental:** pruebas del sistema bajo condiciones representativas del entorno de estudio y comparación de su desempeño en las diferentes etapas del procesamiento.

LIMITACIONES

- **Condiciones de captura:** el desempeño del sistema dependerá de la calidad de las imágenes obtenidas por las cámaras, pudiendo verse afectado por variaciones de ilumi-

nación, distancia, ángulo de observación, movimiento, oclusiones y condiciones de baja visibilidad.

- **Reconocimiento de placas:** el reconocimiento óptico de caracteres podrá presentar errores cuando la placa no sea suficientemente visible, presente obstrucción, reflejos, desenfoque o una resolución insuficiente.
- **Representatividad del conjunto de datos:** el desempeño obtenido estará condicionado por la diversidad y calidad de las imágenes utilizadas para el entrenamiento y evaluación de los modelos.
- **Control de acceso físico:** el sistema contrasta automáticamente la matrícula reconocida contra el padrón institucional de vehículos habilitados y clasifica el acceso emitiendo las alertas correspondientes; sin embargo, el accionamiento electromecánico de barreras físicas o talanqueras no forma parte del alcance implementado, dependiendo de la supervisión del operador en caseta.
- **Capacidad computacional:** el tiempo de inferencia y el rendimiento del sistema dependerán de las características del equipo usado para ejecutar los modelos.
- **Infraestructura de red:** la comunicación entre los componentes del sistema podrá verse afectada por la disponibilidad y estabilidad de la infraestructura de red.
- **Alcance de las cámaras y delimitación de aforo:** el sistema de visión artificial se concentra en los puntos perimetrales de acceso (ingreso y egreso). La determinación de la disponibilidad se realiza a nivel de aforo general del recinto; el monitoreo, seguimiento o guiado hacia plazas o cajones de estacionamiento individuales (*bay monitoring* o *parking guidance*) queda explícitamente fuera del alcance de este trabajo por requerir una red de cámaras cenitales internas o sensores IoT por cajón.
- **Privacidad y tratamiento de información:** el almacenamiento y procesamiento de placas y registros asociados deberá considerar las disposiciones aplicables al tratamiento de datos personales y la información generada durante la implementación.

APORTES

APORTE TEÓRICO

El aporte teórico del proyecto consiste en la sistematización de fundamentos relacionados con visión computacional, aprendizaje profundo, detección de objetos, reconocimiento de matrículas y procesamiento de información vehicular, orientados a su aplicación en un sistema de control de acceso de un parqueadero. Asimismo, se establecerán criterios técnicos para la evaluación del desempeño de los modelos en función de las condiciones del entorno de estudio.

APORTE PRÁCTICO

El aporte práctico consiste en el desarrollo de un pipeline integrado para la detección e identificación de vehículos, localización y reconocimiento de placas mediante OCR, registro de eventos de ingreso y salida y gestión de la información generada por el sistema. El proyecto contempla además la construcción de un conjunto de datos propio obtenido del entorno de estudio para el entrenamiento y evaluación de los modelos.

APORTE ACADÉMICO

El aporte académico consiste en la generación de una experiencia documentada de aplicación de visión computacional y aprendizaje profundo al control de acceso vehicular, incluyendo la construcción de datos, entrenamiento, evaluación e integración de modelos en un sistema informático. La documentación obtenida podrá servir como referencia para futuros trabajos relacionados con inteligencia artificial aplicada en la Carrera de Ingeniería Informática.

INGENIERÍA DEL PROYECTO

Cuadro 0.4: Ingeniería del Proyecto

OBJETIVOS ESPECÍFICOS	MÉTODOS, TÉCNICAS Y/O HERRAMIENTAS	ACTIVIDAD
Analizar las condiciones de operación del parqueadero, las características de los puntos de acceso y los requerimientos técnicos y funcionales del sistema.	Observación directa, inspección técnica, revisión documental, análisis de imágenes y videos, entrevistas cuando corresponda.	Analizar ubicación y características de las cámaras; identificar condiciones de iluminación, distancia y ángulo de captura; identificar tipos de vehículos presentes; determinar requerimientos funcionales y técnicos; establecer condiciones y restricciones del sistema.
Construir y preparar un conjunto de datos representativo de las condiciones reales del parqueadero.	Recolección de imágenes y videos, extracción de fotogramas, selección y depuración de datos, anotación y organización del conjunto de datos, OpenCV.	Seleccionar material audiovisual; extraer imágenes; eliminar redundancias y datos inadecuados; definir clases; realizar anotaciones; organizar los conjuntos de entrenamiento, validación y prueba; documentar las características del dataset.
Adaptar y entrenar modelos de aprendizaje profundo para la detección e identificación de vehículos y localización de placas.	YOLO26, PyTorch, aprendizaje por transferencia, entrenamiento supervisado, ajuste de hiperparámetros y evaluación mediante métricas de detección y clasificación.	Preparar los datos; seleccionar el modelo; hacer entrenamiento y fine-tuning; experimentar con configuraciones; evaluar precisión, recall, F1, IoU y mAP según corresponda; seleccionar el modelo con mejor relación entre desempeño y costo computacional.

Continuación de la Tabla 0.4

OBJETIVOS ESPECÍFICOS	MÉTODOS, TÉCNICAS Y/O HERRAMIENTAS	ACTIVIDAD
Desarrollar y entrenar un modelo de reconocimiento óptico de caracteres orientado a placas de matrícula.	OpenCV, PyTorch, técnicas de preprocesamiento y OCR, aprendizaje supervisado y métricas específicas para reconocimiento de caracteres.	Extraer y preparar recortes de placas; normalizar y preprocesar las imágenes; construir el conjunto de datos para OCR; entrenar el modelo; evaluar caracteres y placas completas; analizar errores según condiciones de visibilidad.
Integrar los modelos en un sistema de gestión para el registro y consulta de eventos vehiculares.	OpenCV, Python, FastAPI, React y PostgreSQL.	Diseñar la arquitectura del sistema; implementar el procesamiento de video; integrar los modelos; desarrollar la API; diseñar la base de datos; implementar el registro de vehículos y eventos; implementar consultas, control de acceso y visualización de información.
Evaluar el desempeño del sistema y el impacto de técnicas de optimización.	Diseño experimental, pruebas de campo, métricas de desempeño, medición de latencia y FPS, evaluación comparativa y técnicas de cuantización/optimización.	Definir escenarios de prueba; ejecutar pruebas; medir desempeño de detección, identificación, OCR y eventos; medir latencia, FPS y consumo de recursos; aplicar optimización/cuantización; comparar modelo original y optimizado; analizar resultados y limitaciones.

Fuente: Elaboración propia

ORGANIZACIÓN DEL TRABAJO

El documento se organiza en cinco capítulos y un apartado de anexos. El Capítulo I establece el marco teórico: visión artificial, aprendizaje profundo, detección de objetos, percepción y seguimiento vehicular, y las métricas de evaluación y optimización que sostienen los criterios adoptados. El Capítulo II presenta el diseño del sistema: requerimientos, arquitectura general y diseño de los subsistemas de visión y gestión, incluida la interfaz de usuario. El Capítulo III documenta la implementación: entorno tecnológico, construcción del conjunto de datos, desarrollo de los modelos, integración del sistema de gestión, optimización y despliegue, pruebas y resultados, y las interfaces construidas. El Capítulo IV estima el costo del proyecto mediante puntos de función y equipamiento. El Capítulo V cierra con el contraste de las hipótesis y los objetivos, las conclusiones y las recomendaciones. Los anexos reúnen el manual de operación, los protocolos de construcción y evaluación del conjunto de datos, y los procedimientos de reproducibilidad.

Capítulo I

Marco Teórico

1.1. Introducción

El diseño e implementación de un sistema autónomo para el control de acceso e identificación vehicular en parqueaderos urbanos requiere una fundamentación teórica multidisciplinaria dentro de la visión por computadora y el aprendizaje profundo. Este capítulo establece las bases conceptuales, los modelos matemáticos y las evoluciones algorítmicas que sustentan el flujo de procesamiento completo del proyecto: desde la captura del fotograma digital en las barreras de ingreso hasta la determinación automatizada de los eventos de acceso.

El desarrollo temático de este marco teórico sigue una estructura jerárquica descendente (*top-down*) organizada en seis secciones que avanzan desde los fundamentos físicos de la percepción visual hasta la evaluación cuantitativa del sistema.

La primera sección examina los principios de la **visión artificial clásica y el procesamiento de imágenes**, abarcando las representaciones matriciales, los espacios cromáticos, el filtrado espacial y las transformaciones morfológicas orientadas al preprocesamiento de matrículas, así como sus limitaciones físicas ante variaciones de iluminación, sombras y oclusiones en entornos no controlados. La segunda establece los **fundamentos del aprendizaje automático y profundo**, desde el paradigma supervisado y la partición de datos hasta las redes neuronales convolucionales, el aprendizaje por transferencia y las técnicas de aumento de datos.

Sobre esa base, la tercera sección aborda la **detección de objetos**, justificando la adopción de arquitecturas de una sola etapa —como la familia YOLO— frente a las alternativas de dos etapas, y analizando los cuellos de botella computacionales que condicionan la inferencia en tiempo real. La cuarta se concentra en la **percepción vehicular y el reconocimiento de matrículas**, cubriendo la clasificación de atributos, la localización de objetos pequeños y los modelos de secuencia con alineación temporal. La quinta desarrolla el **seguimiento vehicular y la determinación de eventos**, desde la formulación del seguimiento multiobjeto hasta la interpretación de trayectorias. Finalmente, la sexta sección cierra con la **evaluación y optimización de los modelos**, definiendo las métricas de desempeño y las técnicas de compresión que habilitan su despliegue en hardware de borde.

1.2. Visión Artificial y Procesamiento de Imágenes

1.2.1. Modelo de la imagen digital y espacios de color (RGB, HSV) aplicados al entorno

Como establecen Gonzalez y Woods [12] y Szeliski [13], una imagen digital bidimensional se representa matemáticamente como una función de intensidad discreta $I(x, y)$, donde (x, y) denotan las coordenadas espaciales en un plano cartesiano y el valor de I corresponde a la amplitud o brillo en dicho punto (píxel). En imágenes multicanal a color, cada posición espacial está definida por un vector tridimensional en un espacio vectorial determinado.

El modelo de color predeterminado en los sensores de las cámaras de seguridad es el espacio RGB (*Red, Green, Blue*). Sin embargo, la representación RGB acopla fuertemente la información cromática con la intensidad luminosa, haciendo que variaciones en la luz ambiental alteren drásticamente los tres canales simultáneamente [12]. Para mitigar esta vulnerabilidad en el parqueadero, el sistema transforma las capturas al espacio de color HSV (*Hue, Saturation, Value*), el cual desacopla el tono (H) y la saturación (S) de la luminancia o brillo (V).

Esta separación es fundamental para la robustez del sistema durante la operación nocturna. Como analizan Liu y Zeng [14] y Auza Banegas et al. [15], cuando un automóvil aproxima a la barrera de acceso con los faros delanteros encendidos, se generan destellos directos y reflexiones de alta intensidad sobre la calzada y la carrocería. Al aislar el canal de luminancia V, el sistema puede aplicar umbralizaciones y análisis cromáticos sobre las

coordenadas de tono y saturación (H, S), permitiendo clasificar el color real del vehículo y localizar la placa de matrícula sin sufrir saturación ni cegueras provocadas por los haces de luz nocturnos.

1.2.2. Filtrado espacial lineal y suavizado Gaussiano

El filtrado espacial lineal consiste en la aplicación de una operación de convolución discreta bidimensional entre la matriz de la imagen $I(x, y)$ y un núcleo o filtro espacial $K(m, n)$ de dimensiones reducidas [13], [16], [17]:

$$(I * K)(x, y) = \sum_{m=-k}^{k} \sum_{n=-k}^{k} I(x - m, y - n)\, K(m, n) \tag{1.1}$$

Entre los filtros de suavizado pasabajo, el **Filtro Gaussiano** utiliza un núcleo cuyos coeficientes están definidos por una distribución Gaussiana bidimensional con desviación estándar σ. Su función principal es atenuar los componentes de alta frecuencia espacial mientras preserva las estructuras de los bordes principales de la escena [12].

En el punto de acceso al parqueadero, los sensores de video capturan continuamente ruido de alta frecuencia introducido por factores climáticos y térmicos. La presencia de textura rugosa en el asfalto mojado, el goteo de lluvia cayendo frente a la lente o el grano fotométrico generado por el sensor en condiciones de baja iluminación introducen artefactos aleatorios. La aplicación del suavizado Gaussiano atenúa estos gradientes espaciales espurios, homogeneizando las superficies antes de ejecutar algoritmos de extracción de bordes o segmentación de vehículos.

1.2.3. Morfología matemática (Top-Hat, Apertura) en el preprocesamiento de matrículas

La morfología matemática en procesamiento de imágenes grises se fundamenta en la teoría de conjuntos y opera mediante la interacción entre la imagen de entrada y un elemento estructurante B de forma y tamaño definidos [18]. Las dos operaciones morfológicas no lineales básicas son la *Erosión* (ϵ_B) y la *Dilatación* (δ_B), a partir de las cuales se construyen la

Apertura ($\gamma_B = \delta_B \circ \epsilon_B$) —que remueve pequeños brillantes— y el *Cierre* ($\phi_B = \epsilon_B \circ \delta_B$) —que rellena pequeñas grietas oscuras—.

Para el realce de estructuras delgadas con iluminación desigual, Soille [18] formula la transformación **Top-Hat** (o Sombrero de Copa):

- **White Top-Hat ($\mathcal{T}_w$):** Diferencia entre la imagen original y su apertura ($\mathcal{T}_w(I) = I - \gamma_B(I)$), aislando elementos más brillantes que su entorno local.
- **Black Top-Hat / Bottom-Hat ($\mathcal{T}_b$):** Diferencia entre el cierre y la imagen original ($\mathcal{T}_b(I) = \phi_B(I) - I$), aislando elementos más oscuros que su entorno local.

Como destacan Anagnostopoulos et al. [19] y Auza Banegas et al. [15], el estándar de matrículas vehiculares (como el boliviano) presenta caracteres alfanuméricos oscuros (azules o negros) estampados sobre un fondo blanco reflectante. Cuando la cámara captura una placa bajo sombras parciales proyectadas por el parachoques o con iluminación solar heterogénea, la intensidad del fondo varía drásticamente. La aplicación de la transformación *Black Top-Hat* aisla y resalta específicamente los trazos oscuros de las letras y números frente al fondo claro, corrigiendo el gradiente de iluminación antes de pasar el recorte al motor de Reconocimiento Óptico de Caracteres (OCR).

1.2.4. Limitaciones del procesamiento clásico ante perturbaciones ambientales (sombras, oclusión)

A pesar del rigor matemático del procesamiento digital de imágenes clásico, su aplicación directa para el control de acceso en entornos no controlados presenta severas limitaciones estructurales. Como demuestran las investigaciones de Tatulea et al. [20], Chandrasekaran et al. [21] y Lee et al. [22], los métodos deterministas (basados en umbrales estáticos de Otsu, substracción de fondo o detectores de bordes de Canny) dependen de la suposición rígida de que las condiciones ópticas de la escena permanecen constantes.

En la realidad operativa de un parqueadero al aire libre, esta premisa se rompe continuamente por perturbaciones ambientales dinámicas [23]:

1. **Sombras móviles y cambios de iluminación pasante:** El movimiento del sol a lo largo del día y la proyección de sombras de árboles, estructuras o de los propios vehículos

alteran la distribución de histogramas de los píxeles, provocando que los algoritmos de substracción de fondo identifiquen las sombras como falsos vehículos.

2. **Reflejos especulares y charcos en la calzada:** El agua acumulada en el pavimento tras una lluvia actúa como un espejo que refleja las luces de tráfico y el cielo, generando contornos ruidosos que distorsionan las máscaras de segmentación clásica.

3. **Oclusiones parciales y variaciones de perspectiva:** Marcos de placa, enganches de remolque, acumulación de barro sobre el parachoques o ángulos de captura laterales agudos deforman la geometría de los objetos. Los métodos basados en plantillas fijas (*template matching*) colapsan incapaces de adaptarse a estas deformaciones geométricas.

Estas deficiencias físicas demuestran que las reglas heurísticas ajustadas manualmente son insuficientes para garantizar la fiabilidad del sistema durante las 24 horas del día. Esta barrera tecnológica justifica de manera categórica la necesidad de adoptar modelos de **Aprendizaje Profundo** (*Deep Learning*), los cuales reemplazan los filtros diseñados a mano por representaciones jerárquicas aprendidas directamente desde los datos, logrando invariancia ante las perturbaciones del mundo real.

1.3. Fundamentos de Aprendizaje Automático y Aprendizaje Profundo

El desarrollo de sistemas modernos de gestión vehicular en estacionamientos ha migrado desde métodos deterministas basados en procesamiento de imágenes clásico hacia arquitecturas de aprendizaje profundo [24]. Esta transición permite abordar la alta variabilidad de los entornos de captura reales (como cambios de iluminación, ángulos de inclinación de la cámara y oclusiones parciales) mediante modelos capaces de extraer jerarquías de características discriminativas a partir de grandes volúmenes de datos.

1.3.1. El Paradigma del Aprendizaje Supervisado y Partición de Datos

Como señalan Russell y Norvig [25] y Hastie et al. [26], el aprendizaje supervisado constituye un paradigma donde un algoritmo deduce una función de mapeo a partir de pares de entrenamiento formados por muestras de entrada y sus correspondientes etiquetas de referencia. En el ámbito del control de acceso vehicular, cada entrada corresponde a un fotograma extraído de las cámaras instaladas en las barreras del parqueadero, mientras que las etiquetas representan el tipo de vehículo, la categoría de color o las coordenadas y contenido alfanumérico de la matrícula.

Para garantizar que el modelo adquiera capacidad predictiva sobre instancias no observadas previamente y no memorice las placas de los automóviles de entrenamiento, Mohri et al. [27] y Goodfellow et al. [28] establecen la necesidad de aplicar una partición de datos en tres conjuntos independientes:

- **Conjunto de Entrenamiento (*Training Set*):** Utilizado por el algoritmo para optimizar iterativamente los pesos internos del modelo a partir de miles de capturas de vehículos en el punto de acceso.
- **Conjunto de Validación (*Validation Set*):** Destinado a monitorear el desempeño durante las etapas de entrenamiento, ajustar hiperparámetros (como la tasa de aprendizaje o el tamaño de filtro) y seleccionar la mejor arquitectura sin sesgar la evaluación.
- **Conjunto de Prueba (*Test Set*):** Reservado estrictamente para medir la precisión final de generalización ante vehículos y matrículas nunca antes procesados por el sistema, simulando la operación real del estacionamiento.

1.3.2. Arquitectura Base: Neuronas, Propagación y Activaciones no Lineales

De acuerdo con Bishop y Bishop [29], las redes neuronales alimentadas hacia adelante (*feedforward networks*) procesan la información de manera unidireccional a través de capas compuestas por unidades computacionales (neuronas). En el nivel básico, cada neurona efectúa una combinación lineal de las señales de entrada mediante pesos y sesgos, pasando el resultado a través de una función de activación.

El propósito fundamental de incorporar funciones de activación no lineales (como ReLU o SiLU) es romper la linealidad del sistema. Sin estas transformaciones, una red de múltiples capas colapsaría matemáticamente en una simple transformación afín, siendo incapaz de resolver fronteras de decisión complejas. En la percepción vehicular, la no linealidad es la que permite al sistema proyectar las intensidades de los píxeles hacia espacios abstractos donde sea posible, por ejemplo, separar el color cromático de la carrocería de un automóvil respecto a las sombras proyectadas en el pavimento o las reflexiones del entorno.

1.3.3. Optimización de Modelos: Descenso de Gradiente y Funciones de Pérdida

El proceso de entrenamiento se formula como la minimización de una función de pérdida, la cual cuantifica la discrepancia entre las predicciones emitidas por el modelo y las anotaciones reales. Para ajustar los parámetros de la red, los algoritmos de optimización —como el Descenso de Gradiente Estocástico (SGD) o estimadores adaptativos como Adam [28], [29]— calculan el gradiente de la función de pérdida con respecto a cada peso mediante el algoritmo de retropropagación (*backpropagation*), que aplica la regla de la cadena en sentido inverso a través de la arquitectura.

En el control de acceso vehicular, el proceso de optimización persigue un equilibrio operativo:

- **Regresión de cajas delimitadoras:** Minimiza el error de localización geométrica entre la caja predicha por el detector y la posición real de la placa o del vehículo.
- **Minimización de falsos positivos:** Reduce la probabilidad de clasificar elementos del fondo (como rejas, sombras o señalética de tráfico) como vehículos autorizados o matrículas válidas.

1.3.4. Generalización: Sesgo-Varianza, Sobreajuste y Regularización

El desempeño de un modelo en producción está gobernado por el dilema entre el sesgo (incapacidad del modelo para capturar la estructura subyacente de los datos) y la varianza

(sensibilidad excesiva del modelo a las fluctuaciones del conjunto de entrenamiento). Un modelo con alto sesgo incurre en subajuste (*underfitting*), mientras que una varianza alta deriva en sobreajuste (*overfitting*).

En el contexto de la gestión de parqueaderos, el sobreajuste representa la principal causa de falla: ocurre cuando la red neuronal "memoriza" las condiciones particulares de la sesión de entrenamiento —por ejemplo, la iluminación solar directa de la mañana o la orientación fija de un grupo de vehículos locales—. Al desplegar el sistema en un entorno real dinámico, el modelo sobreajustado colapsa al enfrentar cambios de iluminación pasante (nubosidad o noche), lluvia sobre la calzada o variaciones en la trayectoria del automóvil al aproximarse a la barrera.

Para mitigar este problema y asegurar una alta capacidad de generalización, Srivastava et al. [30] y LeCun et al. [31] proponen técnicas de regularización estructurales y estocásticas:

1. **Dropout:** Desactiva aleatoriamente una fracción de neuronas durante cada iteración de entrenamiento, impidiendo la co-adaptación de características y obligando a la red a aprender representaciones redundantes y robustas [30].
2. **Penalizaciones de Norma (L_2 / *Weight Decay*):** Restringen la magnitud de los pesos ponderados, suavizando la función de decisión para que pequeñas variaciones en los píxeles de entrada no alteren drásticamente la predicción.
3. **Normalización por Lotes (*Batch Normalization*):** Estabiliza la distribución de las activaciones intermedias, acelerando la convergencia y actuando como un regularizador secundario.

1.3.5. Redes Neuronales Convolucionales (CNN) en la Percepción Visual

Como establecen LeCun et al. [31] y Szeliski [13], las Redes Neuronales Convolucionales (CNN) están optimizadas para datos con topología en rejilla bidimensional mediante tres mecanismos: campos receptivos locales, compartición de pesos y submuestreo (*pooling*). La formulación original de esta arquitectura se remonta al neocognitrón de Fukushima [32]. Estas propiedades otorgan invariancia a la traslación y reducen radicalmente la cantidad de parámetros frente a capas densas convencionales.

El procesamiento de una escena en el acceso al parqueadero se realiza mediante la extracción jerárquica de características visuales:

$$(I * K)(i, j) = \sum_{m} \sum_{n} I(i + m, j + n) K(m, n) \tag{1.2}$$

Donde I es la imagen capturada por la cámara y K es el filtro espacial.

- **Capas Convolucionales Iniciales (Bajo Nivel):** Filtros espaciales pequeños reaccionan ante gradientes de intensidad y primitivas geométricas elementales, como los bordes metálicos de la carrocería, el contorno del parachoques o las líneas de alto contraste del marco de la placa de matrícula.
- **Capas Convolucionales Intermedias (Nivel Medio):** Los mapas de características agrupan las primitivas simples en estructuras compuestas, detectando la simetría rectangular de las placas, la forma circular de las ruedas o la textura de los faros.
- **Capas Convolucionales Profundas (Alto Nivel):** Sintetizan la información espacial en representaciones semánticas globales, permitiendo al sistema clasificar la silueta completa del vehículo (sedán, camioneta, minibús o motocicleta) y distinguir la textura alfanumérica de los caracteres en la matrícula.
- **Capas de Submuestreo (*Pooling*):** Reducen la resolución espacial de los mapas de características, reteniendo las respuestas más dominantes (*Max-Pooling*) y otorgando tolerancia al sistema ante pequeñas variaciones en la posición del vehículo dentro del carril de ingreso.

1.3.6. Aprendizaje por Transferencia (Transfer Learning), Fine-Tuning y Aumento de Datos

Entrenar un detector de objetos desde cero requiere millones de imágenes anotadas y una enorme capacidad computacional. Para superar esta limitación en aplicaciones del entorno real, Pan y Yang [33] formalizan el Aprendizaje por Transferencia (*Transfer Learning*), técnica que permite transferir el conocimiento extraído de un dominio fuente masivo $\mathcal{D}_S$ (como los conjuntos de datos COCO u Objects365) hacia un dominio objetivo $\mathcal{D}_T$.

Sustentado en las demostraciones empíricas de Yosinski et al. [34], las capas iniciales de una CNN entrenada en datasets generales capturan descriptores de bajo nivel universales.

Por lo tanto, en lugar de reiniciar los pesos, se adopta una arquitectura preentrenada de vanguardia (como la familia YOLO) y se ejecuta un ajuste fino (*Fine-Tuning*), adaptando las capas superiores a las condiciones locales del proyecto:

1. **Especialización a la Morfología de Placas Locales:** Los modelos preentrenados globalmente están optimizados para formatos de matrículas estadounidenses o europeas. El *fine-tuning* reconfigura los cabezales de predicción para detectar la geometría, color de fondo, relación de aspecto y tipografía estándar de las matrículas del contexto nacional (Bolivia).
2. **Categorización de la Flota Vehicular Específica:** Permite adaptar la red para clasificar categorías de transporte particulares del entorno urbano local (como minibuses o "surubíes") que no existen de forma explícita en datasets internacionales.
3. **Calibración a la Geometría de Captura:** Adapta el modelo a la distancia focal, ángulo de inclinación y altura fija de las cámaras instaladas en la infraestructura del parqueadero.

De forma complementaria, el Aumento de Datos (*Data Augmentation*) actúa como un regularizador implícito que incrementa la variabilidad del conjunto de entrenamiento sin costo de etiquetado adicional [28], [29]. En el sistema de control de acceso, el aumento de datos simula condiciones físicas y ambientales adversas:

- **Variaciones Fotométricas (HSV y Brillo):** Simulan cambios drásticos de iluminación solar, reflejos o deslumbramientos sobre el parabrisas y condiciones de baja visibilidad nocturna.
- **Transformaciones Geométricas (Escalado, Rotación y Perspectiva):** Modulan las distintas trayectorias de ingreso de los automóviles, simulando maniobras descentradas o ángulos de aproximación no frontales.
- **Simulación de Intemperie y Oclusiones:** Inyecta artefactos sintéticos como ruido estocástico, desenfoque de movimiento y oclusiones parciales (*Mosaic* / *Mixup*) para representar gotas de lluvia sobre la lente, suciedad en la calzada o barro sobre la placa vehicular.

Mediante la integración del *Transfer Learning*, el *Fine-Tuning* y el *Data Augmentation*, el sistema alcanza la estabilidad operativa requerida para garantizar un control de acceso automatizado, preciso y resistente a las inclemencias del entorno real.

1.4. Detección de Objetos mediante Aprendizaje Profundo en Entornos Urbanos

La automatización de procesos de supervisión vehicular en infraestructura urbana requiere la localización y clasificación precisa de las instancias presentes en el flujo de video. En el control de accesos a parqueaderos, los algoritmos de detección de objetos constituyen el núcleo del sistema de percepción, siendo responsables de identificar vehículos completos (sedanes, camionetas, minibuses) y aislar subregiones de interés como las placas de matrícula.

1.4.1. Formulación de la detección: Cajas delimitadoras y Funciones de Pérdida

Como formulan Redmon et al. [35], la detección de objetos reformula la clasificación tradicional al predecir simultáneamente la categoría del objeto y un vector continuo de coordenadas espaciales que define una caja delimitadora (*bounding box*). Para cada objeto identificado en el parqueadero, la red predice una tupla $B = (x, y, w, h)$, donde (x, y) representan las coordenadas del centro de la caja ajustadas dinámicamente a la celda de la Rejilla, mientras que w y h denotan el ancho y el alto normalizados con respecto a las dimensiones totales de la imagen. Este principio permite encerrar desde la silueta completa de un vehículo ingresando al recinto hasta aislar con alta resolución espacial la región rectangular de la matrícula.

Para medir cuantitativamente el grado de solapamiento entre la caja delimitadora predicha por el modelo (B_{pred}) y la caja de referencia anotada manualmente (B_{gt}), se utiliza la métrica de Intersección sobre Unión (*Intersection over Union*, IoU) [36]:

$$\text{IoU}(B_{\text{pred}}, B_{\text{gt}}) = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} \tag{1.3}$$

Dado que la métrica IoU convencional produce gradientes nulos cuando dos cajas no se solapan (IoU = 0), las arquitecturas modernas optimizan la localización mediante funciones de pérdida geométricas continuas como Generalized IoU ($\mathcal{L}_{\text{GIoU}}$) y Complete IoU ($\mathcal{L}_{\text{CIoU}}$) [37], las cuales incorporan penalizaciones por la distancia entre centros de las cajas y la discrepancia en su relación de aspecto:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v \tag{1.4}$$

donde $\rho(b, b^{\text{gt}})$ representa la distancia euclídea entre los puntos centrales de la caja predicha y la real, c es la diagonal del contenedor convexo mínimo que abarca ambas cajas, y v mide la consistencia en la relación de aspecto geométrica de la placa o vehículo.

1.4.2. Taxonomía de detectores: Modelos de una etapa para inferencia en tiempo real

Históricamente, la literatura de visión artificial dividió los detectores profundos en dos grandes familias: los detectores de dos etapas (*two-stage*) y los de una sola etapa (*one-stage*). Como describen Ren et al. [38], los modelos de dos etapas (como Faster R-CNN) emplean una Red de Propuesta de Regiones (*Region Proposal Network*, RPN) para generar candidatos de objetos, los cuales son posteriormente recortados, vectorizados y clasificados por subredes secundarias. Aunque este enfoque ofrece un nivel elevado de precisión, la evaluación secuencial de cientos de propuestas regionales genera un cuello de botella computacional insuperable para el procesamiento de video de alta tasa de fotogramas.

En el contexto del control vehicular en parqueaderos, la latencia de inferencia es un factor crítico. Por ello, los detectores de dos etapas son descartados frente a las arquitecturas de una sola etapa (*one-stage*) como SSD [39] y la familia YOLO [35], [40]. Estos modelos evalúan la escena completa en una única pasada convolucional (*single-pass*), realizando la regresión de coordenadas y la asignación de probabilidades de clase de forma simultánea. Esta unificación del flujo de datos reduce el tiempo de procesamiento a milisegundos por fotograma, consolidando a las arquitecturas de una etapa como el estándar de facto para la supervisión vehicular en tiempo real.

1.4.3. Evolución arquitectónica de la familia YOLO

La evolución de la familia YOLO ha estado marcada por un cambio continuo en la representación de los datos y en la eficiencia computacional. Las primeras generaciones (YOLOv2 a YOLOv5) [35] introdujeron el concepto de cajas ancla (*anchor boxes*), las cuales requerían predefinir manualmente plantillas de forma y tamaño mediante algoritmos de

agrupamiento (k-means) sobre el conjunto de entrenamiento.

Sin embargo, como demuestran Tian et al. [41] y Ge et al. en YOLOX, la dependencia de anclas fijas introduce rigidez y una alta sensibilidad a hiperparámetros. La transición hacia arquitecturas libres de anclas (*anchor-free*), adoptada en YOLOv8, YOLOv10, YOLO11 y YOLO26 [42], [43], [44], eliminó las cajas de referencia predefinidas. En su lugar, el detector predice directamente desde cada punto sobre el mapa de características las cuatro distancias continuas (l, t, r, b) hacia los bordes superior, inferior, izquierdo y derecho de la entidad visual.

Esta transición hacia el enfoque *anchor-free* mejoró drásticamente la capacidad de detección multiescala dentro de la misma escena de parqueadero. Al no depender de dimensiones de ancla preajustadas, la red adapta dinámicamente sus campos receptivos, permitiendo localizar con alta precisión objetos de dimensiones dispares dentro del mismo encuadre: desde un camión de carga de gran volumen hasta una pequeña placa de matrícula instalada en el parachoques de un automóvil.

1.4.4. Cuellos de botella en el Borde (Edge AI): NMS y DFL en oclusión vehicular

El despliegue de sistemas inteligentes en puntos de acceso vehicular exige ejecutar los modelos de visión directamente en dispositivos de borde (*Edge AI*) integrados en las casetas del parqueadero (como tarjetas NVIDIA Jetson o procesadores embebidos), donde la memoria, la potencia de cálculo y la disipación térmica son limitadas [43]. En este escenario, dos componentes arquitectónicos tradicionales representan los principales cuellos de botella computacionales:

1. **Carga computacional de Distribution Focal Loss (DFL):** Módulos como DFL, derivados de la pérdida focal [45], transforman la regresión de cada borde de la caja en una distribución de probabilidad discreta distribuida en K contenedores ($K = 16$). Evaluar funciones *Softmax* y sumas ponderadas sobre miles de puntos espaciales incrementa significativamente las operaciones de punto flotante (FLOPs) y el tamaño del cabezal de detección. En dispositivos con recursos restringidos, DFL ralentiza el tiempo de respuesta y complica la exportación de modelos a formatos de ejecución cuantitizados como TensorRT o INT8 [44].
2. **Latencia estocástica de Supresión No Máxima (NMS):** El algoritmo tradicional

NMS ordena y elimina iterativamente las cajas con alto grado de solapamiento mediante un umbral rígido de IoU [46]. En accesos con tráfico denso, cuando los automóviles avanzan en fila muy cercanos o sufren oclusiones parciales, NMS presenta dos fallas críticas:

- *Latencia variable:* El tiempo de ejecución en CPU/GPU oscila impredeciblemente en función de la cantidad de vehículos detectados en la escena.
- *Falsos negativos por oclusión:* NMS corre el riesgo de suprimir erróneamente la caja delimitadora de un automóvil válido al interpretar que su alto nivel de solapamiento con la caja del vehículo delantero es un duplicado, provocando la pérdida de registros de acceso.

1.4.5. Arquitecturas modernas (NMS-Free) y Detección de Objetos Pequeños

Para eliminar los retrasos del post-procesamiento en entornos de acceso crítico, las versiones más recientes como YOLOv10, YOLOv12 y YOLO26 [40], [43], [47] introducen un diseño de cabezal dual (*dual-head*). Durante el entrenamiento, el modelo utiliza una rama uno-a-muchos (*one-to-many*) para proporcionar una supervisión densa y acelerar la convergencia, y una rama uno-a-uno (*one-to-one*) que aprende a asignar exactamente un predictor positivo por objeto. Durante la inferencia en producción, se utiliza exclusivamente la rama uno-a-uno, logrando una decodificación directa *End-to-End NMS-Free* que erradica la dependencia de NMS y garantiza una latencia constante. Además, YOLO26 elimina por completo el módulo DFL, aligerando el cabezal de regresión y reduciendo en más del 20 % los FLOPs requeridos [44].

Por otro lado, la localización de matrículas lejanas representa el reto de detectar "objetos pequeños" (*small targets*), los cuales ocupan apenas una mínima fracción de píxeles en la captura de alta resolución. En los métodos de asignación tradicionales (*Task-Aligned Learning*, TAL), las operaciones de submuestreo (*downsampling*) reducen la resolución espacial de los mapas de características profundos de tal forma que los objetos muy pequeños no logran contener ningún punto de ancla en su interior, recibiendo cero asignaciones positivas y anulando el gradiente de aprendizaje para dicha muestra [44].

Para resolver esta degradación, modelos como YOLOv13 y YOLO26 incorporan la técnica de Asignación de Etiquetas Adaptativa para Objetos Pequeños (*Small-Target-Aware*

Label Assignment, STAL) [43], [48]. STAL desacopla la geometría utilizada durante el filtrado de candidatos de la superficie real de la caja, garantizando de forma matemática la generación de muestras positivas para objetos cuya superficie sea inferior al salto del mapa de características (*stride*). Gracias a esta innovación, el detector mantiene una supervisión continua sobre las matrículas distantes, resolviendo el problema crítico de la pérdida de detección de patentes lejanas en las barreras de ingreso.

1.5. Percepción Vehicular y Reconocimiento de Matrículas

La automatización avanzada de un parqueadero inteligente requiere trascender la mera detección de presencia vehicular para implementar una capa de percepción integral. En este nivel, el sistema analiza múltiples dimensiones atributo-espaciales de los automóviles que aproximan a las barreras de acceso, combinando la clasificación cromática y tipológica del vehículo con la localización y lectura óptica de su placa de matrícula. Esta verificación multimodal no solo agiliza el flujo de paso, sino que fortalece la seguridad al correlacionar las características físicas de la carrocería con el registro de la matrícula almacenado en la base de datos del recinto.

1.5.1. Clasificación de atributos visuales en espacios de características discriminativos

Como analizan Lima y Laroca [49] y Ni y Huttunen [50], la categorización fina de vehículos (tipo de carrocería y color predominante) proporciona una capa de seguridad crítica en el control de acceso a parqueaderos. La identificación del tipo de vehículo (sedán, camioneta, minibús, motocicleta) y su color permite establecer mecanismos de auditoría cruzada: si un automóvil intenta ingresar utilizando una placa registrada a nombre de un vehículo de distinta categoría o tonalidad, el sistema detecta de inmediato la discrepancia y emite una alerta por posible clonación o intercambio de matrícula.

Desde la perspectiva del procesamiento visual, la extracción de estos atributos enfrenta desafíos ópticos significativos en el punto de acceso, como sombras proyectadas por las casetas, reflejos solares en la pintura y variaciones en las luminarias nocturnas. Para resolver estas

ambigüedades, las arquitecturas profundas modernas (como EfficientNet, ResNet o Vision Transformers) proyectan los recortes del vehículo hacia espacios de características latentes de alta dimensión. En estos espacios, la red aprende a aislar la cromaticidad intrínseca de la pintura frente a los brillos especulares, permitiendo separar con alta precisión clases no lineales en el espacio cromático y morfológico.

1.5.2. Localización de objetos pequeños con geometría específica: detección de matrículas

En el encuadre general de una cámara de supervisión instalada en la barrera de un parqueadero, la placa de matrícula ocupa únicamente una fracción diminuta del total de píxeles del fotograma (configurando el problema clásico de detección de objetos pequeños o *small targets*). Como demuestran Laroca et al. [51] y Auza Banegas et al. [15], localizar con precisión esta subregión rectangular es una etapa crítica, pues cualquier desviación geométrica en el recubrimiento de la placa degrada drásticamente la tasa de éxito del módulo subsecuente de reconocimiento óptico de caracteres (OCR).

Para superar la atenuación espacial que sufren los objetos pequeños al atravesar las capas profundas de submuestreo, los detectores modernos apoyan su arquitectura en Redes de Pirámide de Características (*Feature Pyramid Networks*, FPN) [52] y en el aprovechamiento de patrones geométricos a priori. La matrícula vehicular posee prioris estructurales altamente discriminativos, como una relación de aspecto rectangular definida (aproximadamente $3:1$ o $2:1$), bordes de alto contraste y una densidad interna de gradientes generada por los caracteres alfanuméricos. Los cabezales de detección adaptativos utilizan estas propiedades para predecir cajas delimitadoras precisas sobre la placa, aislando la Región de Interés (ROI) de forma resistente a las variaciones de distancia focal y altura de montaje de las cámaras.

1.5.3. Del OCR clásico por segmentación al modelado de secuencias

Como detallan las revisiones sistemáticas de Anagnostopoulos et al. [19] y Shashirangana et al. [53], los sistemas tradicionales de reconocimiento de matrículas (ALPR) dependían de un enfoque secuencial en tres etapas: localización de la placa, binarización/segmentación individual de cada carácter y clasificación mediante coincidencia de plantillas (*template matching*) o redes densas simples.

Sin embargo, estos métodos clásicos por segmentación colapsan frecuentemente en las condiciones operativas reales de un parqueadero al aire libre o semiconstruido:

- **Fallos por fusión de caracteres:** La presencia de suciedad, barro o sombras sobre la placa provoca que dos letras contiguas se unan en una sola máscara de binarización, imposibilitando su separación.
- **Fraccionamiento por imperfecciones:** Tornillos de sujeción, marcos metálicos o peladuras en la pintura generan falsos fragmentos que el algoritmo interpreta erróneamente como caracteres individuales.
- **Sensibilidad al umbral de binarización:** Cambios bruscos de iluminación entre el día y la noche impiden definir un umbral estático de Otsu o adaptativo que aísle limpiamente el texto sin deformarlo.

Para erradicar esta fragilidad, la visión por computadora moderna migró hacia modelos de reconocimiento de secuencias libres de segmentación (*segmentation-free*). Arquitecturas como CRNN, propuestas por Shi, Bai y Yao [54], y los modelos desarrollados por Li y Wang [55], tratan la lectura de la placa como un problema de traducción de secuencia a secuencia visual, procesando la imagen de la matrícula completa en una única pasada *end-to-end* sin necesidad de cortar letra por letra.

1.5.4. Modelos de secuencias (RNN, LSTM) y alineación mediante CTC

Como establecen Li y Wang [55], la arquitectura de un reconocedor de matrículas libre de segmentación se compone de tres etapas acopladas: un extractor convolucional (CNN), un modelador contextual recurrente (RNN/LSTM) y una capa de decodificación y transcripción. El extractor CNN transforma el recorte rectangular de la placa en una secuencia horizontal de vectores de características alineados de izquierda a derecha. Posteriormente, redes recurrentes bidireccionales con memoria de largo-corto plazo (*Bidirectional Long Short-Term Memory*, BLSTM) [56] analizan la secuencia para capturar dependencias contextuales entre los caracteres adyacentes.

El desafío matemático central en este esquema radica en que la longitud de la secuencia de características generada por la CNN no coincide en dimensión con el número de

caracteres reales impresos en la placa. Como resolvieron Graves et al. [57], el algoritmo de Clasificación Temporal Conexionista (*Connectionist Temporal Classification*, CTC) resuelve intuitivamente este problema de alineación no supervisada sin requerir anotaciones a nivel de píxel para cada letra.

Para entender la lógica del algoritmo CTC de forma intuitiva:

1. A medida que el modelo desliza su ventana de análisis a lo largo de la imagen de la placa, la red emite probabilidades de caracteres para múltiples divisiones verticales delgadas.
2. Como el ancho de un carácter abarca varias divisiones verticales, la red genera predicciones repetidas y espacios vacíos. Por ejemplo, al leer la secuencia "AB1", la salida cruda fotograma a fotograma podría resultar en la cadena emitidamente sobredimensionada: ‘‘`A-A-B--1-1`’’, donde el símbolo ‘‘`-`’’ representa una etiqueta especial de espacio en blanco (*blank*).
3. El algoritmo CTC aplica una función de mapeo de colapso condicional ($\mathcal{B}$) que opera en dos pasos continuos: primero condensa las repeticiones consecutivas de caracteres idénticos no separados por un espacio (‘‘`A-A-B--1-1`’’ $\rightarrow$ ‘‘`A-A-B-1-1`’’), y posteriormente elimina todos los símbolos vacíos (‘‘`-`’’, ‘‘`blank`’’).

Como resultado de esta condensación, la secuencia se transforma limpiamente en la cadena de texto final ‘‘`AB1`’’. Esta operación permite entrenar el modelo de OCR directamente sobre imágenes de matrículas completas etiquetadas únicamente con su texto de referencia, dotando al sistema de una robustez extrema ante ligeras deformaciones geométricas o imperfecciones en el fondo de la placa.

1.5.5. Dominio de aplicación: estructura y gramática formal de las matrículas en Bolivia

Como documentan Auza Banegas et al. [15] en el desarrollo del marco BLPR y la normativa del Registro Único para la Administración Tributaria Municipal (RUAT), el estándar de matrículas vehiculares en Bolivia (vigente desde 1997) sigue una estructura alfanumérica formalmente estandarizada. La placa convencional para automóviles se compone de tres a cuatro dígitos numéricos seguidos de tres letras mayúsculas (formatos habituales ‘‘`1234ABC`’’

o ‘‘456DEF’’), estampadas en color azul sobre un fondo blanco reflectante con la leyenda ‘‘BOLIVIA’’ en la parte superior.

Conocer a priori esta sintaxis matemática y gramatical permite implementar una capa de post-procesamiento determinista basada en reglas que corrige automáticamente posibles ambigüedades producidas por el reconocedor visual OCR:

$$\text{Regla de Corrección Sintáctica} = \begin{cases} \text{'O'} \rightarrow \text{'0'},\ \text{'I'} \rightarrow \text{'1'},\ \text{'B'} \rightarrow \text{'8'} \\ \text{si la posición } p \in \{1, 2, 3, 4\} \\ \text{'0'} \rightarrow \text{'O'},\ \text{'1'} \rightarrow \text{'I'},\ \text{'8'} \rightarrow \text{'B'} \\ \text{si la posición } p \in \{5, 6, 7\} \end{cases} \tag{1.5}$$

Por ejemplo, si la cámara captura un vehículo ingresando al parqueadero y el modelo de OCR predice la cadena ambigua ‘‘1204ABC’’, el módulo de post-procesamiento evalúa la gramática oficial de Bolivia. Al detectar que la tercera posición corresponde estrictamente al bloque numérico inicial, el sistema fuerza la conversión del carácter alfabético ambiguo ‘‘O’’ hacia el dígito numérico equivalente ‘‘0’’, corrigiendo la lectura a ‘‘1204ABC’’. De forma análoga, si el reconocedor detecta un número ‘‘1’’ en el bloque posterior reservado para letras, el filtro lo mapea al carácter alfabético ‘‘I’’. Esta validación gramatical previa a la consulta en la base de datos incrementa significativamente la tasa de coincidencia exacta (*Exact Match*) y reduce el índice de error por carácter (CER) en el recinto.

1.5.6. Factores de degradación visual en el reconocimiento de matrículas

En la operación continua de un parqueadero real, la calidad de la imagen capturada en las barreras se ve afectada por múltiples factores ambientales y mecánicos. De acuerdo con las categorizaciones de anomalías desarrolladas por Auza Banegas et al. [15] y Rio-Alvarez et al. [23], los principales factores de degradación visual se estructuran en:

1. **Distorsión por perspectiva y ángulo agudo (*Skewed Plates*):** Representa la degradación más frecuente. Debido a que las cámaras suelen montarse en postes late-

rales o esquinas superiores de las casetas, la imagen de la placa sufre una deformación perspectiva pronunciada, deformando la geometría de los caracteres alfanuméricos.

2. **Desenfoque por movimiento (*Motion Blur*):** Ocurre cuando el vehículo aproxima a la barrera a una velocidad superior a la tasa de refresco del obturador de la cámara, dispersando los bordes de los caracteres a lo largo de la trayectoria.

3. **Variaciones extremas de iluminación y brillos especulares (*Low-Illumination / Glare*):** Incluye desde la escasez de luz en horarios nocturnos hasta el deslumbramiento provocado por las luces delanteras del automóvil o el sol incidente de frente, eliminando el contraste entre el texto azul y el fondo blanco.

4. **Oclusiones parciales, suciedad y deterioro físico (*Damaged / Occluded Plates*):** Placas dobladas, cubiertas por barro en temporadas de lluvia o parcialmente tapadas por enganches de remolque o marcos protectores de plástico.

Para neutralizar estos factores sin sobrecargar la latencia del sistema, las soluciones modernas integran etapas de preprocesamiento condicional —como rectificación geométrica por homografía mediante detección de esquinas, ajuste dinámico de contraste (CLAHE/Gamma) y filtros de respuesta estocástica— antes de pasar la imagen normalizada al motor de OCR.

1.6. Seguimiento Vehicular y Determinación de Eventos

En un sistema de control de acceso para parqueaderos, la detección frame a frame de los vehículos resulta insuficiente si no se acopla a un mecanismo de continuidad temporal. El módulo de seguimiento vehicular (*Vehicle Tracking*) tiene como objetivo vincular las detecciones espaciales a lo largo del tiempo para construir la trayectoria continua de cada automóvil. Esta etapa es fundamental para transformar coordenadas crudas de píxeles en eventos operativos estructurados, tales como la autorización de paso, el conteo en tiempo real y la clasificación direccional de ingresos y salidas.

1.6.1. Formulación matemática del seguimiento de objetos múltiples (MOT)

Como sintetizan Luo et al. [58], el seguimiento de objetos múltiples (*Multiple Object Tracking*, MOT) se formula como un problema de estimación multivariable donde, dada una secuencia continua de video, se busca determinar el estado de M_t objetos presentes en cada fotograma t. En el paradigma estándar de seguimiento por detección (*Tracking-by-Detection*), el detector de objetos (como YOLO) proporciona en cada frame t un conjunto de observaciones $O_t = \{o_1^t, o_2^t, \ldots, o_{N_t}^t\}$, mientras que el sistema de seguimiento mantiene el conjunto de estados hipotéticos $S_t = \{s_1^t, s_2^t, \ldots, s_{M_t}^t\}$.

La tarea central del problema MOT reside en la asignación temporal de identidades: asignar y mantener un identificador único e invariable ($ID_k \in \mathbb{N}$) a cada vehículo individual a medida que se desplaza dentro del campo de visión de la cámara.

En la operación real de un parqueadero, la asignación de este ID único es indispensable. Si el sistema dependiera únicamente de detecciones independientes por frame a una tasa de 30 fotogramas por segundo, un automóvil detenido o avanzando lentamente hacia la barrera generaría decenas de registros aislados por segundo. Mantener una trayectoria persistente (*tracklet*) mediante un ID único garantiza que el ciclo de vida del vehículo sea tratado como una sola entidad continua desde que entra en el área de aproximación hasta que cruza la barrera, evitando la duplicación estocástica de eventos de acceso.

1.6.2. Modelado de incertidumbre y estimación del estado mediante Filtro de Kalman

Como demostró originalmente Kalman [59] y adaptaron Bewley et al. [60] para el seguimiento en tiempo real, el movimiento de una caja delimitadora puede modelarse dinámicamente mediante un Filtro de Kalman lineal bajo la suposición de velocidad constante. Para el caso de un vehículo, el vector de estado sintético s_t se define típicamente en el espacio de la imagen como:

$$s_t = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^T \tag{1.6}$$

donde (u, v) representan el centroide espacial de la caja delimitadora en píxeles, s es la escala o área total de la caja ($s = w \times h$), r es la relación de aspecto fija ($r = w/h$), y los términos $(\dot{u}, \dot{v}, \dot{s})$ representan sus respectivas velocidades lineales de cambio entre fotogramas consecutivos.

El Filtro de Kalman opera de manera recursiva en dos fases conceptuales continuas:

1. **Fase de Predicción (*Predict*):** Utilizando la ecuación del modelo cinemático previa, el filtro proyecta hacia adelante el estado estimado y su matriz de covariaza de incertidumbre para el fotograma $t + 1$, anticipando dónde debería ubicarse el vehículo antes de procesar la nueva imagen.
2. **Fase de Actualización (*Update*):** Una vez que el detector emite las observaciones reales del fotograma $t + 1$, el filtro calcula la ganancia de Kalman para ponderar la incertidumbre del sensor frente a la predicción teórica, ajustando corregidamente el estado final del vehículo.

Esta capacidad de estimación cinemática proporciona al sistema un mecanismo de "memoria temporal". En el acceso a un parqueadero, es habitual que la placa o el parachoques del vehículo sufran oclusiones parciales temporales —por ejemplo, cuando un peatón camina frente a la cámara, cuando la barrera física se levanta o ante destellos de luz solar directa—. Durante los fotogramas en los que el detector no genera ninguna observación ($o_t = \emptyset$), el Filtro de Kalman mantiene la predicción de la posición del vehículo en función de su velocidad previa, evitando que la trayectoria se rompa o que el sistema pierda la identidad del automóvil durante breves intervalos de ceguera óptica.

1.6.3. Asociación de datos y optimización combinatoria: Algoritmo Húngaro y SORT

Para enlazar las predicciones proyectadas por el Filtro de Kalman con las nuevas detecciones generadas por el modelo de visión en cada frame, Bewley et al. [60] formularon el algoritmo SORT (*Simple Online and Realtime Tracking*). SORT aborda la asociación de datos resolviendo un problema de asignación lineal global mediante el Algoritmo Húngaro (Kuhn) [61], una técnica de optimización combinatoria que minimiza el costo total de emparejar dos conjuntos disjuntos.

Dadas M trayectorias predichas por Kalman ($\hat{B}_{\text{pred}}$) y N nuevas cajas detectadas por YOLO (B_{det}), se construye una matriz de costo asociativo $C \in \mathbb{R}^{M \times N}$. La métrica elegida para medir la distancia espacial entre la predicción i y la detección j es el complemento de la Intersección sobre Unión (IoU):

$$C_{i,j} = 1 - \text{IoU}\left(\hat{B}_{\text{pred},i},\ B_{\text{det},j}\right) \tag{1.7}$$

El Algoritmo Húngaro procesa la matriz C para encontrar la asignación óptima que minimice la suma global de distancias espaciales. Para evitar emparejamientos erróneos entre vehículos lejanos, se aplica un umbral rígido de rechazo (IoU_{min}, típicamente 0.3):

- Si $C_{i,j} > 1 - \text{IoU}_{\text{min}}$, la asignación se descarta.
- Las detecciones no asociadas a ninguna trayectoria existente se inicializan como candidatos a nuevos vehículos.
- Las trayectorias que no reciben ninguna detección asociada durante un número límite de fotogramas (T_{lost}) son eliminadas definitivamente de la memoria del sistema.

Esta combinación de baja complejidad computacional permite a SORT resolver la asociación de datos en una fracción de milisegundo por frame, manteniendo el procesamiento en tiempo real.

1.6.4. Determinación de eventos a partir de trayectorias vehiculares

La etapa final del pipeline transforma las trayectorias asociadas (*Tracklets*) en eventos discretos con validez operativa para la gestión del parqueadero. Como analizan Liu y Zeng [14] y Karbalaie et al. [62], la lógica de negocio para la detección de eventos se fundamenta en la definición de líneas virtuales de control en el plano de la imagen, conocidas técnicamente como *Virtual Tripwires*.

Una línea virtual L_{tripwire} se define geométricamente como un segmento orientado en el espacio de la imagen mediante dos puntos de control $P_1(x_1, y_1)$ y $P_2(x_2, y_2)$, ubicada estratégicamente sobre la calzada frente a la barrera de acceso.

A medida que un vehículo con identidad ID_k avanza, el sistema registra la secuencia histórica de los centroides de su caja delimitadora $\mathbf{p}_t = (u_t, v_t)$ para instantes $t_1 < t_2 < \ldots < t_n$. La determinación del evento se evalúa geométricamente mediante el cruce de línea:

1. **Detección del Cruce Espacial:** Se verifica analíticamente si el segmento trazado entre la posición previa del vehículo $\mathbf{p}_{t-1}$ y su posición actual $\mathbf{p}_t$ se interseca espacialmente con el segmento de la línea virtual L_{tripwire}.
2. **Cálculo del Vector de Dirección:** Una vez confirmado el cruce, se evalúa el producto escalar o el producto cruz vectorial entre el vector de desplazamiento del vehículo $\mathbf{v}_{\text{mov}} = \mathbf{p}_t - \mathbf{p}_{t-1}$ y el vector normal unitario de la línea virtual $\mathbf{n}_L$:

$$\text{Dirección} = \text{signo}\left(\mathbf{v}_{\text{mov}} \cdot \mathbf{n}_L\right) \tag{1.8}$$

Dependiendo del signo del resultado geométrico, el sistema clasifica el evento de manera determinista:

- **Evento de Ingreso (*Entry Event*):** Si el vector de movimiento apunta hacia el interior del parqueadero (Dirección > 0), el sistema consolida el evento registrando el ID del vehículo, la lectura de la placa vinculada, la fecha, la hora exacta y habilita la apertura de la barrera.
- **Evento de Salida (*Exit Event*):** Si el vector de movimiento apunta en sentido opuesto (Dirección < 0), el sistema registra la salida, calcula el tiempo de permanencia consumido en el recinto y procede al cobro o liberación de la plaza.

Mediante esta arquitectura integrada de detección, filtrado cinemático, asociación combinatoria y análisis geométrico de trayectorias, el sistema garantiza un registro de accesos automatizado, auditable y resistente a falsas disparaciones.

1.7. Evaluación y Optimización de Modelos de Visión Artificial

La validación cuantitativa y la preparación para el despliegue en producción representan la etapa final en el desarrollo de un sistema de percepción vehicular. En el control

de acceso a estacionamientos, los modelos de aprendizaje profundo no solo deben alcanzar un alto grado de exactitud matemática, sino que deben operar bajo estrictas restricciones de latencia en dispositivos de cómputo embebidos. Esta sección examina las métricas estandarizadas para evaluar la detección, el reconocimiento secuencial y el seguimiento temporal, así como las técnicas de compresión de modelos necesarias para garantizar su rendimiento en tiempo real en el borde (*Edge AI*).

1.7.1. Métricas de clasificación y detección: Precision, Recall, F1-Score, mAP

Como establecen Sokolova y Lapalme [63] y Padilla et al. [64], la evaluación de clasificadores y detectores se fundamenta en la matriz de confusión, la cual categoriza las predicciones frente a las anotaciones de referencia (*ground truth*) en cuatro cuadrantes: Verdaderos Positivos (TP), Falsos Positivos (FP), Falsos Negativos (FN) y Verdaderos Negativos (TN).

En la operación real de un parqueadero inteligente, la interpretación de estos errores adquiere un significado crítico para la seguridad y la calidad del servicio:

- **Falso Positivo (FP):** Ocurre cuando el detector predice la presencia de un vehículo o matrícula donde no existe, o cuando clasifica erróneamente un automóvil no registrado como autorizado. En la práctica, un FP representa una **falla de seguridad grave**, pues implica abrir automáticamente la barrera de acceso a un vehículo no permitido o registrar un ingreso inexistente.
- **Falso Negativo (FN):** Ocurre cuando el modelo omite la presencia de un automóvil en el carril o no logra detectar la matrícula de un cliente registrado. Operativamente, un FN genera congestión en la barrera, ya que deja esperando injustificadamente al usuario autorizado y requiere la intervención manual del operador.

A partir de estas categorías, se definen las métricas fundamentales de evaluación:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.9}$$

Mientras que la *Precision* mide la validez de las aperturas de barrera efectuadas (evitando falsos accesos), el *Recall* mide la tasa de recuperación de vehículos autorizados (evitando demoras innecesarias). El *F1-Score* constituye la media armónica entre ambas, proporcionando un indicador sintético del desempeño global.

Para evaluar la localización espacial de las cajas delimitadoras (*bounding boxes*) de los automóviles y sus matrículas, Padilla et al. [64] formulan la precisión media promedio (*Mean Average Precision*, mAP). El mAP evalúa el área bajo la curva de Precision-Recall considerando distintos umbrales de Intersección sobre Unión (IoU). Típicamente, el mAP@0.5 evalúa la capacidad del detector para encerrar de forma aproximada el vehículo o la placa (IoU ≥ 0.5), mientras que el mAP@0.5 : 0.95 promedia el desempeño en diez umbrales estrictos [65], midiendo la precisión geométrica del ajuste de la caja delimitadora.

1.7.2. Métricas de reconocimiento de secuencias: Exactitud por carácter y CER

En el módulo de Reconocimiento Óptico de Caracteres (OCR), la evaluación trasciende la clasificación de imágenes individuales para medir la fidelidad en la lectura de cadenas alfanuméricas continuas. Como detallan Shi et al. [54] y Auza Banegas et al. [15], la métrica principal para evaluar la precisión del OCR es la Tasa de Error por Carácter (*Character Error Rate*, CER), la cual se fundamenta en la distancia de edición de Levenshtein:

$$\mathrm{CER} = \frac{S + D + I}{N} \tag{1.10}$$

donde S representa el número de sustituciones de caracteres, D las deleciones (caracteres omitidos), I las inserciones (caracteres fantasma añadidos) y N la cantidad total de caracteres reales en la matrícula de referencia.

En la lectura de matrículas, el impacto de una sustitución es directo. Por ejemplo, si el motor de OCR confunde el carácter alfabético "O" con el dígito numérico "0" en una placa real (prediciendo "1204ABC" en lugar de "1204ABC"), la distancia de Levenshtein contabiliza una sustitución ($S = 1$). Aunque la precisión a nivel de carácter sea elevada ($6/7 = 85.7\,\%$), la métrica de coincidencia exacta (*exact match*) cae a cero. Dado que las consultas en la base de datos del parqueadero requieren una coincidencia alfanumérica exacta para autorizar

el paso, minimizar el CER mediante reglas de corrección sintáctica es indispensable para garantizar el funcionamiento del sistema.

1.7.3. Métricas de seguimiento de objetos: MOTA, MOTP e IDF1

Para evaluar la estabilidad del módulo de seguimiento multiobjeto a lo largo del tiempo, Bernardin y Stiefelhagen [66] y Ristani et al. [67] formalizaron el conjunto de métricas CLEAR MOT:

1. **Precisión de Seguimiento Multiobjeto (*Multiple Object Tracking Precision*, MOTP):** Mide la precisión geométrica promedio en la estimación de las posiciones de los vehículos coincidentes a lo largo de la secuencia de video:

$$\text{MOTP} = \frac{\sum_{t,i} d_{t,i}}{\sum_t c_t} \tag{1.11}$$

   donde $d_{t,i}$ representa la distancia o desalineación espacial entre la caja predicha y la real para la coincidencia i en el fotograma t, y c_t es el número total de coincidencias.

2. **Exactitud de Seguimiento Multiobjeto (*Multiple Object Tracking Accuracy*, MOTA):** Evalúa la capacidad global del sistema para mantener trayectorias continuas, integrando los tres tipos de errores posibles:

$$\text{MOTA} = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t} \tag{1.12}$$

   donde GT_t es la cantidad de vehículos reales en el fotograma t, y $IDSW_t$ representa el número de cambios de identidad (*Identity Switches*). En el parqueadero, un cambio de ID erróneo ($IDSW$) ocurre cuando el rastreador asigna un nuevo número de seguimiento a un automóvil que avanza lentamente hacia la barrera, provocando registros duplicados del mismo evento de ingreso.

3. **Puntuación IDF1 (*Identification F1-Score*):** Mide la proporción de detecciones correctamente identificadas a lo largo de toda la trayectoria completa del vehículo, evaluando la consistencia del mapeo de identidades a largo plazo frente a fragmentaciones u oclusiones temporales [67].

Estas métricas delimitan el marco de evaluación del seguimiento, pero este trabajo no las reporta: su cálculo exige verdad de referencia por fotograma —una caja e identidad

correctas para cada vehículo en cada instante—, que no fue anotada. La determinación de eventos se contrastó al nivel del evento y no al de la trayectoria (Subsección 3.7.7): la referencia humana registra cruces, no identidades por fotograma, de modo que MOTA e IDF1 no pueden calcularse. La descomposición de errores que esas métricas formalizan —falsos negativos, falsos positivos y cambios de identidad— sigue siendo, en cambio, el marco con el que se interpretan los cruces perdidos y espurios. Las métricas se conservan aquí porque definen qué haría falta anotar para cerrar esa brecha.

1.7.4. Compromiso entre exactitud y latencia: Rendimiento en el Borde

En aplicaciones industriales de control de accesos, el desempeño de un algoritmo no se juzga únicamente por su exactitud en conjuntos de datos estáticos, sino por su capacidad para responder bajo límites temporales estrictos en el entorno operativo. Para garantizar una experiencia fluida al usuario y evitar embotellamientos en la vía pública, el pipeline completo de visión (detección de vehículo, localización de placa, lectura de OCR y decisión de barrera) debe ejecutarse con una **latencia total inferior a 1 segundo por evento** [15].

El análisis experimental de rendimiento evalúa el compromiso (*trade-off*) entre exactitud y velocidad mediante dos indicadores clave:

- **Tiempo de Inferencia (ms/img):** La latencia individual requerida por la red neuronal para procesar una imagen de entrada en la GPU o acelerador de cómputo.
- **Fotogramas por Segundo (FPS):** La tasa de refresco continua que puede sostener el pipeline en tiempo de ejecución.

El desafío reside en que los dispositivos de borde (*Edge AI*) instalados en las casetas del parqueadero (tales como módulos NVIDIA Jetson u ordenadores embebidos) disponen de restricciones severas de consumo energético, memoria VRAM y capacidad de cómputo en comparación con servidores en la nube. Por ello, la arquitectura del detector debe equilibrar el número de parámetros y la cantidad de operaciones de punto flotante (GFLOPs) para sostener tasas superiores a 30 FPS en resolución de alta definición.

1.7.5. Compresión de modelos: Cuantización de pesos (FP32, FP16, INT8)

Para adaptar redes neuronales profundas y complejas a hardware embebido restringido sin sacrificar su precisión predictiva, Sapkota et al. [43] y Jocher et al. [44] destacan la aplicación de técnicas de compresión de modelos, siendo la **cuantización de pesos** la más efectiva para acelerar la inferencia.

La cuantización consiste en mapear la representación numérica de los parámetros y activaciones de la red desde formatos continuos de alta precisión hacia formatos discretos de menor ancho de banda de memoria:

- **FP32 (Punto Flotante de 32 bits):** Es el formato de precisión simple estándar utilizado durante el entrenamiento en GPU, ofreciendo un amplio rango dinámico a costa de una elevada huella de memoria y mayor consumo energético.
- **FP16 (Media Precisión de 16 bits):** Reduce a la mitad el tamaño del modelo en memoria RAM/VRAM y acelera la inferencia en aceleradores modernos con unidades de tensor nativas, conservando prácticamente el 100 % de la precisión original.
- **INT8 (Enteros de 8 bits):** Mapea los pesos de 32 bits a valores enteros discretos dentro del rango $[-128, 127]$.

Pasar un modelo de detección u OCR desde FP32 a INT8 reduce la huella de memoria del archivo de pesos en un 75 % y triplica la velocidad de inferencia (FPS) en aceleradores como TensorRT, OpenVINO o ONNX Runtime [43], [44]. Dado que la cuantización transforma operaciones de multiplicación matricial complejas en aritmética de enteros de bajo consumo, los dispositivos de borde pueden procesar los flujos de video con una latencia de milisegundos, experimentando una degradación en la precisión (mAP) casi imperceptible ($\leq 1\,\%$). Esta optimización permite desplegar modelos de vanguardia (como la familia YOLO) directamente en las casetas del estacionamiento, garantizando un control de acceso autónomo, robusto y eficiente.

1.7.6. Valores de referencia y umbrales de aceptación del proyecto

Las métricas definidas en los apartados precedentes carecen de poder de decisión mientras no se establezca un valor de referencia que permita calificar el desempeño de un modelo como suficiente o insuficiente. Un umbral desprovisto de justificación constituye un criterio arbitrario, por lo que en este apartado se explicitan los valores adoptados y su procedencia.

La justificación combina dos fuentes complementarias. La primera es el rango de desempeño reportado en la literatura para tareas equivalentes, que sitúa el umbral dentro de lo que la comunidad científica considera alcanzable. La segunda es el requisito operativo del sistema: el nivel de error a partir del cual el control de acceso deja de ser confiable y exige intervención manual. Cuando la literatura revisada no reporta un rango directamente comparable, el umbral se sustenta exclusivamente en la segunda fuente y así se declara de forma explícita.

La Tabla 1.1 resume los umbrales adoptados, el valor de referencia que los respalda y su procedencia.

Cuadro 1.1: Umbrales de aceptación y valores de referencia

Indicador	**Umbral**	**Valor de referencia**	**Procedencia**
mAP@50 (detección de vehículos)	$\geq 80\,\%$	Entre 40.9 y 57.5 de mAP@50–95 en COCO para la familia YOLO26.	[43], [44]
F1-score macro (tipo de vehículo)	$\geq 85\,\%$	Sin rango directamente comparable en las fuentes revisadas.	Criterio operativo
F1-score macro (color predominante)	$\geq 85\,\%$	Entre 44.6 % y 62.8 % de F1 macro en el benchmark de condiciones adversas (once clases, operación nocturna).	[49]
Recall (detección de placas)	$\geq 90\,\%$	Entre 91.3 % y 98.8 % de exactitud de detección en la revisión sistemática de sistemas de reconocimiento de placas.	[19], [53]

Continuación de la Tabla 1.1

Indicador	Umbral	Valor de referencia	Procedencia
Exactitud por carácter (OCR)	$\geq 90\,\%$	89.6 % de exactitud a nivel de carácter sobre datos reales con matrículas del contexto local.	[15]
Exactitud de eventos	$\geq 90\,\%$	Las fuentes revisadas definen las métricas de seguimiento pero no un umbral de aceptación.	Criterio operativo
Latencia por evento	≤ 1 s	Sin rango comparable; condicionada por la exigencia de operación en tiempo real.	Criterio operativo

Fuente: Elaboración propia

Detección de vehículos. La familia YOLO26 reporta en COCO valores de mAP@50–95 comprendidos entre 40.9 y 57.5 según la escala del modelo [43], [44], métrica que resulta más exigente que el mAP@50 evaluado en este proyecto, pues promedia diez umbrales de intersección sobre unión en lugar de uno. Considerando además que el dominio de estudio es sustancialmente más acotado que COCO —una única escena, condiciones de captura estables y una sola familia de objetos—, el umbral de 80 % de mAP@50 se sitúa por encima del desempeño generalista reportado y es, por tanto, exigente pero alcanzable.

Reconocimiento de color. Los valores de referencia disponibles provienen de conjuntos con condiciones más favorables que las del recinto y no deben leerse como un rango único. Los trabajos que reportan 83.5 % y 97.8 % lo hacen sobre imágenes de autopista y de vista trasera en condiciones controladas, respectivamente. El benchmark específico de condiciones adversas [49], con once clases y operación nocturna, reporta entre 44.6 % y 62.8 % de F1 macro, con el máximo en el mejor de los modelos evaluados. El umbral de 85 % no se sitúa, por tanto, en el extremo conservador de la literatura: la excede en más de veinte puntos. Se adopta como *exigencia operativa* del recinto —un error superior obliga a verificación manual del color en una proporción apreciable de vehículos— y no como estándar reportado, y así se declara en la Tabla 1.1.

Reconocimiento de placas. El umbral de Recall adoptado (90 %) se sitúa en el extremo inferior del rango de detección reportado (91.3 % a 98.8 %), lo que lo convierte en el

criterio menos exigente del conjunto. En cambio, el umbral de exactitud por carácter (90 %) se apoya en un resultado obtenido sobre datos reales con matrículas del contexto local, que reporta 89.6 %; adoptar 90 % implica exigir un desempeño igual o superior al reportado, lo que constituye uno de los criterios más demandantes del proyecto.

Tipo de vehículo y determinación de eventos. Las fuentes revisadas fundamentan la elección de las métricas —media armónica para el desbalance entre clases y las métricas CLEAR MOT para el seguimiento— pero no establecen un valor de aceptación universal. El umbral de 85 % para el tipo de vehículo y el de 90 % para los eventos se justifican, en consecuencia, por vía operativa: un error superior a esos niveles implicaría que una proporción apreciable de vehículos requeriría verificación manual en la barrera, anulando el beneficio de la automatización.

Latencia. La restricción de un segundo por evento se apoya en la exigencia operativa del recinto y en la referencia del contexto local [15], que la adopta como límite de operación; no existe, en cambio, un rango de desempeño reportado que permita fijar el umbral por comparación. Por encima de ese valor, el tiempo de espera del conductor en la barrera deja de ser aceptable. Su verificación es experimental y se realiza sobre el hardware de despliegue.

En síntesis, tres de los siete criterios se apoyan en un rango de desempeño reportado en la literatura y en el requisito operativo, mientras que los cuatro restantes se justifican por vía operativa; en el caso de la latencia, la literatura respalda la exigencia temporal, pero no un rango de referencia. Esta distinción se declara de forma explícita para que la evaluación posterior permita distinguir entre un resultado que alcanza un estándar de la disciplina y uno que satisface una exigencia operativa del proyecto.

Capítulo II

Diseño del Sistema

2.1. Introducción

El presente capítulo define la arquitectura del sistema inteligente de gestión de parqueaderos, traduciendo los fundamentos teóricos desarrollados en el capítulo anterior en decisiones concretas de diseño. Mientras el marco teórico estableció *qué* técnicas de visión artificial y aprendizaje profundo existen y *por qué* resultan adecuadas para el problema planteado, este capítulo responde a las preguntas de *cómo* se organizan dichas técnicas en una solución integrada, *qué* componentes la conforman y *cómo* interactúan entre sí para automatizar la identificación vehicular y el registro de eventos de ingreso y salida.

El diseño se estructura de forma descendente (*top-down*), partiendo de una visión global del sistema y refinando progresivamente cada uno de sus subsistemas. En primer lugar, se especifican los requerimientos funcionales, no funcionales y de infraestructura que delimitan el alcance del sistema y sirven como criterio de aceptación para las etapas posteriores de implementación y evaluación. A continuación, se describe la arquitectura general, identificando los componentes principales y las interfaces de comunicación entre el punto de captura, el motor de percepción y la plataforma de gestión.

Posteriormente, el capítulo profundiza en el diseño del subsistema de visión artificial, que constituye el núcleo del proyecto y articula el *pipeline* de detección, reconocimiento y seguimiento vehicular descrito en el marco teórico. Luego se aborda el diseño del subsistema de gestión, compuesto por la interfaz de programación de aplicaciones (API), el modelo de

datos y los mecanismos de persistencia de la información. Finalmente, se presenta el diseño de la interfaz de usuario orientada al operador del recinto.

Es importante destacar que este capítulo se limita a la fase de diseño y modelado: no incluye código fuente ni resultados experimentales, los cuales corresponden al capítulo de implementación y pruebas. Las decisiones aquí documentadas, tales como los umbrales de desempeño, los diagramas de arquitectura y los modelos de datos, constituyen la referencia formal contra la cual se verificará el sistema construido.

2.2. Requerimientos del Sistema

La especificación de requerimientos constituye el contrato técnico que delimita el alcance del sistema y establece los criterios verificables de aceptación. A partir de los objetivos planteados y de las condiciones de operación del parqueadero, los requerimientos se clasifican en funcionales, no funcionales y de infraestructura.

2.2.1. Requerimientos funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe proporcionar para cumplir con su propósito operativo. La Tabla 2.1 los enumera y describe.

Cuadro 2.1: Requerimientos funcionales del sistema

ID	Requerimiento	Descripción
RF-01	Adquisición de video	Recibir y procesar el flujo de video proveniente de las cámaras ubicadas en los puntos de acceso del parqueadero, con capacidad de operación continua.
RF-02	Detección de vehículos	Detectar y localizar cada vehículo presente en el área de aproximación, representándolo mediante una caja delimitadora.

Continuación de la Tabla 2.1

ID	Requerimiento	Descripción
RF-03	Identificación del tipo de vehículo	Clasificar el vehículo detectado en las categorías definidas para el entorno de estudio (por ejemplo, sedán, camioneta, minibús o motocicleta).
RF-04	Identificación del color predominante	Determinar el color predominante de la carrocería del vehículo dentro de un conjunto de clases cromáticas previamente definido.
RF-05	Detección de la placa de matrícula	Localizar la región de la placa de matrícula dentro de la caja del vehículo mediante una caja delimitadora específica.
RF-06	Reconocimiento de la matrícula	Reconocer los caracteres alfanuméricos de la placa detectada y construir la cadena de texto correspondiente al formato de matrícula vigente.
RF-07	Seguimiento del vehículo	Asignar y mantener un identificador temporal único a cada vehículo mientras permanece en el campo de visión de la cámara.
RF-08	Determinación de eventos	Clasificar el movimiento del vehículo como ingreso o salida a partir del cruce de una línea virtual de control y del sentido de su trayectoria.
RF-09	Registro del evento	Almacenar cada evento con la placa reconocida, el tipo y color del vehículo, la fecha, la hora, la cámara y el sentido correspondiente.
RF-10	Control de autorización	Verificar la matrícula reconocida contra los registros de vehículos autorizados y reportar los accesos no autorizados.

Continuación de la Tabla 2.1

ID	Requerimiento	Descripción
RF-11	Consulta de información	Permitir la consulta y visualización del historial de eventos vehiculares registrados por el sistema.
RF-12	Generación de estadísticas	Producir información estadística a partir de los registros históricos, como afluencia vehicular y tiempos de permanencia.
RF-13	Gestión de usuarios	Administrar el acceso de los usuarios al sistema web conforme a roles definidos (administrador, operador y dueño).

Fuente: Elaboración propia

El requerimiento RF-10 se materializa mediante un padrón institucional de vehículos habilitados gestionado por el administrador. El sistema contrasta la matrícula leída en cada ingreso contra dicho padrón para clasificar el acceso y emitir la alerta correspondiente en caso de un vehículo no registrado. El detalle de la entidad y de la interfaz de administración se documenta en los capítulos de diseño e implementación.

2.2.2. Requerimientos no funcionales

Los requerimientos no funcionales establecen restricciones de calidad y condiciones de operación, en línea con las características de calidad de producto de la norma ISO/IEC 25010 [68]. Su cumplimiento se verifica en la fase de evaluación mediante las métricas descritas en el marco teórico; los valores objetivo son coherentes con las hipótesis planteadas en el perfil del proyecto.

Cuadro 2.2: Requerimientos no funcionales del sistema

ID	Categoría	Criterio de cumplimiento
RNF-01	Rendimiento temporal	El *pipeline* completo de visión debe procesar un evento de acceso con una latencia promedio igual o inferior a un segundo.
RNF-02	Exactitud de detección	El modelo de detección de vehículos debe alcanzar un mAP@50 igual o superior al 80 %.
RNF-03	Exactitud de clasificación	Los modelos de identificación de tipo y color predominante deben alcanzar un F1-score macro igual o superior al 85 %.
RNF-04	Exactitud de reconocimiento	El módulo de reconocimiento de placas debe alcanzar un Recall igual o superior al 90 % en la detección y una exactitud por carácter igual o superior al 90 % en la lectura.
RNF-05	Exactitud de eventos	La determinación de los eventos de ingreso y salida debe alcanzar una exactitud igual o superior al 90 %.
RNF-06	Disponibilidad	El sistema debe operar de forma continua durante la jornada de operación del parqueadero, resistiendo reinicios controlados de sus componentes.
RNF-07	Escalabilidad	La arquitectura debe permitir incorporar nuevos puntos de acceso sin modificar el núcleo del subsistema de visión ni degradar el rendimiento.
RNF-08	Eficiencia en el borde	El motor de percepción debe ejecutarse en hardware con recursos restringidos, empleando modelos optimizados y de precisión reducida.

Continuación de la Tabla 2.2

ID	Categoría	Criterio de cumplimiento
RNF-09	Seguridad y privacidad	El acceso a la información y a las imágenes debe estar protegido, y el tratamiento de datos debe considerar las disposiciones aplicables sobre protección de datos personales.
RNF-10	Trazabilidad	El sistema debe registrar eventos y errores en bitácoras que permitan auditar su comportamiento y reconstruir un evento ante una incidencia.

Fuente: Elaboración propia

2.2.3. Requerimientos de infraestructura y entorno

El sistema se despliega sobre una infraestructura física mínima compuesta por dispositivos de captura en los puntos de acceso, una unidad de procesamiento en el borde encargada de la inferencia, un servidor para la plataforma de gestión y la base de datos, y la red local que interconecta estos componentes. La Tabla 2.3 resume la función esperada de cada componente.

Cuadro 2.3: Componentes de infraestructura y entorno

Componente	Función en el sistema	Consideración de diseño
Cámara del punto de acceso	Capturar las imágenes y el video de los vehículos que aproximan a la barrera.	La posición, altura y ángulo de montaje determinan la calidad de la placa en el encuadre.
Unidad de procesamiento en el borde	Ejecutar los modelos de detección, reconocimiento y seguimiento.	Recursos limitados de cómputo y memoria; requiere modelos de precisión reducida.

Continuación de la Tabla 2.3

Componente	Función en el sistema	Consideración de diseño
Servidor de aplicación y base de datos	Alojar la API de gestión, la lógica de autorización y la persistencia de los eventos.	Debe garantizar disponibilidad durante la operación y respaldo de la información.
Red local	Transportar el video desde la cámara y los eventos hacia el servidor.	El ancho de banda y la estabilidad condicionan la latencia del sistema.
Almacenamiento	Conservar los registros de eventos y, cuando corresponda, la evidencia visual asociada.	La política de retención debe equilibrar trazabilidad y privacidad.

Fuente: Elaboración propia

Las características técnicas concretas de los dispositivos (modelo de cámara, capacidad de cómputo del equipo de borde y dimensionamiento del servidor) se determinan en la fase de análisis de las condiciones del parqueadero y se documentan en el capítulo de implementación.

2.3. Descripción de la Arquitectura General

El sistema se organiza bajo una arquitectura en capas (*layered architecture*) [69] que separa las responsabilidades de captura, servicios, procesamiento, persistencia y presentación. Esta organización responde a la naturaleza asimétrica de la carga de trabajo: la recepción de solicitudes es ligera y debe permanecer siempre disponible, mientras que la inferencia de los modelos de aprendizaje profundo es intensiva en cómputo y su duración es variable. Separar ambas responsabilidades permite que el servicio de recepción responda de inmediato y delegue la inferencia a un proceso trabajador independiente, evitando que una ráfaga de vehículos bloquee la atención de las solicitudes.

La Figura 2.1 presenta la arquitectura general del sistema, en la que cada capa se apoya exclusivamente en las interfaces públicas de la capa adyacente. El flujo de información es descendente en el sentido de la percepción —desde la imagen capturada hacia el registro

persistido— y ascendente en el sentido de la supervisión, a través de las consultas y de las notificaciones en tiempo real que el sistema envía al panel de monitoreo.

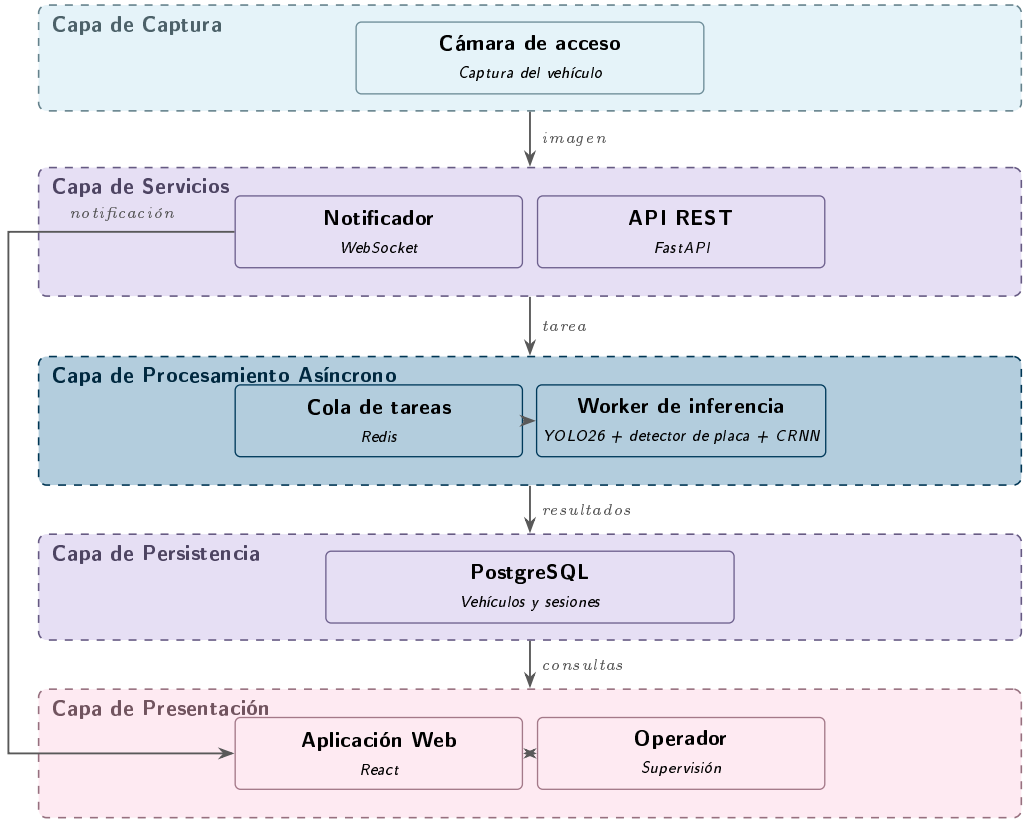


Figura 2.1: Arquitectura general en capas del sistema de gestión inteligente de parqueaderos.

2.3.1. Capa de captura

Constituye el punto de entrada de información al sistema y está conformada por las cámaras instaladas en los accesos del parqueadero. Su responsabilidad es adquirir la imagen del vehículo que aproxima a la barrera y remitirla al servicio de gestión. El diseño de esta capa se mantiene deliberadamente simple para favorecer la reutilización de la infraestructura de captura existente y para desacoplar el sistema de un modelo específico de cámara: basta con que el dispositivo de captura pueda entregar una imagen en un formato conocido.

2.3.2. Capa de servicios

Expone la interfaz pública del sistema y constituye el único punto de acceso a sus funcionalidades. La API REST [70] recibe las solicitudes de registro de ingreso y de salida,

valida su estructura y delega el trabajo pesado a la capa de procesamiento, respondiendo de inmediato con el identificador de la sesión creada. El notificador mantiene un canal persistente con los clientes web y propaga las actualizaciones de estado sin que estos deban consultar periódicamente al servidor. Esta capa no conoce los detalles internos de los modelos de visión, lo que permite evolucionar el motor de inferencia sin alterar el contrato expuesto a los clientes.

2.3.3. Capa de procesamiento asíncrono

Es el núcleo analítico del proyecto. La cola de tareas desacopla la recepción de la solicitud de su ejecución: cada solicitud de registro encola una tarea y el proceso trabajador la consume cuando dispone de capacidad. El trabajador de inferencia ejecuta la detección de vehículos, el seguimiento, el reconocimiento de la matrícula y la clasificación de atributos, y deposita el resultado en la capa de persistencia. Al residir en un proceso separado, esta capa puede desplegarse sobre hardware con aceleración gráfica —incluso en equipos próximos al punto de captura— y aprovechar modelos optimizados para reducir su latencia, sin condicionar la disponibilidad del servicio de recepción.

2.3.4. Capa de persistencia

Almacena el estado del sistema: los vehículos registrados, las sesiones de estacionamiento con sus marcas temporales de ingreso y salida, y la bitácora de detecciones producida por el proceso de inferencia. La base de datos relacional actúa como fuente única de verdad y es manipulada exclusivamente por los servicios de la capa superior, garantizando que las reglas de integridad se apliquen en un único punto.

2.3.5. Capa de presentación

Ofrece la interfaz mediante la cual el personal del parqueadero supervisa la operación. La aplicación web consume la API REST para las consultas históricas y mantiene una conexión de notificación para recibir las actualizaciones en tiempo real. Al ser una capa puramente de consumo, puede desplegarse de forma independiente y ser reemplazada o ampliada sin impacto sobre el resto de la arquitectura.

2.3.6. Flujo de comunicación

La comunicación entre capas sigue un contrato estricto. La imagen capturada se remite al servicio de gestión, que crea el registro de la sesión y encola la tarea de inferencia; el trabajador consume la tarea, ejecuta los modelos y actualiza la sesión con los resultados obtenidos. Una vez consolidado el resultado, el notificador propaga el cambio a la aplicación web. Esta secuencia garantiza que ningún componente acceda directamente a los datos de otro: la base de datos es manipulada por los servicios, y los modelos de visión no conocen la estructura del almacenamiento. Dicha separación de responsabilidades simplifica el mantenimiento, facilita las pruebas por componente y sostiene la escalabilidad del sistema ante la incorporación de nuevos puntos de acceso.

2.4. Diseño del Subsistema de Visión Artificial

El subsistema de visión artificial concentra la funcionalidad central del proyecto: transformar el material visual capturado en los accesos en información estructurada y verificable. A diferencia de un detector aislado, este subsistema se organiza como un *pipeline* en el que la salida de cada etapa alimenta a la siguiente, por lo que sus interfaces de datos se especifican de forma explícita para garantizar la trazabilidad y la verificabilidad de cada componente.

2.4.1. Arquitectura de los pipelines de visión

El subsistema de visión comprende dos *pipelines* con propósitos y regímenes de ejecución distintos, cuya separación es deliberada.

El primero es el **pipeline de operación**, que se ejecuta en tiempo real sobre el flujo de video de cada punto de acceso y conserva un estado por cámara mientras un vehículo permanece dentro del campo de visión. Es el responsable de producir los eventos de acceso, y es el que se describe en esta sección.

El segundo es el **pipeline de construcción del conjunto de datos**, que se ejecuta fuera de línea sobre el material de video archivado. Su propósito no es operar el parqueadero, sino producir las muestras etiquetadas con las que se entrenan y evalúan los modelos; constituye, por tanto, un instrumento metodológico, y se documenta junto con la construcción

efectiva del conjunto de datos en el Capítulo III.

Ambos comparten los módulos de detección y de reconocimiento, pero difieren en su régimen de ejecución y en su producto: el primero emite eventos de acceso, el segundo emite muestras anotadas. La Figura 2.2 presenta el pipeline de operación, cuyas etapas y artefactos de datos se detallan en la Tabla 2.4.

El pipeline de operación se organiza en seis etapas secuenciales. Dado que la determinación de eventos depende de la trayectoria del vehículo, estas etapas se ejecutan sobre una secuencia de fotogramas y no sobre capturas aisladas: el estado de cada vehículo se conserva entre fotogramas dentro del alcance de su cámara, lo que habilita el seguimiento y la interpretación de la trayectoria. El servicio admite además un modo de captura puntual, en el que se recibe una única imagen del vehículo; ese modo cubre las etapas de detección, reconocimiento de placa y clasificación de atributos, pero no la determinación de eventos por trayectoria, que requiere continuidad temporal.

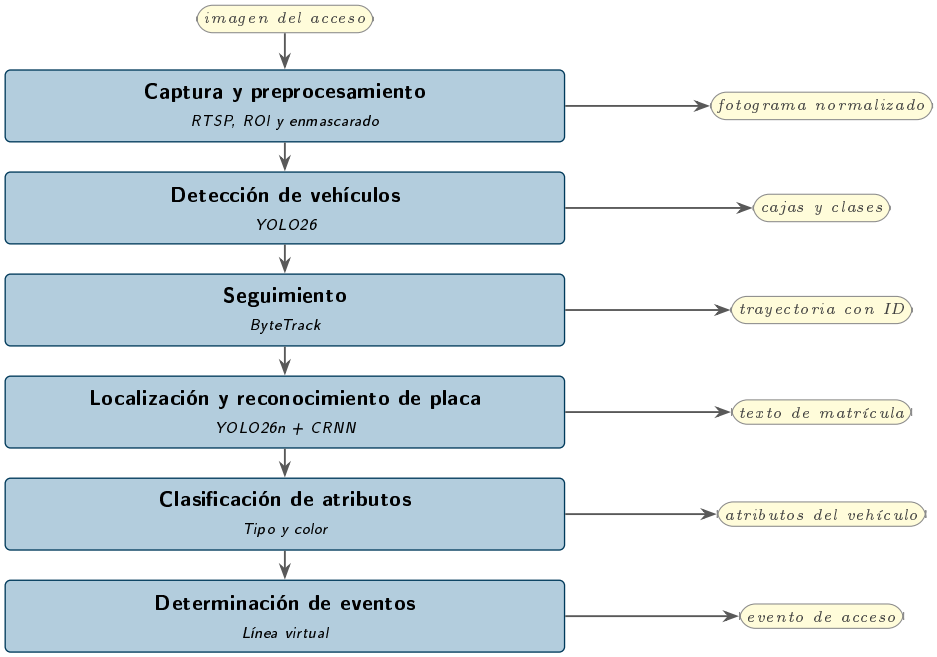


Figura 2.2: Pipeline de operación del subsistema de visión artificial.

Cuadro 2.4: Módulos del pipeline de percepción

Módulo	Entrada	Salida	Restricción de diseño
Captura y pre-procesamiento	Imagen del acceso	Fotograma normalizado	Debe recortar la región de interés y enmascarar la información incrustada por el grabador.
Detección de vehículos	Fotograma normalizado	Cajas con clase y confianza	Debe operar sin post-procesamiento dependiente del número de objetos para mantener latencia constante.
Seguimiento	Cajas detectadas	Trayectoria con identificador	Debe sobrevivir a oclusiones breves sin perder la identidad del vehículo.
Localización y reconocimiento de placa	Recorte del vehículo	Cadena de matrícula	Debe localizar la placa dentro del vehículo y transcribirla sin requerir segmentación individual de caracteres ni depender de un umbral de binarización fijo.
Clasificación de atributos	Recorte del vehículo	Tipo y color	Debe ser robusto frente a reflejos especulares y variaciones de iluminación.
Determinación de eventos	Trayectoria del vehículo	Evento de ingreso o salida	Debe evitar disparos múltiples para un mismo vehículo.

Fuente: Elaboración propia

2.4.2. Detección de vehículos

Este módulo constituye la puerta de entrada analítica del sistema. Recibe el fotograma normalizado y emite el conjunto de cajas delimitadoras con su clase y su nivel de confianza, distinguiendo entre la silueta completa del vehículo y la región de la placa de matrícula. La decisión de diseño central es el empleo de un detector de una sola etapa con inferencia libre de supresión no máxima, lo que garantiza que el tiempo de cómputo no dependa del número de objetos presentes en la escena. Esta propiedad resulta indispensable en un acceso vehicular, donde dos o tres vehículos pueden solaparse en el encuadre y la supresión no máxima convencional podría descartar una detección válida o introducir una latencia variable, tal como se fundamentó en el capítulo anterior.

La salida de este módulo se organiza como una lista de detecciones, cada una compuesta por la clase, las coordenadas de la caja, la confianza asociada y un identificador de fotograma, lo que permite reconstruir posteriormente el origen de cualquier decisión del sistema.

2.4.3. Seguimiento vehicular

El módulo de seguimiento asigna y mantiene un identificador único por vehículo a lo largo del tiempo. Su función es impedir que un mismo automóvil sea tratado como una sucesión de detecciones independientes, situación que generaría registros duplicados de un único evento de acceso. Para ello, el módulo modela el movimiento de cada caja mediante un estimador de estado que predice su posición en el fotograma siguiente y la corrige con cada nueva detección, combinando dicha predicción con la similitud geométrica entre las cajas.

La asociación entre predicciones y detecciones se resuelve priorizando las coincidencias de alta confianza y recurriendo a las de confianza menor solo para completar las trayectorias que permanecen sin asignar. Este criterio, propio del enfoque de seguimiento en dos etapas, reduce las fragmentaciones de identidad cuando un vehículo atraviesa oclusiones parciales provocadas por peatones, por la propia barrera o por destellos de luz. Las trayectorias que dejan de recibir detecciones se conservan durante un número limitado de fotogramas antes de descartarse, y aquellas con un número insuficiente de observaciones se consideran ruido y se eliminan. La salida del módulo es una trayectoria con identificador, compuesta por la secuencia temporal de las posiciones del vehículo y los atributos reconocidos.

2.4.4. Reconocimiento de matrículas

El módulo de reconocimiento recibe el recorte del vehículo y devuelve la cadena alfanumérica correspondiente. Comprende dos etapas: la localización de la placa dentro del recorte y la transcripción de la región localizada. Su diseño adopta un enfoque libre de segmentación: en lugar de aislar y clasificar cada carácter, un modelo de secuencia —una red convolucional recurrente con decodificación CTC— procesa la imagen completa de la placa y emite la transcripción de forma directa. Esta decisión responde a las condiciones reales del entorno, donde la suciedad, los marcos metálicos o las sombras degradan la separación entre caracteres y provocan el colapso de los enfoques clásicos por segmentación.

El módulo incorpora, además, un criterio de calidad previo al reconocimiento: los recortes cuya resolución o nitidez resultan insuficientes, o cuya posición en el encuadre no corresponde a la franja esperada de la placa, se descartan antes de invocar al motor de reconocimiento. Se prevé también una etapa de validación sintáctica opcional que contrasta el resultado con la gramática formal de las matrículas del contexto de estudio. Su efecto no se presume: se mide sobre el conjunto de prueba, porque una regla que reescribe posiciones puede convertir una lectura correcta en incorrecta cuando el formato admite letras dentro del bloque numérico. La implementación reporta ese resultado y la decisión adoptada (Sección 3.7.6). La salida del módulo comprende la matrícula reconocida, su cadena original y un indicador de confianza.

2.4.5. Clasificación de atributos del vehículo

Una vez localizado el vehículo, el sistema recorta su región y la somete a un módulo de clasificación encargado de determinar el tipo de carrocería y el color predominante. Este módulo no interviene en la decisión de apertura, pero aporta una capa de verificación cruzada: si la matrícula reconocida corresponde a un vehículo registrado con un tipo o un color distintos a los observados, el sistema puede señalar una posible suplantación de matrícula.

Para reducir la sensibilidad a los reflejos y a la iluminación variable, el recorte se transforma a un espacio cromático que desacopla la información de color de la intensidad luminosa antes de ingresar al clasificador. El resultado se expresa como la categoría más probable junto con su confianza, y se conserva únicamente si dicha confianza supera un umbral configurable; en caso contrario, el atributo se registra como no determinado, evitando incorporar información poco fiable al registro del evento.

2.4.6. Determinación de eventos y contrato de datos

La etapa final interpreta la trayectoria para decidir si el vehículo ingresó o salió del recinto. Para ello se define una línea virtual sobre la calzada frente a la barrera, complementada con dos regiones de control que delimitan el espacio exterior y el interior del recinto. El evento se genera cuando el centro de la caja del vehículo cruza dicha línea, y el sentido se determina a partir del vector de desplazamiento respecto de la normal de la línea y de la región de origen y destino de la trayectoria. Este criterio garantiza que cada vehículo produzca exactamente un evento por cruce, evitando las falsas activaciones que produciría un disparo basado únicamente en la presencia dentro de una región.

El resultado del *pipeline* constituye la interfaz formal entre el subsistema de visión y la capa de persistencia: un evento de acceso, no una sesión. La Tabla 2.5 define sus campos. La distinción importa porque la sesión —con su identificador, su estado y su duración— no viaja en el mensaje: se deriva al persistir el evento, que es lo único que la visión puede afirmar con evidencia.

Cuadro 2.5: Contrato de datos del evento de acceso

Campo	Tipo	Descripción
`camera_id`	texto	Identificador del punto de acceso que originó la observación.
`track_id`	entero	Identificador de la trayectoria asignada por el agregador; distingue dos vehículos de la misma cámara.
`box`	entero[4]	Coordenadas en píxeles de la caja del vehículo, tomadas de la vista de mayor área de la trayectoria.
`observed_at`	marca temporal	Instante del fotograma en que se confirmó la trayectoria.
`attributes.vehicle_type`	texto	Categoría de carrocería identificada, o nulo si la confianza resulta insuficiente.

Continuación de la Tabla 2.5

Campo	Tipo	Descripción
`attributes.vehicle_type_confidence`	real	Confianza del clasificador de tipo; se conserva junto a la etiqueta para que las etapas posteriores puedan descartar lecturas débiles.
`attributes.color`	texto	Color predominante identificado, o nulo si la confianza resulta insuficiente.
`attributes.color_confidence`	real	Confianza del clasificador de color.
`detection_confidence`	real	Confianza del detector de vehículos para la caja seleccionada.
`plate.text`	texto	Cadena resultante del reconocimiento y de la validación sintáctica del formato de matrícula; ausente cuando no se leyó placa.
`plate.confidence`	real	Confianza del reconocedor de caracteres, empleada por la política de emparejamiento.
`gate`	enumeración	Punto de acceso de la observación: `entry` o `exit`.
`image_path`	texto	Ruta del recorte del vehículo conservado como evidencia; puede ser nulo.
`model_name, latency_ms`	texto, real	Identificador del modelo empleado y tiempo de inferencia del fotograma.

Fuente: Elaboración propia

2.5. Diseño del Subsistema de Gestión

El subsistema de gestión recibe las imágenes capturadas en los accesos, coordina su procesamiento y convierte el resultado en información persistente, consultable y auditable. Su diseño se rige por el principio de responsabilidad única: la base de datos es manipulada exclusivamente por los servicios, y los servicios no conocen los detalles internos de los modelos de visión. De este modo, el reentrenamiento o la sustitución de un modelo no impacta sobre las reglas de negocio ni sobre la estructura de los datos.

2.5.1. Arquitectura del subsistema de gestión

El subsistema se organiza en cuatro componentes. La interfaz de programación de aplicaciones (API REST) expone los servicios del sistema, valida la estructura de las solicitudes y constituye el único punto de acceso a los datos. El notificador mantiene los canales persistentes con los clientes web y propaga las actualizaciones de estado. La cola de tareas desacopla la recepción de una solicitud de su procesamiento, y el trabajador de inferencia consume las tareas encoladas y deposita sus resultados en la capa de persistencia. Esta separación permite que las reglas de negocio evolucionen sin alterar el motor de inferencia, y que el motor de inferencia se optimice sin reescribir la lógica de gestión.

2.5.2. Modelo entidad-relación

El modelo de datos se estructura en seis entidades. La entidad *vehículo* representa a cada automóvil observado en los accesos y conserva los atributos visuales reconocidos en su primera aparición. La entidad *sesión* representa una visita al recinto: se crea en el cruce de ingreso y se cierra en el cruce de salida, registrando la duración de la permanencia, el estado del cierre y, cuando el cierre es manual, el operador y el motivo. La entidad *detección* constituye la bitácora de inferencia: conserva cada observación producida por el modelo, con su clase, confianza, latencia y trayectoria asociada. A estas tres se suman la entidad *cámara*, que registra los puntos de acceso configurados con su puerta, su URL de captura y su estado de habilitación; la entidad *usuario*, que almacena la credencial, el rol y el estado de la cuenta; y la entidad *vehículo autorizado*, que materializa el padrón institucional contra el cual se contrasta cada matrícula reconocida para el control de autorización (RF-10). La Figura 2.3 presenta la totalidad de las entidades y sus relaciones con la cardinalidad correspondiente.

Las relaciones trazadas con línea continua corresponden a claves foráneas declaradas en el esquema; las trazadas con línea discontinua son asociaciones lógicas por coincidencia de identificador o de matrícula, que el sistema resuelve en la capa de aplicación.

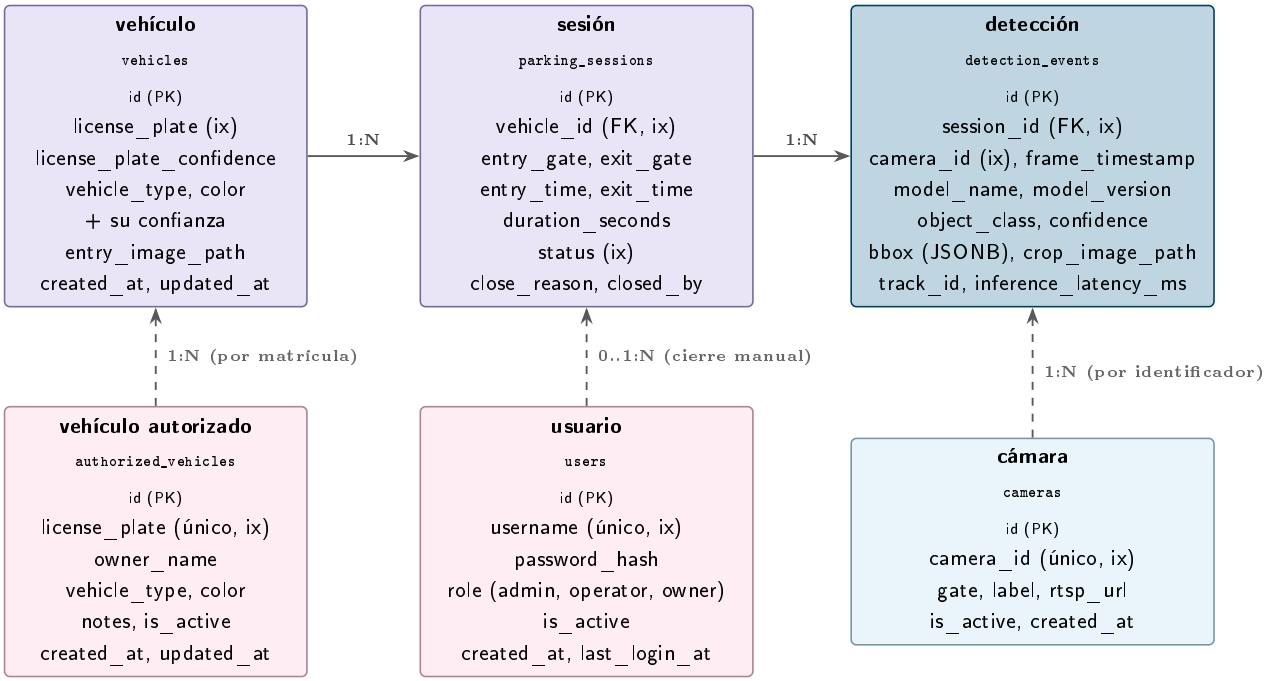


Figura 2.3: Modelo entidad-relación del subsistema de gestión. Cada entidad indica su nombre conceptual y, en tipografía monoespaciada, el nombre físico de la tabla en la base de datos.

La Tabla 2.6 detalla el diccionario de datos. Cada bloque identifica la entidad con su nombre conceptual y el nombre físico de la tabla, y cada fila emplea el nombre real de la columna tal como se declara en las migraciones, de modo que el esquema pueda verificarse contra el código sin traducciones intermedias. Las claves foráneas preservan la consistencia referencial y permiten reconstruir la trazabilidad completa de una visita, desde el vehículo identificado hasta cada una de las observaciones que sustentaron su identificación. El esquema se gestiona mediante migraciones versionadas, de modo que toda modificación estructural quede registrada y pueda reproducirse sobre cualquier entorno de despliegue.

Cuadro 2.6: Diccionario de datos del subsistema de gestión

Columna	Tipo	**Restricción**
vehículo — `vehicles`		
`id`	`UUID`	Clave primaria.

Continuación de la Tabla 2.6

Columna	Tipo	**Restricción**
license_plate	VARCHAR(20)	Indexado; nulo si no pudo reconocerse.
license_plate_confidence	REAL	Puede ser nulo.
vehicle_type	VARCHAR(50)	Puede ser nulo.
vehicle_type_confidence	REAL	Puede ser nulo.
color	VARCHAR(30)	Puede ser nulo.
color_confidence	REAL	Puede ser nulo.
entry_image_path	TEXT	Puede ser nulo.
created_at	TIMESTAMP	No nulo.
updated_at	TIMESTAMP	No nulo; se actualiza en cada modificación.
sesión — parking_sessions		
id	UUID	Clave primaria.
vehicle_id	UUID	Clave foránea a vehicles; no nulo e indexado.
entry_gate	VARCHAR(20)	Puede ser nulo.
exit_gate	VARCHAR(20)	Puede ser nulo.
entry_time	TIMESTAMP	No nulo.
exit_time	TIMESTAMP	Nulo mientras la sesión permanece activa.
duration_seconds	INTEGER	Segundos de permanencia; calculado al cierre.
status	ENUM	*active*, *completed* o *cancelled*; indexado.
close_reason	VARCHAR(50)	Nulo cuando la sesión sigue activa; motivo del cierre.

Continuación de la Tabla 2.6

Columna	Tipo	**Restricción**
closed_by	VARCHAR(50)	Nulo cuando el cierre es automático; operador que cerró.
detección — detection_events		
id	UUID	Clave primaria.
session_id	UUID	Clave foránea a parking_sessions; puede ser nulo e indexado.
camera_id	VARCHAR(50)	No nulo e indexado.
frame_timestamp	TIMESTAMP	No nulo.
model_name	VARCHAR(20)	Identificador del modelo empleado.
model_version	VARCHAR(20)	Puede ser nulo.
object_class	VARCHAR(50)	Clase detectada.
confidence	REAL	No nulo.
bbox	JSONB	Coordenadas normalizadas de la caja delimitadora.
crop_image_path	TEXT	Puede ser nulo.
track_id	INTEGER	Identificador de trayectoria asignado por el seguimiento.
inference_latency_ms	REAL	Tiempo de inferencia en milisegundos.
cámara — cameras		
id	UUID	Clave primaria.
camera_id	VARCHAR(64)	Único, indexado y no nulo; debe coincidir con el empleado por el capturador y el trabajador.
gate	VARCHAR(20)	No nulo; punto de acceso asociado.

Continuación de la Tabla 2.6

Columna	Tipo	**Restricción**
label	VARCHAR(120)	Puede ser nulo; denominación legible.
rtsp_url	TEXT	Puede ser nulo; cadena de captura del flujo.
is_active	BOOLEAN	No nulo; indica si la cámara se espera en operación.
created_at	TIMESTAMP	No nulo.
usuario — users		
id	UUID	Clave primaria.
username	VARCHAR(64)	Único, indexado y no nulo.
password_hash	VARCHAR(255)	No nulo; se almacena únicamente el resumen criptográfico, nunca la contraseña.
role	VARCHAR(20)	Restringido por CHECK a *admin*, *operator* u *owner*.
is_active	BOOLEAN	No nulo; habilita o suspende la cuenta.
created_at	TIMESTAMP	No nulo.
last_login_at	TIMESTAMP	Puede ser nulo.
vehículo autorizado — authorized_vehicles		
id	UUID	Clave primaria.
license_plate	VARCHAR(20)	Única, indexada y no nulo; sostiene el contraste de RF-10.
owner_name	VARCHAR(120)	Puede ser nulo.
vehicle_type	VARCHAR(50)	Puede ser nulo.
color	VARCHAR(30)	Puede ser nulo.
notes	TEXT	Puede ser nulo.

Continuación de la Tabla 2.6

Columna	Tipo	**Restricción**
`is_active`	`BOOLEAN`	No nulo; una baja lógica conserva el historial de eventos.
`created_at`	`TIMESTAMP`	No nulo.
`updated_at`	`TIMESTAMP`	No nulo; se actualiza en cada modificación.

Fuente: Elaboración propia

Para sostener el desempeño de las consultas sobre un volumen creciente de registros, el esquema define índices sobre `license_plate`, sobre `vehicle_id`, sobre `status`, sobre `camera_id` y sobre `session_id`. A ellos se suman los índices únicos sobre `username`, sobre `camera_id` y sobre la matrícula del padrón (`authorized_vehicles.license_plate`), que impiden duplicados en las tres entidades de control. Estos índices responden a los patrones de consulta predominantes del sistema: la recuperación del historial de un vehículo, la consulta de las sesiones activas para el cálculo de la ocupación, la revisión de las observaciones asociadas a una visita determinada y el contraste de una matrícula leída contra el padrón de vehículos habilitados.

2.5.3. Modelo de aforo perimetral y gestión de ocupación

Un aspecto de diseño que delimita el alcance del sistema es el modelo adoptado para la gestión de ocupación. Frente a los esquemas basados en supervisión de plazas individuales (*bay monitoring* o *slot guidance*), que demandan una infraestructura extensiva de cámaras cenitales o sensores de proximidad en cada cajón, este trabajo adopta el modelo de **aforo perimetral**.

Bajo este enfoque, el sistema modela el parqueadero como un recinto delimitado cuyo estado se gobierna por la ley de balance de flujo continuo:

$$O(t) = \min\left(C,\ \max\left(0,\ O(t_0) + \sum_{k} I_k(t_0, t) - \sum_{k} E_k(t_0, t)\right)\right) \tag{2.1}$$

donde $O(t)$ representa el número de vehículos activos en el recinto en el instante t, C es la capacidad nominal configurada, e I_k y E_k son los eventos de ingreso y egreso confirmados por el subsistema de visión artificial en los puntos perimetrales de acceso k. El porcentaje de ocupación relativo se expresa como $P(t) = (O(t)/C) \times 100$.

Este diseño presenta tres ventajas operativas y técnicas determinantes:

1. **Eficiencia de infraestructura:** requiere cobertura óptica exclusivamente en las vías de entrada y salida, reduciendo los costos de adquisición, cableado y mantenimiento frente a un esquema de cámaras internas.
2. **Inmunidad a oclusiones internas:** no depende de líneas de visión directas hacia cada cajón, las cuales suelen verse degradadas por columnas, techumbres bajas o vehículos de gran porte adyacentes.
3. **Trazabilidad vinculada a eventos:** cada unidad computada en el aforo corresponde rigurosamente a una sesión abierta con evidencia visual (recorte del vehículo y matrícula auditada), evitando discrepancias entre sensores dispersos.

2.5.4. Especificación de la interfaz de programación

La API se diseña siguiendo el estilo REST y expone sus servicios bajo un prefijo versionado, de modo que futuras modificaciones no invaliden a los clientes existentes. La Tabla 2.7 especifica los recursos previstos por el diseño. Nótese que el registro de ingreso y de salida queda reservado a los dispositivos de captura, mientras que las operaciones de consulta se restringen a los usuarios autenticados. En esa formulación, los recursos de registro responden de forma inmediata con el identificador de la sesión creada, sin esperar a que concluya la inferencia, que se ejecuta de manera asíncrona.

La implementación conserva la separación de responsabilidades, pero desplaza la ingesta fuera de la interfaz HTTP: el proceso de captura publica la tarea de inferencia directamente en la cola y el trabajador escribe el resultado en la base de datos, de modo que no existen recursos de registro de ingreso ni de salida y el camino de escritura no atraviesa el ciclo de atención de una petición. El vocabulario de las rutas se unificó en inglés, conforme a la convención de nomenclatura del código. Los recursos efectivamente expuestos se enumeran en la Tabla 3.7 del capítulo de implementación.

Cuadro 2.7: Especificación de los recursos de la API

Método	**Ruta**	**Descripción**	**Acceso**
POST	/api/v1/entry	Recibe la imagen del vehículo en el acceso, crea la sesión y encola la inferencia.	Dispositivo de captura
POST	/api/v1/exit	Recibe la imagen del vehículo en la salida y solicita el cierre de la sesión.	Dispositivo de captura
GET	/api/v1/vehiculos	Consulta los vehículos registrados con sus atributos visuales y su historial.	Operador
POST	/api/v1/whitelist	Registra o actualiza un vehículo en el padrón del recinto (RF-10).	Administrador
GET	/api/v1/stats/active	Lista las sesiones actualmente activas dentro del recinto.	Operador
GET	/api/v1/stats/history	Consulta el historial de sesiones con filtros por fecha.	Operador
GET	/api/v1/stats/occupancy	Devuelve el nivel de ocupación del recinto.	Operador
GET	/api/v1/stats/daily	Devuelve los indicadores agregados del día en curso.	Operador
WS	/api/v1/ws/occupancy	Canal persistente que propaga las variaciones de ocupación en tiempo real.	Operador

Fuente: Elaboración propia

La especificación formal de estos recursos se publica de forma automática a partir de las definiciones de tipo del propio servicio, lo que garantiza que la documentación de la interfaz permanezca sincronizada con su implementación. La implementación incorpora

además los recursos de gestión de usuarios y de registro de cámaras, que se documentan en el capítulo de implementación.

2.5.5. Procesamiento asíncrono y notificación en tiempo real

La inferencia de los modelos de visión es intensiva en cómputo y de duración variable, por lo que no puede ejecutarse dentro del ciclo de atención de una solicitud HTTP sin comprometer la disponibilidad del servicio. Ambos requerimientos se resuelven mediante dos mecanismos complementarios.

El primero es la ejecución asíncrona: al recibir una imagen, el servicio crea el registro de la sesión en estado activo y encola una tarea de inferencia, respondiendo de inmediato al dispositivo de captura. El trabajador consume la tarea cuando dispone de capacidad y, al concluir, actualiza la sesión con los atributos reconocidos. De este modo, una ráfaga de vehículos se traduce en una cola de tareas pendientes y no en una degradación del tiempo de respuesta de la API.

El segundo es la notificación en tiempo real. La aplicación web mantiene un canal persistente con el servicio, a través del cual recibe las variaciones de ocupación sin necesidad de consultar periódicamente al servidor. Este diseño reduce el tráfico de red y, sobre todo, elimina el retardo inherente al sondeo: el panel de monitoreo refleja el cambio en el instante en que el sistema lo consolida. Ante una interrupción del canal, el cliente restablece la conexión y reconcilia su estado consultando los recursos de historial y de ocupación.

2.5.6. Trazabilidad y control de autorización

La trazabilidad se garantiza en tres niveles. En el nivel de integridad, las claves foráneas y las restricciones del esquema impiden la existencia de sesiones sin vehículo asociado. En el nivel de evidencia, la bitácora de detecciones conserva cada observación producida por los modelos junto con su confianza, su latencia de inferencia y el identificador de la versión del modelo empleado, lo que permite reconstruir con qué versión del sistema se tomó cada decisión. En el nivel de correlación, cada sesión queda vinculada al vehículo identificado, a la puerta de acceso y a las detecciones que sustentaron la lectura de su matrícula.

El contraste de la matrícula reconocida contra un padrón de vehículos autorizados

(RF-10) se materializa mediante la entidad *vehículo_ autorizado* (`authorized_vehicles`), complementando a la entidad operativa *vehículo* que registra las observaciones de acceso. Al consolidarse un evento de ingreso con placa reconocida, el sistema verifica la existencia y vigencia de la matrícula en el padrón institucional, alertando al operador ante accesos de vehículos no habilitados.

Dado que el trabajador de inferencia puede reintentar la actualización de una sesión tras una interrupción, la operación de cierre se diseña como idempotente: la marca temporal de salida y la duración se calculan una única vez, y los reintentos posteriores sobre una sesión ya cerrada se ignoran. Esta decisión evita que una falla de comunicación derive en una duración inconsistente o en un cierre duplicado.

Asimismo, la arquitectura incorpora un mecanismo explícito de tolerancia a fallos frente a lecturas imprecisas del reconocedor de matrículas. En caso de que la matrícula leída en un evento de salida no encuentre una coincidencia exacta activa en la base de datos —situación esperable ante la tasa de error por carácter del OCR—, el sistema no bloquea el flujo ni corrompe el estado transaccional: clasifica el evento como una discrepancia y genera una alerta en la interfaz operativa de incidentes. De este modo, el operador dispone en pantalla de los fotogramas de entrada y salida, el recorte de la placa y los registros candidatos por orden de aproximación temporal, permitiendo realizar el cierre de sesión de forma manual, controlada y auditada (`cierre_operador` y `cierre_motivo`).

2.6. Diseño de la Interfaz de Usuario

La interfaz de usuario constituye la capa de presentación del sistema y tiene como propósito poner a disposición del personal del parqueadero la información generada por el motor de percepción. Su diseño se subordina a tres criterios: la separación de funciones según el rol del usuario, la prioridad de la información operativa sobre la información histórica, y la reducción del número de interacciones necesarias para resolver una incidencia de acceso.

2.6.1. Criterios de diseño y estructura de navegación

La interfaz reconoce tres roles con capacidades diferenciadas. El **operador** accede a las vistas de monitoreo, historial e incidentes, orientadas a la supervisión en tiempo real y

a la resolución de incidencias; el **administrador** accede además a la gestión de usuarios, al registro de cámaras, a la consulta estadística y a la administración del padrón de vehículos habilitados (RF-10); el rol de **dueño** (*owner*) accede únicamente a la lectura de ocupación, historial e indicadores, sin capacidad de modificar registros. Esta separación responde al principio de menor privilegio, según el cual cada usuario dispone únicamente de las capacidades necesarias para cumplir su función.

La navegación se organiza de forma plana a partir de un menú principal que se construye dinámicamente según el rol autenticado, evitando niveles de anidamiento que dificulten el acceso a las vistas de uso frecuente. La Figura 2.4 presenta la estructura de navegación resultante.

Un criterio transversal a todas las vistas es la actualización sin intervención del usuario. El cliente mantiene un canal persistente con el servicio y recibe por esa vía las variaciones del estado del recinto, de modo que el panel refleja un ingreso o una salida en el instante en que el sistema lo consolida, sin sondeo periódico ni acción manual de refresco. La consulta histórica, en cambio, se resuelve bajo demanda contra los recursos de la API, ya que su naturaleza no requiere actualización continua.

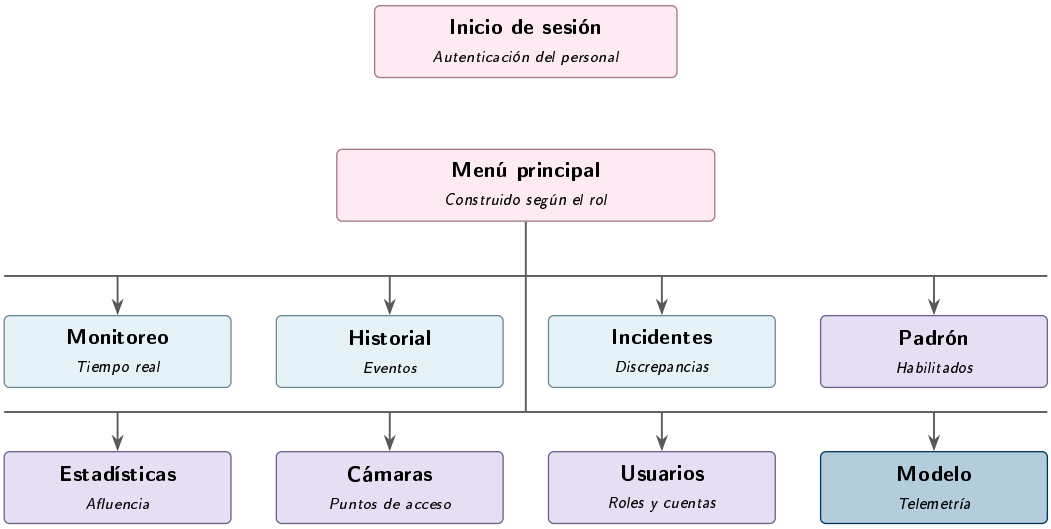


Figura 2.4: Estructura de navegación de la aplicación web.

2.6.2. Vistas de operación

Las vistas de operación concentran las funciones de supervisión. La Figura 2.5 presenta sus maquetas esquemáticas. La vista de monitoreo en tiempo real (Figura 2.5a) presenta el flujo de video del punto de acceso con las cajas delimitadoras superpuestas, el nivel de ocupa-

ción del recinto y, de forma simultánea, el resultado del último evento procesado: matrícula reconocida, tipo, color y sentido del cruce. Esta disposición permite al operador verificar en un solo golpe de vista si la identificación del sistema coincide con lo que observa en la imagen, y detectar de inmediato una eventual discrepancia entre la ocupación registrada y la ocupación real.

La vista de historial (Figura 2.5b) ofrece una consulta tabular de los eventos registrados, habilitando filtros por rango de fechas, por punto de acceso y por matrícula. Cada fila expone la marca temporal, la matrícula, la cámara de origen, el sentido del cruce y el estado de la sesión, de modo que el operador pueda localizar y auditar un acceso específico sin recurrir a la base de datos.



(a) Monitoreo en tiempo real.

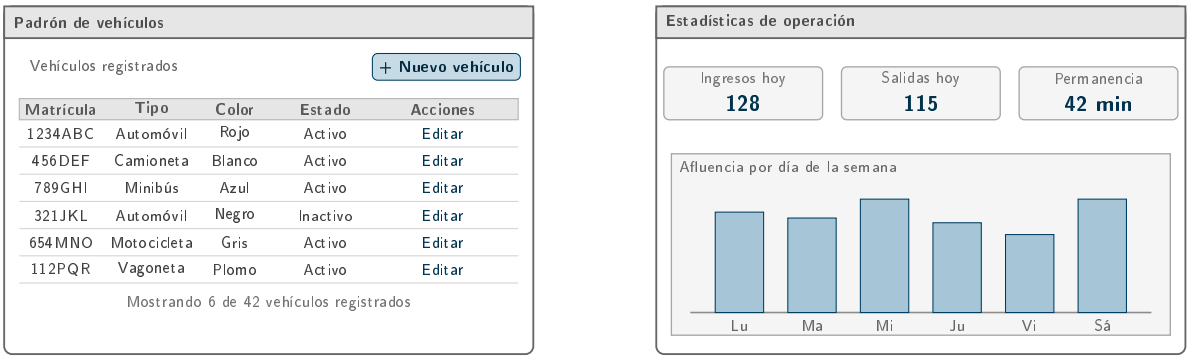
(b) Historial de eventos.

Figura 2.5: Maquetas de las vistas de operación.

2.6.3. Vistas de administración y análisis

Las vistas de administración y análisis están reservadas al rol de administrador. La Figura 2.6 presenta sus maquetas. La vista de padrón (Figura 2.6a) permite registrar, modificar y desactivar vehículos autorizados, así como auditar su pertenencia al recinto institucional (RF-10). La vista de estadísticas (Figura 2.6b) resume indicadores agregados de la operación —ingresos y salidas del día y tiempo de permanencia promedio— y presenta la distribución de la afluencia por día de la semana, información que sustenta la toma de decisiones sobre la operación del recinto. La implementación integró armoniosamente esta estructura: la vista de estadísticas nutrió el panel de monitoreo principal (Figura 3.5), la consulta detallada y exportación reside en el historial de sesiones, y el padrón de vehículos se materializó en una vista administrativa completa conectada a la detección en tiempo real. La navegación incorporó además la vista de incidentes y la de telemetría del modelo, y la gestión de usuarios y cámaras, cuyas maquetas se presentan en la Subsección siguiente y cuya materialización se

documenta con capturas del sistema en operación en el capítulo de implementación.



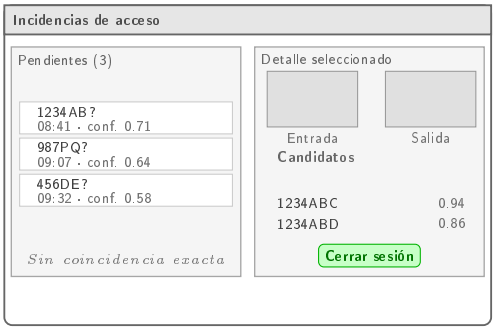
(a) Padrón de vehículos. (b) Estadísticas de operación.

Figura 2.6: Maquetas de las vistas de administración y análisis.

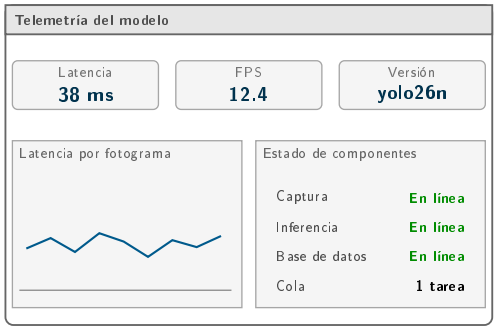
2.6.4. Vistas de incidencia y diagnóstico

El sistema reconoce que el reconocimiento automático de matrículas es probabilístico y que, bajo las condiciones de captura del entorno de estudio, una fracción de las salidas no encontrará coincidencia exacta con una sesión activa. Por esa razón, la interfaz incorpora una vista específica de incidencias (Figura 2.7a) cuyo propósito es resolver la discrepancia en el menor número de interacciones posible. La vista presenta, a la izquierda, la lista de discrepancias pendientes con la matrícula leída, su confianza y la hora; y a la derecha, el detalle de la incidencia seleccionada con los fotogramas de ingreso y salida, la lista de sesiones candidatas ordenadas por aproximación temporal y la acción de cierre manual, que queda registrada con el operador y el motivo.

La vista de telemetría del modelo (Figura 2.7b) completa el conjunto. Está destinada al administrador y al responsable técnico: expone la latencia de inferencia por fotograma, los indicadores agregados de desempeño y la versión del modelo en operación, de modo que una degradación del servicio pueda atribuirse a la captura, a la red o al motor de percepción sin acceso directo a la base de datos.



(a) Incidencias de acceso y cierre asistido.



(b) Telemetría del modelo y estado de componentes.

Figura 2.7: Maquetas de las vistas de incidencia y diagnóstico.

Capítulo III

Implementación y Pruebas del Sistema

3.1. Introducción

El presente capítulo documenta la materialización del diseño especificado en el capítulo anterior, así como la verificación experimental de su desempeño. Su propósito es doble: por un lado, describir la implementación efectiva de cada subsistema —el motor de percepción, la plataforma de gestión y la interfaz de usuario— junto con las decisiones técnicas adoptadas durante la construcción; por otro, presentar los resultados obtenidos en las pruebas realizadas bajo condiciones representativas del entorno de estudio.

A diferencia del capítulo de diseño, que establece la arquitectura conceptual y los criterios de aceptación, este capítulo se apoya en artefactos verificables: fragmentos de código, esquemas de base de datos, archivos de configuración, métricas medidas y capturas de la aplicación en funcionamiento. La trazabilidad entre ambas fases se mantiene mediante los identificadores de requerimiento definidos en las Tablas 2.1 y 2.2, de modo que cada requisito pueda vincularse con el componente que lo satisface y con la prueba que lo verifica.

El capítulo se organiza siguiendo la secuencia de construcción del sistema. En primer lugar, se describe el modelo de implementación: el entorno tecnológico, la arquitectura de despliegue y la organización del repositorio. A continuación, se documenta la construcción del conjunto de datos, etapa habilitante para el entrenamiento de los modelos. Posteriormente, se detalla la implementación del subsistema de visión artificial y del subsistema de gestión. Luego se presenta la estrategia de optimización y despliegue en hardware de borde. Finalmente, se

exponen el protocolo experimental y los resultados obtenidos, y se muestran las interfaces del sistema en operación.

3.2. Modelo de Implementación

Este apartado documenta el entorno tecnológico sobre el que se construyó el sistema, la forma en que sus componentes se despliegan y la organización del código fuente. La descripción es deliberadamente concreta: frente al capítulo de diseño, que establece la arquitectura conceptual, aquí se registran las tecnologías efectivamente empleadas y su disposición física, de modo que la implementación pueda reproducirse sobre otro entorno.

3.2.1. Entorno tecnológico y dependencias

La selección de tecnologías responde a las restricciones identificadas en el diseño: la necesidad de ejecutar inferencia profunda sobre hardware con recursos limitados, de desacoplar el trabajo pesado de la atención de solicitudes y de mantener una única fuente de verdad para los datos. La Tabla 3.1 resume el entorno tecnológico.

Cuadro 3.1: Entorno tecnológico del sistema

Componente	**Tecnología y versión**	**Rol en el sistema**
Lenguaje y entorno	Python 3.12; gestor de dependencias *uv* con archivo de bloqueo	Servicios, trabajador y herramientas de datos
Servicio web	FastAPI ≥ 0.115; Uvicorn ≥ 0.32	API REST y documentación automática de la interfaz
Acceso a datos	SQLAlchemy ≥ 2.0 en modo asíncrono; asyncpg ≥ 0.30	Persistencia no bloqueante desde el servicio
Migraciones	Alembic ≥ 1.14	Versionado del esquema de datos
Base de datos	PostgreSQL 16	Almacenamiento relacional de vehículos, sesiones y detecciones

Continuación de la Tabla 3.1

Componente	**Tecnología y versión**	**Rol en el sistema**
Cola de tareas	Celery $\geq$ 5.4; Redis $\geq$ 5.1	Ejecución asíncrona de la inferencia
Detección de objetos	Ultralytics (YOLO26) $\geq$ 8.4	Localización de vehículos y placas de matrícula
Aprendizaje profundo	PyTorch $\geq$ 2.4; torchvision $\geq$ 0.19	Entrenamiento e inferencia de los modelos
Visión clásica	OpenCV (sin interfaz gráfica) $\geq$ 4.10	Captura, preprocesamiento y calibración de zonas
Seguimiento	ByteTrack mediante *lapx* $\geq$ 0.5.2	Asociación temporal de detecciones entre fotogramas
Reconocimiento de texto	PaddleOCR $\geq$ 2.8; PaddlePaddle $\geq$ 3.0	Pseudo-etiquetado de placas y comparación de referencia
Optimización de modelos	ONNX, onnxslim, ONNX Runtime GPU y TensorRT	Exportación y aceleración de la inferencia
Notificación	*websockets* $\geq$ 13	Canal persistente con el panel de monitoreo
Interfaz web	React 19; Vite 6; Tailwind CSS 3.4	Aplicación del personal del parqueadero
Estado del cliente	TanStack Query 5; TypeScript 5.7	Sincronización con la API y tipado del cliente
Contenerización	Docker; Docker Compose	Despliegue reproducible de los seis contenedores

Fuente: Elaboración propia

La gestión de dependencias se realiza con un archivo de bloqueo que fija las versiones exactas de cada paquete, de modo que la instalación resulte reproducible. La resolución se restringe explícitamente a Python 3.12, dado que las versiones posteriores del intérprete

imponen una versión de la biblioteca numérica incompatible con la requerida por el motor de reconocimiento de texto.

3.2.2. Arquitectura de despliegue

El sistema se despliega como un conjunto de seis contenedores que comparten una red interna y se comunican por nombre de servicio: cuatro servicios de aplicación —API, trabajador, planificador y cliente web—, una base de datos y un intermediario de cola. La Figura 3.1 presenta su disposición; la Tabla 3.2 detalla la función de cada uno y sus dependencias de arranque. El planificador se omite de la figura por claridad: su única función es disparar la rutina diaria de retención.

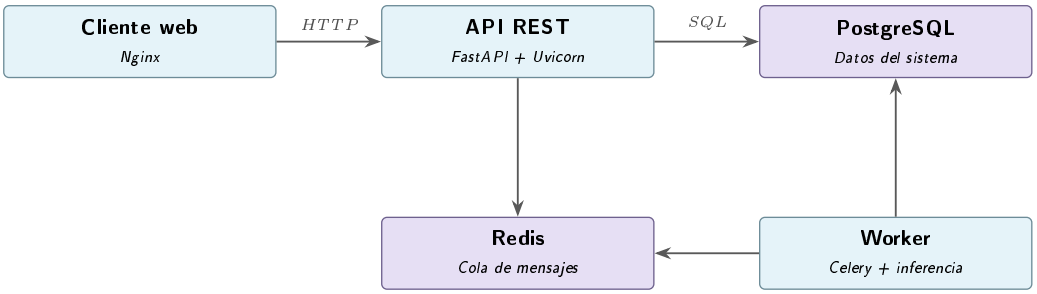


Figura 3.1: Arquitectura de despliegue en contenedores.

Cuadro 3.2: Servicios de despliegue y sus dependencias

Servicio	Tecnología	Función y dependencias
`api`	FastAPI sobre Uvicorn	Expone los recursos REST y el canal de notificación. Depende de que la base de datos esté disponible y de que la cola se encuentre en ejecución.
`worker`	Celery	Consume las tareas de inferencia encoladas y actualiza los resultados en la base de datos. Comparte las mismas dependencias de arranque que el servicio.

Continuación de la Tabla 3.2

Servicio	Tecnología	Función y dependencias
`beat`	Celery Beat	Ejecuta la rutina diaria de retención de telemetría y recortes. Depende de la base de datos y de la cola; su caída no detiene la inferencia.
`postgres`	PostgreSQL 16	Almacena el estado del sistema sobre un volumen persistente. Declara una comprobación de salud que condiciona el arranque de los servicios que la consumen.
`redis`	Redis 7	Actúa como intermediario de mensajes y como almacén de resultados de las tareas.
`frontend`	React servido por Nginx	Entrega la aplicación web compilada y depende del servicio de la API.

Fuente: Elaboración propia

Los servicios de aplicación se construyen a partir de imágenes base de Python 3.12 en dos etapas: una de compilación, en la que se resuelven las dependencias mediante el gestor de paquetes, y otra de ejecución, que recibe únicamente lo estrictamente necesario. El cliente web se compila en una imagen de Node y se sirve como contenido estático sobre Nginx. Los datos se conservan en volúmenes declarados explícitamente, de modo que la reconstrucción de los contenedores no suponga pérdida de información.

El trabajador de inferencia se ejecuta con una única unidad de concurrencia, ya que cada tarea satura la unidad de procesamiento gráfico disponible. La reservación del dispositivo gráfico para este contenedor se declara en el archivo de composición y se habilita según el entorno de despliegue: en una estación de trabajo con acelerador dedicado se activa, mientras que en un entorno sin aceleración el trabajador opera sobre el procesador.

3.2.3. Estructura del repositorio y organización del código

El código se organiza en cuatro áreas: el servicio de gestión, el trabajador de inferencia, el cliente web y las herramientas de datos y de despliegue de modelos. La Tabla 3.3 describe la responsabilidad de cada directorio.

Cuadro 3.3: Organización del código fuente

Ruta	**Responsabilidad**
`backend/app/api/routes/`	Definición de los recursos HTTP y del canal de notificación.
`backend/app/models/`	Modelos de datos: vehículo, sesión de estacionamiento y detección.
`backend/app/schemas/`	Contratos de entrada y salida de la interfaz de programación.
`backend/app/core/`	Configuración, sesión de base de datos y cliente de la cola.
`backend/app/main.py`	Composición de la aplicación y gestión del ciclo de vida.
`backend/worker/`	Aplicación Celery y tareas de inferencia.
`backend/alembic/`	Migraciones versionadas del esquema.
`backend/tests/`	Pruebas automatizadas del servicio.
`frontend/src/pages/`	Vistas del panel de monitoreo y del historial.
`frontend/src/components/`	Componentes reutilizables de la interfaz.
`frontend/src/hooks/`	Enlaces del cliente, incluido el canal persistente con el servicio.
`frontend/src/services/`	Acceso a la API y gestión del canal de notificación.
`rtsp-capture/`	Captura de flujos RTSP y despacho de tareas de inferencia.

Continuación de la Tabla 3.3

Ruta	Responsabilidad
`tools/`	Calibración de zonas, extracción del conjunto de datos y exportación de modelos.

Fuente: Elaboración propia

La separación entre el servicio de gestión y el trabajador de inferencia se refleja en el propio árbol de directorios: el primero concentra las reglas de negocio y el acceso a los datos, mientras que el segundo aloja exclusivamente la lógica de procesamiento. Las herramientas del directorio `tools` se ejecutan fuera del ciclo de operación y no forman parte de los servicios desplegados.

3.3. Construcción del Conjunto de Datos

El conjunto de datos se construye a partir del material de video registrado en el propio recinto, en lugar de reutilizar un conjunto público. Esta decisión responde a que las condiciones que determinan el desempeño del sistema —perspectiva de montaje, rango dinámico de la cámara, formato y tipografía de las matrículas locales, patrones de iluminación del sitio— no están representadas en los conjuntos de uso general. El costo de esta decisión es la necesidad de construir el etiquetado desde cero, lo que motiva el diseño de un proceso de extracción y generación asistida que se describe en este apartado.

El proceso no consiste en muestrear fotogramas al azar, sino en localizar primero los intervalos en los que ocurre algo relevante, reconstruir después la trayectoria completa de cada vehículo y conservar únicamente los fotogramas con potencial de contener una placa legible. La Figura 3.2 presenta el proceso, organizado en dos pasadas, y la Tabla 3.4 enumera los parámetros que lo gobiernan.

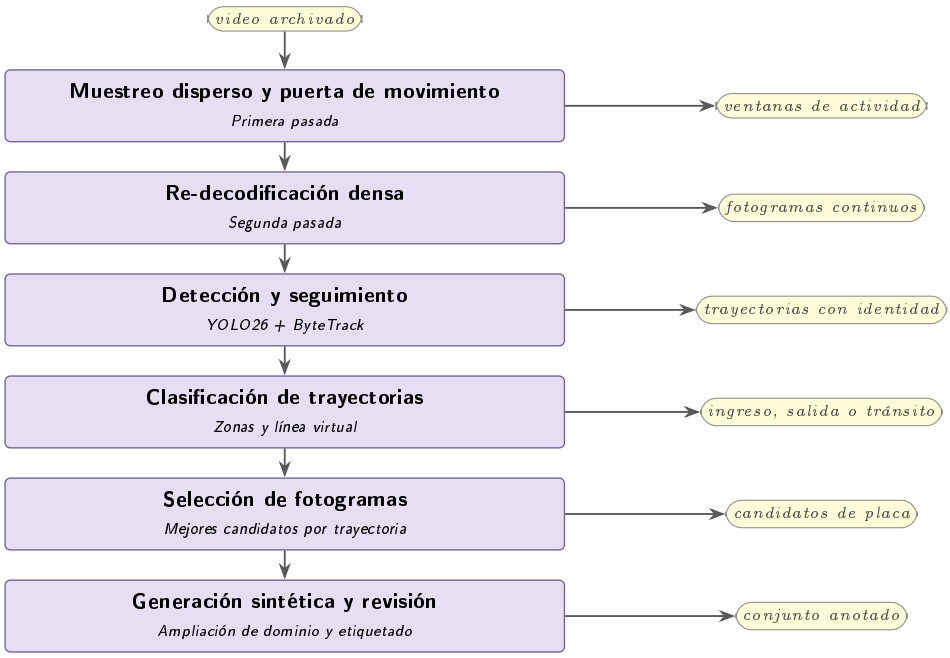


Figura 3.2: Pipeline de construcción del conjunto de datos.

Cuadro 3.4: Parámetros de configuración del extractor

Parámetro	Valor	Función
`scan_stride`	12	Separación entre muestras en la primera pasada.
`motion_downscale`	0.25	Reducción aplicada antes de estimar el movimiento.
`motion_pixel_threshold`	25	Diferencia de intensidad que cuenta como cambio.
`motion_threshold`	0.008	Fracción de píxeles cambiados para declarar actividad.
`segment_pad_frames`	45	Margen añadido a cada ventana detectada.
`segment_merge_gap`	60	Separación máxima para fusionar ventanas contiguas.
`dense_stride`	1	Procesamiento fotograma a fotograma en la segunda pasada.
`grace_frames`	30	Fotogramas de tolerancia antes de cerrar una trayectoria.

Continuación de la Tabla 3.4

Parámetro	Valor	Función
`min_track_hits`	3	Observaciones mínimas para aceptar una trayectoria.
`top_k_frames`	10	Fotogramas conservados por trayectoria.
`vehicle_conf`	0.35	Confianza mínima de detección del vehículo.
`min_area_pixels`	3000	Área mínima de la caja del vehículo.
`ocr_min_conf`	0.60	Confianza mínima para aceptar la lectura preliminar de la placa.
`min_plate_px`	40	Altura mínima del recorte de placa.
`min_plate_sharpness`	15.0	Nitidez mínima exigida al recorte de placa.
`plate_y_min` / `plate_y_max`	0.55 / 1.00	Franja vertical esperada de la placa dentro del vehículo.

Fuente: Elaboración propia

3.3.1. Calibración de zonas de interés y línea virtual

La pertenencia de un vehículo al recinto no puede resolverse mediante una única región rectangular: el tránsito que circula por la vía pública y los vehículos que ingresan ocupan la misma banda de la imagen, de modo que la posición por sí sola no discrimina. Por esa razón la geometría se define mediante polígonos en coordenadas normalizadas —independientes de la resolución de captura— que distinguen tres elementos:

- **Zona de vía:** polígono que cubre la calzada pública.
- **Zona interior:** polígono estrictamente posterior al acceso, que representa el espacio dentro del recinto.
- **Línea virtual:** segmento orientado que separa ambas zonas y cuya intersección detecta el cruce.

El punto de referencia empleado para evaluar la pertenencia y el cruce es el *punto de*

contacto con el suelo —el centro del borde inferior de la caja del vehículo— y no su centroide. La elección es física: un vehículo toca la calzada por la parte inferior de su silueta, por lo que ese punto es el único que atraviesa la línea en el mismo instante en que el vehículo lo hace.

La calibración no se realiza por inspección visual del fotograma, sino sobre evidencia. Se dispone de una herramienta de visualización de trayectorias que ejecuta el seguimiento sobre un fragmento de video sin aplicar ningún filtro de zona y superpone todas las trayectorias reconstruidas sobre un fotograma de referencia. Los vehículos que ingresan y los que sólo transitan producen dos familias de curvas visualmente distinguibles, de manera que los polígonos y la línea se trazan a partir de la separación observada entre ambas familias y no de una estimación previa.

Un aspecto particular del entorno de captura es la presencia de información incrustada por el grabador: el equipo superpone una etiqueta de canal y una marca temporal directamente sobre el video. Dado que el motor de reconocimiento de texto interpreta esa información como si fuese una matrícula —llegando a leer el identificador del canal—, la configuración declara regiones normalizadas que se enmascaran antes de invocar al reconocimiento. Esta corrección es condición necesaria para la validez de las muestras extraídas.

3.3.2. Detección de movimiento y segmentación de ventanas

Procesar la totalidad de los fotogramas de un archivo de video de varias horas resulta prohibitive, pero alimentar un seguidor con fotogramas dispersos tampoco es viable: los algoritmos de seguimiento suponen continuidad entre fotogramas consecutivos y pierden las trayectorias cuando esa continuidad no existe. La solución adoptada consiste en separar el problema en dos pasadas.

La primera pasada recorre el video de forma dispersa y calcula, en cada muestra, una señal de movimiento de costo casi nulo: el fotograma se reduce a un cuarto de su tamaño, se convierte a escala de grises, se suaviza y se compara con la muestra anterior. La fracción de píxeles cuya diferencia supera un umbral de intensidad constituye la energía de movimiento. Cuando esa energía supera un valor mínimo, la muestra se marca como actividad.

Los instantes marcados se agrupan en ventanas temporales, a las que se añade un margen por ambos extremos —para no truncar la aproximación del vehículo— y que se fusionan cuando su separación es menor que un umbral, evitando fragmentar un mismo

tránsito en varios segmentos.

La segunda pasada vuelve a decodificar únicamente las ventanas identificadas, esta vez fotograma a fotograma. Sólo en esta pasada se ejecutan la detección y el seguimiento, sobre una secuencia continua. El resultado es una reducción drástica del cómputo: el trabajo pesado se concentra en la fracción del video donde efectivamente hay vehículos.

3.3.3. Selección de fotogramas representativos

El extractor razona sobre la trayectoria completa de cada vehículo y nunca sobre un fotograma aislado. Esta decisión es la que permite distinguir un ingreso de un simple tránsito sin depender de una única imagen: un vehículo que ingresa cruza la línea virtual en dirección al recinto y su desplazamiento es predominantemente en profundidad, con un aumento progresivo de su escala; un vehículo que sólo transita mantiene una dirección horizontal y nunca alcanza la zona interior.

Las trayectorias con menos observaciones que el mínimo establecido se descartan por considerarse ruido de detección. Para las restantes se conservan únicamente los mejores fotogramas, seleccionados mediante un indicador económico de legibilidad de placa. Ese indicador no evalúa el vehículo completo, sino su parte inferior —donde se ubica la placa—, donde mide la nitidez mediante la varianza del laplaciano, y pondera el resultado por el tamaño del vehículo. Los recortes que ocupan casi la totalidad del encuadre se penalizan deliberadamente, porque corresponden a vehículos tan próximos que su placa queda fuera de cuadro o recortada.

Sobre los fotogramas así elegidos se aplican además restricciones de calidad: se exige una altura mínima del recorte de placa, una nitidez mínima y que la placa se encuentre dentro de la franja vertical esperada. Los fotogramas que no satisfacen estas condiciones no se descartan de inmediato, sino que se derivan al proceso de revisión descrito a continuación.

3.3.4. Pseudo-etiquetado, generación sintética y etiquetado humano

Para garantizar la fiabilidad del entrenamiento y evitar la circularidad metodológica, el etiquetado se organiza bajo reglas estrictas según el tipo de atributo:

1. **Pre-etiquetado y revisión de matrículas:** El motor PaddleOCR actúa exclusivamente como pseudo-etiquetador preliminar. Sus detecciones pasan a una carpeta de revisión (`review/`) para ser verificadas o corregidas por un humano antes de ingresar al conjunto de datos oficial. Para filtrar ruido antes de la revisión, se descartan cadenas sin dígitos (que corresponden a nombres de marcas impresas en la carrocería) y recortes con relaciones de aspecto ajenas al estándar rectangular.

2. **Etiquetado humano directo de atributos (tipo y color):** Los atributos visuales del vehículo (categoría de carrocería y color predominante) son etiquetados en su totalidad por un operador humano mediante una herramienta interactiva especializada (`label_type.py`). Se prohíbe explícitamente el uso de clasificadores automáticos para generar estas etiquetas: hacerlo implicaría entrenar el modelo final con las predicciones de una regla contra la cual se pretende evaluar su desempeño, incurriendo en una invalidez científica.

3. **Generación sintética de matrículas (pre-entrenamiento):** Debido a que la recolección de placas reales requiere procesar cientos de horas de video, se implementa un generador sintético (`generate_synth_plates.py`). Este módulo renderiza la tipografía, colores y símbolos oficiales de las placas vehiculares de Bolivia (formatos `NNNNLLL` y `NNNLLL` con leyenda nacional) y aplica un *pipeline* de degradación física (deformación de perspectiva, variación fotométrica, sombras sintéticas, desenfoque de movimiento y ruido de sensor) ajustando su tamaño a la distribución real observada (mediana de 96×39 píxeles). Las muestras sintéticas se identifican en el manifiesto con la marca `synth: true` y se utilizan exclusivamente durante la fase de pre-entrenamiento, jamás para la evaluación final.

La salida del proceso se organiza según la Tabla 3.5.

Cuadro 3.5: Organización de la salida del extractor y generador

Salida	Contenido
`vehicles/images/`	Recorte del vehículo con la placa visible.
`vehicles/labels/`	Caja delimitadora de la placa en coordenadas del recorte.
`plates/images/`	Recorte ajustado de la placa, destinado al OCR.
`plates/labels/`	Texto de la matrícula confirmado por el operador.

Continuación de la Tabla 3.5

Salida	Contenido
`attributes/labels.jsonl`	Etiquetas de tipo y color asignadas por el operador.
`review/`	Lecturas de baja confianza o mala nitidez, pendientes de revisión.
`negatives/`	Recortes de vehículos que transitan por la vía sin ingresar.
`manifest.jsonl`	Registro central de muestras (etiqueta real o `synth: true`).

Fuente: Elaboración propia

3.3.5. Participación, composición y protocolo de ablación del conjunto de datos

La partición entre entrenamiento, validación y prueba se establece por sesión de grabación, agrupada en bloques horarios, y no de forma aleatoria por imagen. Un mismo vehículo aparece en decenas de fotogramas consecutivos; si esos fotogramas se distribuyeran al azar, el modelo memorizaría el vehículo específico en lugar de aprender a generalizar. Al mantener cada sesión en un único subconjunto, ninguna trayectoria queda repartida entre entrenamiento y prueba, de modo que se elimina la fuga de información por identidad.

Para evaluar el impacto de la generación sintética sobre el reconocimiento de matrículas, se construye un protocolo de ablación determinista (`prepare_ocr_ablation.py`) fijando una semilla aleatoria ($seed = 7$) y una proporción de prueba del 20 %.

La Tabla 3.6 resume la composición del conjunto de datos de reconocimiento de matrículas. La composición completa de los conjuntos de atributos y de placas se detalla en el Anexo B.

Cuadro 3.6: Composición del conjunto de datos de reconocimiento de matrículas

Conjunto	Muestras	Propósito y origen
`train_real`	1,085	Muestras reales extraídas de las sesiones de grabación.
`train_synth`	360,000	Muestras sintéticas generadas con formato boliviano.
`test` (evaluación)	152	Muestras reales independientes para la prueba final.
Total real procesado	**1,237**	640 **placas distintas, particionadas por cadena de matrícula.**

Fuente: Elaboración propia

Además de la partición de entrenamiento y prueba del reconocedor, la evaluación de extremo a extremo se realizó sobre 99 recortes de vehículo con texto de placa verificado por un operador, provenientes de los recortes del conjunto de prueba que conservan la imagen original del vehículo. Ese conjunto no participa del entrenamiento y se describe en el Anexo B. El conjunto de color, por su parte, excluye los minibuses y la clase residual *otro*, de modo que su tamaño no coincide con el del conjunto de tipo; la composición y el motivo de cada exclusión se detallan en el mismo anexo.

Dos limitaciones del material condicionan el alcance de los resultados. La primera es el volumen de matrículas reales: 640 confirmadas por un operador, de las cuales 541 placas se destinan al ajuste fino, constituyen la restricción dominante del módulo de reconocimiento y no la arquitectura del modelo (Sección 3.7.6). Los protocolos de extracción están diseñados para escalar la recolección sin alterar la metodología ni los criterios de evaluación. La segunda es la procedencia: todas las sesiones provienen de las mismas cámaras del recinto, de modo que la partición por sesión controla la fuga por identidad dentro de una misma toma, y la partición por cadena de matrícula elimina la fuga de placa, pero no se controla la variabilidad entre modelos de cámara o condiciones de instalación distintas.

La integridad del material se verificó antes de la partición. Durante la transferencia desde el disco externo de trabajo —que presentó errores de entrada/salida en aproximadamente el 1.3 % de los recortes del pool— ocho archivos sintéticos quedaron irrecuperables y otros veinticinco se truncaron. Los treinta y tres se regeneraron de forma determinística: el generador deriva el estado aleatorio de cada placa de su índice, de modo que la misma revisión del código reproduce el mismo espécimen. En los veinticinco casos con etiqueta previa se

verificó que el texto regenerado coincidiera exactamente con el original; en los ocho restantes la revisión se confirmó por continuidad con índices vecinos recuperados. El conjunto final conserva 360,000 especímenes válidos.

El episodio se documenta porque un generador determinístico por índice solo es reproducible para una revisión fija del código. Una modificación posterior del alfabeto de caracteres hace que un mismo índice produzca una placa distinta, y por lo tanto que reanudar una generación interrumpida reescriba el material en vez de continuarlo.

3.4. Implementación del Subsistema de Visión Artificial

Este apartado describe la implementación del *pipeline* de visión artificial, materializando el diseño establecido en la Sección 2.2. A diferencia de soluciones basadas en servicios de terceros o librerías cerradas, los módulos de detección y reconocimiento óptico de caracteres fueron adaptados y entrenados a medida para operar con el formato alfanumérico y las condiciones de captura del proyecto.

3.4.1. Adquisición del flujo de video

La captura de video se implementa como un proceso independiente por cámara (`rtsp-capture/capture.py`). El módulo establece una conexión con el flujo RTSP del grabador digital, aplica un muestreo temporal configurable (`DETECTION_INTERVAL_MS = 500`) para evitar saturar el bus de datos y reencamisa las imágenes hacia la cola de procesamiento.

Para tolerar cortes de red o caídas temporales de energía en el recinto, el capturador incorpora un mecanismo de reconexión automática que reintenta la lectura sin abortar la ejecución del servicio.

3.4.2. Detección de vehículos

La localización de los automóviles en operación se ejecuta mediante el modelo YOLO26n, la variante liviana de la familia, cargada desde los pesos nativos de PyTorch (`yolo26n.pt`) o desde su motor optimizado en TensorRT de acuerdo con la disponibilidad de hardware. La extracción del conjunto de datos, en cambio, emplea YOLO26x, la variante de mayor capacidad, porque se ejecuta fuera de línea y sin restricción de latencia: allí interesa la calidad del recorte, no el costo por fotograma. La separación es deliberada y se sostiene en que el modelo que produce los eventos debe caber en el hardware de borde, mientras que el que construye el conjunto puede ser más pesado.

La detección se restringe en ambos regímenes a las clases de transporte relevantes (`vehicle_classes: [2, 3, 5, 7]`, correspondientes a automóvil, motocicleta, autobús y camión). En operación, el motor aplica la confianza configurada —0.5 por omisión— y descarta cajas menores a 10×10 píxeles. En la extracción offline el umbral desciende a 0.35 y se exige un área mínima de 3000 píxeles cuadrados: allí el criterio se invierte, porque se prefiere conservar un candidato dudoso que perder un vehículo que luego revisa un operador.

3.4.3. Seguimiento vehicular

El seguimiento multiobjeto se implementa integrando el algoritmo ByteTrack (`bytetrack.yaml`). El rastreador mantiene la continuidad espacial y temporal de los vehículos asignando un identificador numérico único (`track_id`) que se conserva a través de los fotogramas.

A diferencia del algoritmo SORT convencional, ByteTrack aprovecha tanto las detecciones de alta confianza como las de baja confianza para recuperar trayectorias durante oclusiones parciales o momentos en que el vehículo pasa bajo sombras pronunciadas. El sistema requiere un mínimo de 3 observaciones válidas (`min_track_hits`) para confirmar la validez de una trayectoria y acepta una tolerancia de hasta 30 fotogramas perdidos (`grace_frames`) antes de eliminar el identificador de la memoria.

3.4.4. Detección de placas de matrícula

El reconocedor CRNN de la Subsección 3.4.5 opera sobre un recorte ajustado de la placa. El trabajador de visión, en cambio, dispone únicamente del recorte del vehículo que entrega el detector de la Subsección 3.4.2. Entre ambos media una etapa de localización que el diseño original delegaba en el motor PaddleOCR; medirla mostró que esa delegación era el eslabón más débil del *pipeline*, por lo que se entrenó un detector propio.

Conjunto de datos y revisión humana

Se partió de los 4,006 recortes de vehículo del pool congelado. Sobre ellos, PaddleOCR actuó como pseudo-etiquetador en el sentido estricto del término: propuso una caja delimitadora por recorte, sin que ninguna de esas cajas ingresara al conjunto de entrenamiento sin revisión humana.

- **Propuestas automáticas:** 727 de 4,006 recortes (18.1 %) contenían una región compatible con una placa. El 81.9 % restante corresponde mayoritariamente a vistas laterales del vehículo, donde la placa no es visible; por lo tanto no son fallos del etiquetador sino negativos reales.
- **Revisión humana:** se revisaron 1,127 recortes (todas las propuestas más una muestra aleatoria de 400 negativos). El operador aceptó, corrigió o descartó cada caja. El resultado fueron 868 recortes con placa (77 %) y 259 sin placa (23 %) en esa primera revisión, a los que se sumaron recortes ceñidos y 196 recortes adicionales revisados a partir de propuestas del detector de placas. De las 700 propuestas automáticas que sí correspondían a una placa, solo 183 (26.1 %) resultaron geométricamente utilizables sin corrección: el etiquetador automático encontraba la placa, pero su caja la recortaba.
- **Participación:** 768 recortes de entrenamiento, 181 de validación y 178 de prueba. La partición se realizó por *bloque de grabación* y no por trayectoria, de modo que el conjunto de prueba contenga horas de captura no vistas durante el entrenamiento. No existe ningún bloque compartido entre entrenamiento y evaluación.

Modelo y resultados

Se ajustó un detector YOLO26n preentrenado en COCO a una única clase (`plate`), durante 100 épocas, con resolución de 640 píxeles y lote de 8. Sobre el conjunto de prueba (178 recortes, 140 placas, 58 bloques de grabación no vistos) se obtuvo precisión 0.986, exhaustividad 0.971, mAP@50 0.990 y mAP@50–95 0.633.

La comparación con el etiquetador automático sobre el mismo conjunto de referencia humana es el argumento que justifica el detector propio: PaddleOCR alcanza una exhaustividad de 80.6 % y una precisión de 96.3 % como localizador, es decir, no encuentra una de cada cinco placas visibles para el operador. El detector entrenado eleva la exhaustividad a 97.1 %.

3.4.5. Reconocimiento óptico de caracteres (CRNN + CTC)

El módulo de reconocimiento de matrículas fue implementado desde cero mediante una arquitectura profunda combinada *ResNet18 + BiLSTM + CTC* (`train_ocr_crnn.py`), reemplazando los motores genéricos de terceros por un modelo adaptado a la tipografía y formato de Bolivia.

Arquitectura del modelo ResNet-CRNN

La red se compone de tres bloques funcionales acoplados:

1. **Extractor de características (Backbone CNN):** Basado en ResNet-18 modificada. Para evitar la pérdida de resolución horizontal crítica en secuencias de texto, se alteró el avance (*stride*) de las capas convolucionales profundas a $(2, 1)$, reduciendo la altura a 1 píxel mientras se preserva el ancho espacial de 48 vectores de características. Se añade una capa de Normalización por Lotes (*BatchNorm1d*) para estabilizar las activaciones antes del bloque recurrente.
2. **Modelador de secuencias (RNN):** Red LSTM bidireccional de 2 capas (*BiLSTM*) con 512 unidades ocultas, encargada de modelar las dependencias contextuales entre caracteres contiguos de izquierda a derecha.

3. **Clasificador y decodificador CTC:** Capa lineal final con una tasa de desacople (*Dropout*) de 0.3 conectada a una función de pérdida CTC (*Connectionist Temporal Classification*). El alfabeto se define con 37 símbolos: 10 dígitos (0–9), 26 letras mayúsculas (*A–Z*) y 1 token de espacio en blanco (*blank*) en el índice 0.

Preprocesamiento: normalización de contraste

Antes de ingresar a la red, cada recorte se normaliza con ecualización adaptativa de contraste (CLAHE, *Contrast Limited Adaptive Histogram Equalization*) sobre el canal de luminancia del espacio CIELAB, con límite de recorte 2.0 y retícula de 4×4 píxeles. El paso no es opcional.

La justificación es empírica y cuantificable. Los recortes reales del parqueadero son capturas nocturnas: su luminancia mediana es de 76.6 contra 106.4 en las sintéticas, y su desviación estándar de 20.7 contra 34.4. Sin normalización, el modelo recibe una distribución de entrada que no vio durante el entrenamiento. Medido sobre 32 placas reales con la misma configuración de entrenamiento y el mismo número de iteraciones, el modelo pasa de 64.3 % a 99.6 % de exactitud por carácter al incorporar CLAHE. En la corrida completa de dos etapas, la diferencia es de 14 % a 67.6 % de exactitud por carácter sobre el conjunto de prueba congelado.

El mismo paso se aplica en producción (`preprocess()` en `backend/adapters/outbound/ml/plate_crnn.py`). Un reentrenamiento del modelo con CLAHE servido sin CLAHE reintroduce exactamente el problema que motivó su incorporación, razón por la cual el preprocesamiento está duplicado en el adaptador y cubierto por una prueba automática.

Estrategia de transferencia en dos etapas (Two-Stage Transfer Learning)

Para superar la limitación del tamaño reducido de las muestras reales, el entrenamiento se ejecuta en dos etapas diferenciadas:

- **Etapa 1 (preentrenamiento sintético):** 2 épocas sobre 360,000 matrículas sintéticas con formato boliviano. Se congelan las capas iniciales del extractor (desde `conv1` hasta `layer2`) y se habilitan `layer3` y `layer4`, con tasas de aprendizaje diferenciadas: $3 \times$

10^{-4} para el backbone y 3×10^{-3} para la cabeza BiLSTM-clasificador. La tasa escala linealmente durante los primeros 1,500 lotes (*warmup*).

- **Etapa 2 (ajuste fino real):** 100 épocas sobre los 1,085 recortes reales de 541 placas, con el backbone completamente descongelado, tasa de aprendizaje con decaimiento programado y aumento de datos geométrico (*crop jitter*): cada recorte se desplaza y escala aleatoriamente entre 0.90 y 1.12 antes de normalizarse. Se guarda el checkpoint con mayor coincidencia exacta y se reporta ese.

Cada una de estas decisiones reemplaza una configuración anterior que fue medida y descartada, y la sustitución se documenta porque el resultado sin ellas era sistemáticamente peor:

- **Congelar todo el backbone** en la etapa 1 limita la exactitud por carácter al 55 %: solo 10.5 de 21.7 millones de parámetros resultan entrenables, y la cabeza no puede reconfigurar características genéricas de ImageNet en representaciones de glifos.
- **Aplicar la tasa de aprendizaje plena desde el primer lote** provoca el colapso de CTC hacia la secuencia vacía: la cabeza recién inicializada cae en el equilibrio de salida en blanco en aproximadamente la mitad de las corridas. Medido sobre cuatro corridas idénticas, la exactitud por carácter del primer epoch fue 11.2 %, 10.0 %, 89.3 % y 92.3 %. El escalado lineal de los primeros 1,500 lotes elimina la variabilidad y fija el primer epoch en 92.3 %. La semilla aleatoria no resuelve el problema: fija los pesos iniciales, pero no la trayectoria numérica.
- **Uniformar la tasa de aprendizaje** en 3×10^{-3} para todos los parámetros —incluido el backbone— provoca el mismo colapso: pérdida plana en 2.83 y 0 % de coincidencia exacta durante 14 épocas consecutivas.
- **Un planificador de tasa por meseta** (*ReduceLROnPlateau*) en la etapa 2 reduce la tasa a 6×10^{-7} en aproximadamente 60 épocas, porque la pérdida es ruidosa. El entrenamiento se congela sin emitir ningún aviso.
- **Entrenar** 30 **épocas reales** equivale a 360 pasos de gradiente para 21.7 millones de parámetros: insuficiente por dos órdenes de magnitud frente a la etapa 1.
- **Extender la etapa 1 a** 5 **épocas** mejora la validación sintética (93.3 % → 94.8 %) y degrada la transferencia real de 83.1 % a 58.5 % de exactitud por carácter. La validación sintética mide ajuste al generador, no transferencia al dominio real, y por eso no se usa como criterio de selección.

El *crop jitter* merece una nota aparte porque su efecto fue el mayor de todos. Los recortes de entrenamiento fueron producidos por la caja de PaddleOCR, ajustada a los glifos efectivamente reconocidos; en producción los recorta el detector propio, con otra geometría. Esa discrepancia se midió sobre 88 recortes de vehículo con texto verificado por un operador: el sistema completo pasó de 59.4 % a 73.7 % de exactitud por carácter al incorporar el aumento geométrico, y de 9.1 % a 26.1 % de coincidencia exacta. Es decir, una parte sustancial del error atribuido al reconocimiento era en realidad sensibilidad al encuadre.

Post-procesamiento sintáctico: medido y desactivado

El sistema incluye un post-procesamiento determinista (`post_process_bolivian_plate`) basado en la gramática oficial, que reescribe las posiciones numéricas y alfabéticas según las confusiones visuales más frecuentes.

Su evaluación sobre el conjunto de prueba congelado arroja un resultado contrario al esperado: reduce la exactitud por carácter de 78.9 % a 78.6 %. La causa es que el formato boliviano admite letras dentro del bloque numérico —por ejemplo `2160ZAC`— y la regla convierte una lectura correcta en incorrecta. En consecuencia, el post-procesamiento queda desactivado por defecto tanto en el entrenamiento como en el adaptador de producción, y se mantiene únicamente como referencia de la regla sintáctica.

3.4.6. Determinación de eventos y trayectorias

La conversión de trayectorias en eventos de acceso se realiza en el módulo `tracking.py`. Se definen dos zonas clave sobre la escena: `street_zone` (vía pública) e `inside_zone` (rampa interior del parqueadero), divididas por la línea virtual de control (`tripwire`).

El sistema monitorea la posición del punto de contacto con el suelo (centro inferior de la caja del vehículo). Un vehículo se clasifica como evento de *ingreso* únicamente si su trayectoria nace o atraviesa la zona de vía y su punto inferior cruza la línea virtual penetrando en la zona interior. Las trayectorias que permanecen paralelas a la vía o no alcanzan la zona interior se clasifican como *tránsito* y se guardan como negativos duros para evitar falsos registros.

3.4.7. Clasificación de atributos del vehículo

Los atributos del vehículo (tipo de carrocería y color) se obtienen procesando los recortes generados por la detección de YOLO con dos clasificadores ResNet18 independientes, entrenados sobre el mismo pool congelado de atributos y con aumento de datos diferenciado por tarea. El clasificador de tipo distingue cinco categorías (automóvil, camioneta, minibús, moto y vagoneta) y admite variación de color, porque su criterio es la silueta; el de color distingue once categorías y aplica únicamente variación de exposición, nunca de saturación ni de tono, porque el tono es la etiqueta que debe predecir. Los resultados medidos de ambos se reportan en las Subsecciones 3.7.3 y 3.7.4.

3.5. Implementación del Subsistema de Gestión

El subsistema de gestión constituye el punto de acceso único al sistema: recibe las imágenes capturadas en los accesos, coordina su procesamiento y expone la información resultante. Este apartado documenta su implementación efectiva y cierra con una declaración explícita del estado de cada componente junto con la evidencia que lo respalda, con el propósito de distinguir con precisión lo construido de lo pendiente.

3.5.1. Modelo físico de datos y migraciones

El esquema físico comprende las seis tablas que materializan el modelo de datos del diseño. Las tres operativas —`vehicles`, `parking_sessions` y `detection_events`— se detallan en la Tabla 2.6; las tres de control —`users`, `cameras` y `authorized_vehicles`— se describen a continuación. La implementación añade las siguientes decisiones que no se derivan directamente del modelo conceptual:

- **Identificadores generados en la aplicación.** Las claves primarias son identificadores únicos universales generados por el servicio antes de la inserción, y no valores secuenciales asignados por el motor de base de datos. Esto permite que el servicio conozca el identificador de la sesión en el mismo instante en que la crea, sin requerir una lectura posterior, y facilita la correlación de registros entre componentes.

- **Procedencia del vehículo preservada.** Cada sesión referencia obligatoriamente a un vehículo, y el borrado del vehículo arrastra sus sesiones asociadas. La clave foránea es la que garantiza que no exista una sesión sin vehículo identificado.
- **Caja delimitadora como documento estructurado.** Las coordenadas de las detecciones se almacenan en un campo de tipo documento estructurado y no en cuatro columnas escalares. La decisión responde a que la caja es un dato de auditoría y no un atributo de consulta: no se filtra por coordenadas, pero interesa conservar su estructura para reconstruir una detección.
- **Estado de la sesión como enumeración.** El ciclo de vida de una sesión se restringe a los valores activa, completada o cancelada, de modo que el motor de base de datos rechace cualquier estado no previsto.
- **Marcas temporales con zona horaria.** Todos los instantes se almacenan con zona horaria y valor por omisión asignado por el servidor, lo que evita ambigüedades cuando el servicio y la base de datos residen en contenedores con configuración regional distinta.

El modelo declara índices sobre la matrícula, sobre el identificador de vehículo y el estado de la sesión, y sobre el identificador de sesión y la cámara en la bitácora de detecciones. Estos índices corresponden a los patrones de consulta efectivos del sistema: recuperar el historial de un vehículo, listar las sesiones activas para calcular la ocupación y reconstruir las observaciones de una visita.

El acceso a los datos se resuelve mediante una dependencia que entrega una sesión asíncrona por solicitud y delimita su transacción: los cambios se confirman al concluir la atención de la solicitud y se revierten si se produce una excepción. Esta política garantiza que una operación que involucra varias escrituras —crear el vehículo y crear su sesión— se aplique de forma atómica. El motor mantiene un grupo de conexiones reutilizables, dimensionado para sostener la concurrencia esperada del servicio sin saturar la base de datos.

El esquema no se crea de forma directa: se materializa mediante cinco revisiones versionadas que se aplican en orden y dejan registro del cambio estructural. La primera establece las tres entidades principales con sus restricciones e índices; la segunda añade el motivo y el operador del cierre manual, que se registran junto al instante de salida previsto en el esquema inicial; la tercera retira columnas que ningún componente produce: el vector de características, que se midió y se descartó como señal de identidad, y los campos de marca, que ningún clasificador entrega; la cuarta incorpora la autenticación y el registro de cámaras, con las tablas `users` y `cameras`; la quinta añade el padrón institucional en `authorized_vehicles`.

Esa tercera revisión merece señalarse porque documenta una decisión: conservar campos que nadie escribe obliga a cada lector del esquema a preguntarse cuáles son reales.

Las entidades de soporte son acotadas. `users` conserva el nombre de acceso único, el hash de la contraseña —nunca la contraseña—, el rol (*admin*, *operator* u *owner*), el estado activo y las marcas de creación y último acceso. El vocabulario se almacena en inglés por convención del código y corresponde, en el dominio del problema, a *administrador*, *operador* y *dueño*, respectivamente. Una restricción de verificación impide un rol fuera del vocabulario y el servicio impide degradar o desactivar al último administrador activo. `cameras` conserva el identificador que publica el capturador, la puerta asignada, la etiqueta, la URL RTSP y el estado activo; el identificador es único porque debe coincidir exactamente con el que etiqueta los eventos y con el que reporta el latido. El latido en sí no se persiste: vive en Redis con expiración propia, de modo que una cámara que deja de reportar se detecta por ausencia y no por un registro obsoleto. Una cámara se considera fuera de línea cuando su último latido supera los 20 segundos.

La aplicación de las cinco revisiones sobre una base vacía es reproducible y forma parte de la verificación automática: el banco de pruebas de integración levanta una base temporal, aplica las migraciones y ejercita sobre ella el ciclo completo de registro y cierre.

3.5.2. Interfaz de programación

La interfaz se expone bajo un prefijo versionado y agrupa sus recursos por área funcional: operación, consulta, autenticación y diagnóstico. El ingreso de fotogramas no utiliza HTTP: el proceso de captura publica la tarea de inferencia en la cola y el trabajador escribe el resultado directamente en la base de datos, de modo que la interfaz no se interpone en el camino crítico. La Tabla 3.7 enumera las veintiséis rutas efectivamente implementadas y sus parámetros; las de gestión de usuarios y cámaras exigen rol de administrador.

Cuadro 3.7: Recursos implementados de la interfaz de programación

Método	**Ruta**	**Función**	**Parámetros**
GET	`/api/v1/stats/active`	Lista los vehículos actualmente dentro del recinto, con sus atributos.	—

Continuación de la Tabla 3.7

Método	Ruta	Función	Parámetros
GET	`/api/v1/stats/history`	Lista las sesiones completadas, ordenadas de la más reciente a la más antigua.	Límite (1–500) y desplazamiento.
GET	`/api/v1/stats/occupancy`	Devuelve el número de sesiones activas y el instante de la consulta.	—
GET	`/api/v1/stats/daily`	Devuelve, por día, el total de sesiones y la duración media.	Número de días (1–90).
WS	`/api/v1/ws/occupancy`	Canal persistente que propaga las variaciones de ocupación.	—
POST	`/api/v1/auth/login`	Autentica al operador y entrega el token de sesión.	Usuario y contraseña.
POST	`/api/v1/auth/logout`	Revoca la sesión vigente.	—
GET	`/api/v1/auth/me`	Devuelve el operador autenticado.	—
GET	`/api/v1/cameras`	Estado, puerta y último latido de cada cámara, incluidas las registradas que nunca reportaron.	—
POST	`/api/v1/cameras`	Registra una cámara en el padrón de puntos de acceso.	Identificador, puerta, etiqueta, URL RTSP y estado.
PATCH	`/api/v1/cameras/{camera_id}`	Actualiza puerta, etiqueta, URL o estado de una cámara.	Campos a modificar.
DELETE	`/api/v1/cameras/{camera_id}`	Elimina el registro de una cámara.	Identificador.
GET	`/api/v1/users`	Lista las cuentas registradas con su rol y estado.	—

Continuación de la Tabla 3.7

Método	Ruta	Función	Parámetros
POST	`/api/v1/users`	Crea una cuenta con rol y contraseña.	Usuario, contraseña y rol.
PATCH	`/api/v1/users/{id}`	Cambia rol, estado o contraseña; impide degradar al último administrador activo.	Campos a modificar.
DELETE	`/api/v1/users/{id}`	Elimina una cuenta; impide el borrado de la propia cuenta y del último administrador.	Identificador.
GET	`/api/v1/users/roles`	Devuelve el vocabulario de roles para que el cliente no lo codifique.	—
POST	`/api/v1/sessions/{id}/close`	Cierra una sesión activa con operador y motivo, y deja registro de auditoría.	Identificador y motivo.
GET	`/api/v1/vehicles/search`	Busca vehículos en una ventana temporal, con filtro blando de color y orden por cercanía al instante recordado.	Ventana, color, tipo, confianza, instante y paginación.
GET	`/api/v1/vehicles/{id}`	Detalle del vehículo con su historial de sesiones.	Identificador.
GET	`/api/v1/vehicles/{id}/image`	Entrega el recorte asociado al vehículo.	Identificador.
GET	`/api/v1/stats/model`	Telemetría de inferencia por versión de modelo: eventos, confianza media y percentiles de latencia.	Ventana en días.
GET	`/api/v1/stats/colors`	Distribución por color y tipo; publica seis colores y agrega el resto en *minority*, con umbral de confianza.	Ventana y confianza mínima.

Continuación de la Tabla 3.7

Método	Ruta	Función	Parámetros
GET	/health, /health/live y /health/ready	Disponibilidad y preparación del servicio para recibir tráfico.	—

Fuente: Elaboración propia

El recurso de historial implementa paginación por límite y desplazamiento —con un máximo de quinientos registros por solicitud— y el de agregados diarios acota el intervalo consultable, evitando que una petición sin restricciones recorra la totalidad de la bitácora. Los contratos de entrada y salida se declaran como tipos validados, de modo que una solicitud malformada se rechace antes de alcanzar la lógica de negocio. La especificación formal de la interfaz se publica de forma automática a partir de esas declaraciones, lo que garantiza que la documentación no se desincronice de la implementación.

El acceso está protegido: los recursos de lectura no responden sin un token de sesión válido, que el cliente conserva en una cookie inaccesible al código del navegador. Las contraseñas no se almacenan: se conserva su hash derivado con *scrypt*, y el inicio de sesión está sujeto a un límite de 10 intentos por ventana de 5 minutos y por dirección de origen. El límite falla abierto si el almacén de conteo no está disponible, porque un servicio caído no debe dejar al operador fuera del sistema que monitorea. La autorización distingue tres roles —administrador, operador y dueño— y la distinción entre recursos de escritura —destinados a los dispositivos de captura— y de consulta —destinados al personal del parqueadero— se mantiene en la autorización y no solo en la nomenclatura de las rutas.

El cierre de una sesión admite dos caminos. El primero es automático: cuando el reconocedor entrega una matrícula, la política de emparejamiento busca entre las sesiones activas una coincidencia exacta de matrícula, exige que ambas lecturas —la del ingreso y la del egreso— superen un umbral de confianza de 0.6 y, si varias sesiones comparten la matrícula, cierra la más reciente. Si no hay coincidencia o la confianza es insuficiente, **no devuelve la mejor aproximación**: deja la sesión abierta. La decisión es deliberada y se midió antes de adoptarla. Se evaluó primero la alternativa barata, usar el extractor del clasificador de tipo como vector de identidad, sobre 400 recortes del conjunto congelado: la similitud promedio entre el mismo vehículo fue 0.860, mientras que entre vehículos distintos el percentil 90 alcanzó 0.973 y el 99 llegó a 0.991. Es decir, el 19.9 % de los pares de vehículos diferentes supera el umbral de 0.95, de modo que ningún punto de corte los separa: el extractor está

entrenado para la silueta, no para la identidad. El segundo camino es el cierre manual por parte del operador, con motivo y responsable registrados en los campos de auditoría. La regla que gobierna ambos caminos es la misma: un egreso sin emparejar es visible en el nivel de ocupación, mientras que un emparejamiento incorrecto vacía el recinto de forma silenciosa.

3.5.3. Procesamiento asíncrono

La aplicación de tareas se configura con la cola como intermediario de mensajes y como almacén de resultados, serialización en formato de texto estructurado y tiempo universal coordinado como zona horaria de referencia. Se declaran dos tareas, correspondientes al procesamiento de una imagen de ingreso y de una imagen de salida, y se impone un límite temporal a su ejecución: cinco minutos como máximo absoluto y cuatro como límite flexible, de modo que una tarea bloqueada no ocupe indefinidamente al trabajador. El registro del inicio de cada tarea se encuentra habilitado, lo que permite distinguir una tarea encolada de una en ejecución.

Una tercera tarea programada, fuera del camino de escritura, ejecuta la política de retención: cada día elimina la telemetría de inferencia y los recortes con más de 90 días, mientras conserva el vehículo y la sesión. La separación es deliberada: la evidencia visual y la latencia tienen valor diagnóstico y costo de almacenamiento, mientras que el registro del acceso debe sobrevivir para la trazabilidad. El planificador que dispara esa tarea se ejecuta como un contenedor propio, de modo que su caída no detiene la inferencia.

Las dos tareas se encuentran implementadas y su cuerpo está separado de la cola para poder ejercitarlo sin intermediario. La tarea decodifica el fotograma, interpreta el instante de captura y delega en un manejador que concentra la lógica: invoca al motor de visión y mide la latencia de esa inferencia, agrega las detecciones por trayectoria hasta confirmar un evento, persiste el recorte, invoca al lector de placas sobre la caja del vehículo y registra la observación de forma atómica junto con la publicación del evento. El instante que se persiste es el de muestreo del fotograma y no el de ejecución de la tarea: un atraso en la cola no debe desplazar un vehículo a una hora posterior. Un fotograma que no puede decodificarse se informa y no produce efecto alguno. La agregación por trayectoria garantiza la idempotencia del camino de escritura: tres fotogramas de un mismo vehículo producen un vehículo y una sesión, no tres.

3.5.4. Notificación en tiempo real

El canal de notificación se implementa mediante un gestor de conexiones que mantiene la lista de clientes suscritos, incorpora una conexión al aceptarla, la retira al cerrarse y difunde un mensaje a todos los suscriptores. La difusión es tolerante a fallos: si el envío a un cliente produce una excepción, esa conexión se marca como inactiva y se retira del conjunto, de manera que un cliente desconectado de forma abrupta no interrumpa la notificación al resto.

Los mensajes se estructuran por tipo, distinguiendo el registro de un ingreso, el de una salida, la actualización del nivel de ocupación y un mensaje de mantenimiento del canal. Esta clasificación permite que el cliente actualice únicamente la porción de la interfaz afectada en lugar de recargar el estado completo.

3.5.5. Aplicación web

El cliente se construye con React y TypeScript sobre Vite, y organiza su navegación en un componente de disposición común que envuelve las vistas y redirige al panel cualquier ruta no reconocida. Se implementan nueve vistas: seis de operación —el acceso con autenticación, el panel de monitoreo con el estado del recinto y el flujo de eventos, el historial de sesiones, la búsqueda de incidentes, el detalle del vehículo con su historial y la vista de desempeño del modelo— y tres de administración —la gestión de usuarios, el registro de cámaras y el padrón de vehículos habilitados—. El acceso a datos se centraliza en un servicio que declara tipos para cada respuesta de la interfaz de programación, de modo que el compilador detecte una discrepancia entre el contrato esperado y el recibido.

La actualización en tiempo real se encapsula en un enlace que establece la conexión con el canal persistente al montar el componente y la libera al desmontarlo, exponiendo el último mensaje recibido. Se implementa además un mecanismo de consulta periódica como alternativa para los datos que no dependen de la notificación, lo que permite que la interfaz permanezca operativa aun si el canal persistente no está disponible.

3.5.6. Estado de implementación del subsistema

La Tabla 3.8 declara el estado de cada componente del subsistema de gestión y la evidencia que lo respalda. Se explicita con el propósito de que la evaluación posterior permita atribuir cada resultado al componente que efectivamente lo produjo.

Cuadro 3.8: Estado de implementación del subsistema de gestión

Componente	**Estado**	**Observación**
Modelo de datos y esquema	Implementado	Las seis tablas (tres operativas y tres de control) se materializan con sus restricciones e índices.
Versionado de migraciones	Implementado	Cinco revisiones aplicadas en orden; la aplicación sobre una base vacía es reproducible y se ejercita en la verificación automática.
Registro de ingreso	Implementado	Crea el vehículo y la sesión, y responde de inmediato. Verificado en pruebas de caso de uso y de integración.
Registro de salida	Implementado	Emparejamiento por matrícula exacta con umbral de confianza, y cierre manual auditado como alternativa.
Control de autorización	Implementado	El contraste contra el padrón de vehículos habilitados (RF-10) se realiza mediante la entidad `authorized_vehicles` y la ruta `/whitelist`.
Recursos de consulta	Implementado	Ocupación, sesiones activas, historial paginado con filtros de portón y fecha (RF-11), agregados diarios, búsqueda de incidentes, detalle de vehículo y padrón.

Continuación de la Tabla 3.8

Componente	Estado	Observación
Tareas de inferencia	Implementado	La tarea decodifica el fotograma, invoca al motor de visión, agrega por trayectoria y persiste los eventos confirmados con su latencia.
Notificación en tiempo real	Implementado	Canal persistente con difusión tolerante a desconexiones.
Aplicación web	Implementado	Nueve vistas: seis de operación (acceso, panel, historial con filtros y CSV, incidentes, detalle del vehículo y desempeño del modelo) y tres de administración (padrón de vehículos RF-10, usuarios y cámaras).
Pruebas automatizadas	Implementado	156 pruebas: unitarias y de integración contra una base real con las migraciones aplicadas; cobertura del 83 % sobre `backend/app`.

Fuente: Elaboración propia

3.6. Optimización y Despliegue en el Borde

Los modelos empleados en el sistema son de gran porte —la variante de mayor tamaño supera los cien megabytes en sus pesos originales—, mientras que el destino de ejecución es un equipo con recursos acotados que debe procesar video de forma continua. Este apartado documenta las transformaciones aplicadas a los modelos para reducir su costo de ejecución y las estrategias que disminuyen el volumen de cómputo requerido, y cierra señalando qué mediciones restan para cuantificar su efecto.

3.6.1. Exportación a formatos intermedios

Los pesos originales de la familia de detectores se distribuyen en un formato orientado al entrenamiento, que no es el más eficiente para inferencia. El primer paso consiste en exportarlos a un formato intermedio de grafo de cómputo que describa la red de manera independiente del marco de trabajo, lo que permite desplegarla sobre distintos ejecutores.

La exportación se realiza con una resolución de entrada fija, condición necesaria para que el compilador posterior pueda especializar las operaciones. Cuando el formato admite el almacenamiento de los pesos como datos externos, el grafo y los parámetros quedan en archivos separados; la Tabla 3.9 registra la huella efectiva de cada formato para la variante empleada en la extracción.

Cuadro 3.9: Huella en disco por formato de modelo

Formato	**Precisión**	**Tamaño**	**Observación**
Pesos (variante pequeña)	FP32	4.8 MB	Alternativa más liviana.
Pesos (variante media)	FP32	42.2 MB	Alternativa intermedia.
Pesos (variante grande)	FP32	113.2 MB	Variante empleada en la extracción.
Grafo (FP32)	FP32	212.9 MB + 214.4 MB	Pesos almacenados como datos externos.
Grafo (FP16)	FP16	106.6 MB	Media precisión.
Motor compilado	FP16	345.5 MB	Específico del dispositivo de compilación.

Fuente: Elaboración propia

La huella del artefacto en media precisión (106.6 MB) frente a la del grafo en precisión simple con pesos externos (212.9 MB + 214.4 MB = 427.3 MB) representa una reducción de aproximadamente el 75 %. Esa diferencia no proviene únicamente del cambio de precisión: FP16, por sí solo, reduciría los pesos a la mitad. La cifra corresponde a la huella efectiva del

artefacto exportado y simplificado, de modo que se reporta como reducción del artefacto y no como efecto aislado de la media precisión. El motor compilado, en cambio, ocupa un espacio mayor que los pesos originales: un motor de inferencia contiene, además de los parámetros, los núcleos de ejecución y las estrategias de paralelización seleccionadas para el dispositivo. Se trata, por tanto, de un intercambio deliberado de espacio en disco por velocidad de ejecución, cuyo efecto sobre la latencia no se midió en este trabajo.

3.6.2. Compilación y ejecución con TensorRT

El formato intermedio se compila finalmente a un motor especializado para el acelerador gráfico de destino. La compilación toma el grafo exportado y la resolución de entrada, y genera un artefacto optimizado que incluye las operaciones fusionadas y los núcleos seleccionados para la arquitectura específica del dispositivo.

La selección del motor se realiza de forma automática: la configuración resuelve, para cada modelo, la existencia de un motor compilado junto a los pesos y lo prefiere cuando está disponible, recurriendo a los pesos originales en los entornos donde el motor no puede ejecutarse. Gracias a esta regla, la cadena de herramientas no requiere modificación alguna al pasar de un entorno de cómputo a otro: basta con compilar el motor una vez en el equipo de destino.

Cabe precisar una limitación relevante para el despliegue: **los motores compilados son específicos del dispositivo y del controlador** con los que fueron generados. Un motor construido para un acelerador determinado no es portable a otro, por lo que cada equipo de destino requiere su propia compilación. Esta característica condiciona la distribución del sistema y se declara en el apartado de limitaciones.

En cuanto a la precisión de ejecución, el motor se compila en media precisión y los pesos que se ejecutan sin compilación se cargan en ese mismo modo. Los motores ya incorporan la precisión con la que fueron compilados, de modo que no se aplica una conversión adicional al cargarlos.

3.6.3. Estrategias de reducción de carga computacional

Más allá de la transformación de los modelos, la arquitectura del proceso reduce el cómputo requerido mediante decisiones de diseño que evitan trabajo innecesario. Las estrategias aplicadas son las siguientes:

1. **Muestreo disperso previo.** La primera pasada del extractor examina una de cada doce muestras del video con una señal de movimiento de costo casi nulo, de modo que la ejecución de la red se reserva para los intervalos con actividad.
2. **Procesamiento restringido a ventanas de actividad.** La segunda pasada decodifica únicamente los intervalos marcados, en lugar del video completo. La reducción es proporcional a la fracción de tiempo con vehículos presentes.
3. **Enmascarado de la información incrustada.** Las regiones donde el grabador superpone la etiqueta de canal y la marca temporal se anulan antes de invocar al reconocimiento de texto, evitando un costo de inferencia que además produciría lecturas espurias.
4. **Selección de los mejores fotogramas por trayectoria.** Sólo los fotogramas mejor puntuados de cada vehículo alcanzan la etapa de reconocimiento, en lugar de la totalidad de los fotogramas en los que aparece.
5. **Filtros de calidad previos al reconocimiento.** Se exige un tamaño mínimo y una nitidez mínima al recorte de placa, y se descartan las detecciones de área inferior al umbral establecido. Estos filtros evitan invocar al motor de reconocimiento sobre recortes sin posibilidad de lectura.
6. **Ejecución asíncrona con concurrencia unitaria.** El trabajador de inferencia se ejecuta con una sola unidad de concurrencia, dado que cada tarea satura el acelerador. Aumentar la concurrencia no reduciría la latencia y, en cambio, competiría por la misma unidad de cómputo.
7. **Media precisión.** Tanto el motor compilado como los pesos que se ejecutan sin compilación operan en media precisión.

El sistema registra, desde la implementación del trabajador, la latencia de inferencia de cada fotograma procesado: cada observación persiste el tiempo que demandó el motor de visión para ese fotograma, de modo que la base de datos ya acumula el dato que el

despliegue necesita. Lo que resta es el banco comparativo que ese registro no reemplaza: la misma secuencia procesada con cada formato de modelo y cada variante de escala, sobre el hardware de destino, junto con la degradación de exactitud atribuible a la media precisión. Esa comparación es la que permite decidir si el intercambio entre velocidad y precisión es aceptable para un punto de acceso concreto, y no se realizó en este trabajo.

3.7. Pruebas y resultados

Este apartado contrasta cada hipótesis específica con mediciones obtenidas sobre conjuntos de prueba que no participaron del entrenamiento. Se explicitan también las hipótesis que **no** pudieron evaluarse: presentar un sistema completo sin declarar qué partes no fueron medidas equivaldría a afirmar más de lo que la evidencia sostiene.

3.7.1. Condiciones experimentales

- **Hardware:** GPU NVIDIA RTX 3050 Laptop (4 GB de memoria de video), almacenamiento NVMe para los 360,488 recortes de patente y para los conjuntos de atributos.
- **Independencia de los conjuntos:** en clasificación de atributos y en detección de placas la partición se realizó por sesión de grabación y por bloque horario, nunca por imagen, verificando que no exista ninguna sesión compartida entre entrenamiento y prueba. En reconocimiento de matrículas la partición es más estricta: se realizó por cadena de matrícula, de modo que ninguna placa aparece simultáneamente en entrenamiento y en prueba. La partición por sesión no es suficiente para este componente porque el 51 % de los recortes comparte su cadena con otra sesión, y el cruce por union-find entre sesiones y placas colapsa en un único componente de 507 de 693 recortes, lo que impide honrar ambas restricciones a la vez. La identidad de placa es la restricción que protege la coincidencia exacta y es la que se conserva.
- **Métricas:** para clasificación se usa el F1-score macro, calculado sobre las clases cuya muestra de prueba alcanza al menos 40 ejemplares (`min_test_support`); las clases por debajo de ese umbral se informan pero no promedian, porque su intervalo de confianza supera los 25 puntos porcentuales. Para reconocimiento de texto se usa la exactitud por carácter y la coincidencia exacta (*Exact Match*), calculada por recorte y acompañada de su equivalente por placa. La exactitud por carácter se calcula como $1-\mathrm{CER}$ promediado

por placa, es decir, un promedio macro que asigna el mismo peso a cada matrícula; no se reporta el agregado micro sobre el total de caracteres. Para detección, precisión, exhaustividad, mAP@50 y mAP@50–95.

- **Intervalos de confianza:** los intervalos que acompañan a cada métrica se aproximan con la varianza binomial de la proporción correspondiente al tamaño del conjunto de prueba. No incorporan la correlación entre recortes de una misma placa ni la varianza entre corridas, por lo que deben leerse como una cota inferior del error.
- **Selección de checkpoint:** en los clasificadores de atributos se eligió la época según el F1 macro de validación restringido a clases con al menos 20 ejemplares. Utilizar el macro sobre todas las clases presentes dejaba la decisión en manos de clases con tres o cuatro imágenes, donde un único acierto desplaza el promedio $1/11 = 0.09$. Corregir ese criterio elevó el F1 macro de color de 0.7937 a 0.8124.
- **Repeticiones:** se ejecutó una corrida por clasificador. La varianza medida entre corridas idénticas es de 1 a 3.4 puntos de F1 macro, de modo que las diferencias menores a ese rango no se interpretan como mejoras.

3.7.2. Detección de vehículos (H1.1): medida

La hipótesis $H_{1.1}$ exige un mAP@50 igual o superior al 80 % para el detector de vehículos. La medición se realizó sobre 337 fotogramas completos extraídos de 55 horas de grabación de las dos cámaras del recinto, con 304 cajas delimitadoras verificadas por una persona.

Construcción del conjunto

El pool descrito en el capítulo anterior contiene recortes de vehículo, no fotogramas completos: una métrica de detección necesita el vehículo en contexto, de modo que el conjunto se construyó sobre las grabaciones. La unidad de muestreo es la *pasada* y no el fotograma. Un vehículo permanece varios muestreos frente al acceso y los fotogramas consecutivos de un mismo vehículo son casi idénticos; muestrear por tiempo produciría decenas de duplicados de pocos vehículos y un intervalo de confianza más estrecho que la evidencia. Se exporta un fotograma por pasada, en el muestreo en que el seguidor confirma el vehículo; como en ese

instante el vehículo todavía se aproxima, la imagen queda en el lado pequeño y la medición resulta ligeramente conservadora.

La población es la misma en las dos cámaras: un vehículo cuenta cuando su punto de contacto con el suelo alcanza la zona interior del acceso, que es la misma regla con la que el extractor decide que un vehículo entró (`zones_config.json`). Aplicarla por igual a los dos encuadres evita dos errores simétricos: descartar la cámara difícil porque su encuadre incluye la calle, y aceptar como muestra el tráfico que simplemente pasa. La cámara de salida aporta 77 pasadas y la de ingreso 260. La diferencia no se interpreta aquí: depende del flujo del recinto en las horas muestreadas y de la posición de la zona calibrada en cada encuadre, no de una diferencia de dificultad.

Anotación y control del sesgo

Las cajas se anotaron por revisión humana sobre propuestas del detector. Esa asistencia acelera el trabajo y, sin control, sesga el resultado: con una caja ya dibujada el anotador deja de buscar los vehículos que el detector no propuso, y el recall deja de ser medible porque los falsos negativos se vuelven invisibles. Para acotar el efecto, una de cada cuatro imágenes se presentó *sin* propuestas y se dibujó desde cero. El 60.5 % de las cajas se aceptaron sin corrección.

Resultado

Cuadro 3.10: Detección de vehículos sobre 337 fotogramas con 304 cajas de referencia humana. Intervalo de AP por remuestreo de imágenes completas (2000 réplicas); intervalo de recall por Wilson sobre cajas.

Modelo	**AP@50**	**IC 95 %**	**Recall@0.5**	**Precisión@0.5**
YOLO26n	0.901	0.876–0.924	0.954	0.628
YOLO26s	0.887	0.861–0.911	0.987	0.581

El límite inferior del intervalo del detector de operación es 0.876**, por encima del** 0.80 **que exige** $H_{1.1}$**: la hipótesis se respalda.**

El control de sesgo sostiene el número. Sobre el subconjunto anotado sin propuestas (63 imágenes) el AP@50 desciende a 0.876 ([0.802; 0.939]), frente a 0.906 ([0.880; 0.931]) en el subconjunto asistido. La diferencia de tres puntos acota el efecto de la asistencia: existe, es medible y no alcanza a cambiar el veredicto.

La comparación entre variantes no se interpreta como una mejora de la variante mayor. YOLO26s obtiene mayor exhaustividad (0.987 frente a 0.954) pero peor precisión (0.581 frente a 0.628) y menor AP@50, de modo que el detector de operación se conserva. El resultado también descarta que el tamaño del modelo sea la palanca: los clasificadores de atributos, que se ejecutan una vez por vehículo, dominan el costo por fotograma (Subsección 3.7.9).

La precisión a umbral 0.5 merece una advertencia: casi cuatro de cada diez detecciones por encima del umbral no corresponden a un vehículo etiquetado. En operación eso no se traduce en eventos espurios, porque el filtro de zona y la línea de cruce descartan el tráfico de la calle antes de abrir una sesión, pero el dato se reporta porque acota lo que el detector aporta por sí solo.

Consecuencia para las etapas posteriores

El error de la etapa de detección se propagaba al resto del pipeline sin cuantificar. Ahora está acotado: una detección perdida no genera evento, de modo que el techo de la determinación de eventos no puede ser superior a la exhaustividad medida aquí (0.954 a umbral 0.5).

3.7.3. Clasificación de tipo de vehículo (H1.2)

El conjunto de prueba contiene 633 recortes de 5 clases. La taxonomía excluye la clase *furgón*, fusionada durante el desarrollo: 98 recortes etiquetados como tal se reauditaron individualmente y se reasignaron a *camioneta* (23), *vagoneta* (20) y *minibús* (6), descartándose 49 por no ser identificables.

Cuadro 3.11: Clasificador de tipo: métricas por clase sobre el conjunto de prueba ($n = 633$).

Clase	Precisión	Exhaustividad	F1	n
automóvil	0.838	0.748	0.791	159
camioneta	0.880	0.880	0.880	83
minibús	0.949	0.937	0.943	79
moto	0.979	1.000	0.989	46
vagoneta	0.816	0.868	0.842	266
F1 macro			**0.8887**	633
Exactitud			**0.8578**	633

El error está concentrado en un único par: de 90 clasificaciones incorrectas, 62 (69 %) corresponden a la confusión entre *automóvil* y *vagoneta* (39 automóviles predichos como vagoneta, 23 en sentido inverso). Es la confusión esperable entre dos siluetas de turismo que, desde una cámara de tráfico, difieren esencialmente en la altura del techo y el volumen trasero.

Contraste: $H_{1.2}$ exige F1 macro ≥ 0.85; el valor medido es 0.8887. **La hipótesis se respalda.**

3.7.4. Clasificación de color predominante (H1.3)

El conjunto de prueba contiene 551 recortes distribuidos en 11 clases, de las cuales solo seis alcanzan el soporte mínimo de 40 ejemplares. El conjunto de color no incluye minibuses: su librea hace que el color dominante dependa del ángulo de captura y la etiqueta no sería comparable con la pintura base de un vehículo particular (Anexo B). Las métricas de este componente describen, por tanto, automóviles, camionetas, motocicletas y vagonetas, no la flota completa.

Cuadro 3.12: Clasificador de color sobre el conjunto de prueba. Las clases con $n < 40$ se listan solo para dejar constancia de su existencia.

Clase	Precisión	Exhaustividad	F1	n	Evaluada
blanco	0.883	0.917	0.900	132	sí
gris	0.778	0.575	0.661	73	sí
plata	0.824	0.800	0.812	105	sí
negro	0.703	0.738	0.720	61	sí
rojo	0.949	0.921	0.935	101	sí
azul	0.887	0.810	0.847	58	sí
verde	0.500	0.545	0.522	11	no
amarillo	0.000	0.000	0.000	1	no
marrón	0.000	0.000	0.000	2	no
beige	0.083	0.250	0.125	4	no
naranja	0.600	1.000	0.750	3	no
F1 macro (evaluadas)			**0.8124**	530	
F1 macro (11 clases)			**0.5702**	551	
Exactitud			**0.8022**	551	

Todo el error se concentra en el bloque acromático. Las confusiones dominantes son *gris* ↔ *plata* (9 y 5 casos), *gris* ↔ *negro* (5 y 4) y *plata* ↔ *blanco* (12 y 5). Separar blanco, plata, gris y negro en capturas nocturnas es un problema de nivel de brillo, no de tono, y el clasificador no dispone de una referencia de exposición absoluta: el mismo vehículo gris claro aparece casi blanco bajo iluminación directa y casi negro a la sombra.

Contraste: $H_{1.3}$ exige F1 macro ≥ 0.85. El valor que el protocolo de evaluación del proyecto reporta es 0.8124, calculado sobre las seis clases con soporte suficiente. Queda 3.8 puntos por debajo del umbral y el intervalo del 95 % de la métrica, obtenido por remuestreo estratificado del conjunto de prueba ($[0.777;\ 0.844]$), queda íntegramente por debajo de 0.85: el incumplimiento no se explica por el error de muestreo del test, aunque el margen es el más estrecho del contraste y no supera el punto porcentual. **La hipótesis no se respalda** y se acepta la hipótesis nula $H_{0.3}$ para este componente, con dos precisiones: se trata del

término más débil del contraste, y su distancia al umbral es menor que la varianza medida entre corridas de entrenamiento (1 a 3.4 puntos, Subsección 3.7.11), de modo que el veredicto describe este modelo entrenado y no la configuración en abstracto.

El macro calculado sobre las once clases presentes en el conjunto de prueba —incluidas cinco con menos de 40 ejemplares— arroja 0.5702. Ese valor no se utiliza para el veredicto: el protocolo del proyecto excluye del promedio las clases cuyo intervalo de confianza supera los 25 puntos, porque un puñado de ejemplos hace que el promedio describa el muestreo y no el modelo. Se informa únicamente para dejar constancia de la diferencia entre ambas lecturas.

La conclusión general de la hipótesis no depende de este término: se apoya en $H_{1.4}$ y, dentro de él, en la coincidencia exacta (0.526 frente a la referencia de 0.80), que es el término con mayor distancia al umbral.

3.7.5. Detección de placas de matrícula (H1.4, componente de detección)

Sobre el conjunto de prueba de 178 recortes de vehículo (140 con placa visible) provenientes de 58 bloques de grabación no vistos durante el entrenamiento:

Cuadro 3.13: Detector de placas frente al etiquetador automático, sobre el mismo conjunto de referencia humana.

Sistema	**Precisión**	**Exhaustividad**	**mAP@50**	**mAP@50–95**
YOLO26n ajustado	0.9857	0.9714	0.9898	0.6329
PaddleOCR (localizador)	0.9630	0.8060	—	—

Contraste: $H_{1.4}$ exige una exhaustividad $\geq 90\,\%$ en la detección. El valor medido es 0.9714. **El componente de detección se respalda.**

El dato que justificó entrenar un detector propio es la diferencia entre encontrar la placa y delimitarla correctamente: de las 700 propuestas automáticas que sí correspondían a una placa, solo 183 (26.1 %) resultaron utilizables sin corrección humana. El etiquetador localiza, pero su caja recorta los glifos.

3.7.6. Reconocimiento de placas (H1.4, componente de reconocimiento)

Sobre recortes de placa aislados

Conjunto de prueba congelado de 152 recortes correspondientes a 99 placas distintas, sin ninguna placa compartida con el conjunto de entrenamiento y sin aumento de datos en la evaluación:

Cuadro 3.14: Reconocimiento sobre recortes de placa aislados ($n = 152$, 99 placas, partición por placa).

Modelo	**Exactitud por carácter**	**Exact Match**
CRNN (ResNet18 + BiLSTM + CTC)	0.851	0.526
PaddleOCR	0.540	0.237

El resultado depende fuertemente de la resolución del recorte, y esa dependencia es la que fija el techo del módulo:

Cuadro 3.15: Exactitud por carácter según la altura del recorte, CRNN. Mismo modelo en todos los tramos.

Altura del recorte	**n**	**Char. accuracy**	**Exact Match**
< 20 px	4	0.643	0.000
20–30 px	15	0.781	0.467
30–40 px	26	0.841	0.577
≥ 40 px	107	0.872	0.542

Un recorte de 40×13 píxeles contiene siete caracteres, es decir del orden de cinco píxeles de ancho por glifo: la información no está en la imagen y ningún modelo puede reconstruirla. Los recortes por debajo de 20 píxeles de altura —cuatro en este conjunto— arrastran el promedio global, aunque su peso es menor que en la versión anterior del test (4 frente a 17) gracias a la revisión que eliminó los recortes ilegibles.

Sobre recortes de vehículo (sistema completo)

La medición anterior asume un recorte perfecto de la placa. La medición del sistema ensamblado —detector propio más CRNN— se realizó sobre los 99 recortes de prueba que disponen de la imagen original del vehículo (los 53 restantes son recortes ceñidos sin imagen de origen). En todos los casos el texto fue verificado manualmente por un operador:

Cuadro 3.16: Sistema completo sobre 99 recortes de vehículo con texto verificado. Detector y lector reciben exactamente la misma entrada. Métricas por recorte; los valores con promedio por placa se reportan en el texto.

Configuración	**Exactitud por carácter**	**Exact Match**
Detector propio + CRNN (producción)	0.882	0.566
Caja humana + CRNN	0.854	0.535

El detector localizó la placa en 99 de 99 casos de esta muestra (100 %). La caja del detector supera a la caja humana en ambas métricas (+2.8 puntos de exactitud por carácter, +3.1 de coincidencia exacta): el encuadre guardado en el conjunto de prueba proviene de un recorte con márgenes fijos, mientras que el detector adapta la caja a cada imagen. El resultado confirma que el error de localización del detector no penaliza al reconocedor y que la medición sobre recortes de placa aislados es, en este caso, una cota inferior conservadora del sistema ensamblado.

Contraste: $H_{1.4}$ es conjuntiva y exige, simultáneamente, exhaustividad de detección $\geq 90\,\%$ y exactitud por carácter $\geq 90\,\%$. El primer término se cumple (0.9714). El segundo no se alcanza: sobre los 152 recortes, agrupados en 99 placas, la exactitud por carácter promediada por placa es 0.851 y su intervalo por remuestreo de placas es [0.811; 0.890], íntegramente por debajo del umbral. El intervalo no depende de la semilla de remuestreo ni del checkpoint: la última época da [0.811; 0.890] y la mejor época, 0.858 con [0.819; 0.894]; ambos excluyen el 0.90. La referencia de coincidencia exacta confirma la misma dirección con mayor margen. Sobre el sistema ensamblado, evaluado únicamente sobre los 99 recortes de vehículo del encuadre servido —un subconjunto más fácil, que excluye los recortes ceñidos de menor resolución— la exactitud por carácter es 0.882 por recorte y 0.867 con promedio por placa, con intervalo [0.818; 0.910]. El intervalo contiene el umbral, de modo que en el sistema ensamblado el incumplimiento de este término es marginal en vez de concluyente, y su coincidencia exacta por placa (0.539, intervalo [0.429; 0.653]) sigue lejos del 0.80. El criterio

de evaluación del perfil declara *a priori* una coincidencia exacta $\geq 80\,\%$ como referencia de aceptación; sobre los 152 recortes aislados vale 0.526 por recorte ([0.428; 0.621]) y 0.478 por placa ([0.391; 0.569]): el veredicto no depende de la unidad de análisis. **La hipótesis $H_{1.4}$ no se respalda** y se acepta $H_{0.4}$ conforme a la regla de decisión: el componente de reconocimiento no satisface el conjunto de valores mínimos establecidos en el criterio de evaluación, porque la exactitud por carácter queda por debajo del umbral y la coincidencia exacta, más de veinte puntos por debajo de la referencia.

Nota metodológica sobre la selección de época

El conjunto de prueba se evaluó al final de cada época durante el entrenamiento del reconocedor. El valor reportado corresponde a la época con mayor coincidencia exacta sobre ese conjunto (98 de 100), guardada automáticamente como mejor checkpoint: exactitud por carácter 0.851 y coincidencia exacta 0.526. La selección del máximo introduce un sesgo de selección sobre el propio test, por lo que se reporta también el checkpoint de la última época, sin selección: 0.850 de exactitud por carácter y 0.520 de coincidencia exacta. La diferencia entre ambos —0.1 y 0.6 puntos respectivamente— cae dentro del error de muestreo del test ($n = 152$, intervalo al 95 % de ± 3.7 puntos en exactitud por carácter), de modo que el sesgo de selección es despreciable frente a la incertidumbre de la medición. El veredicto de la hipótesis es el mismo con cualquiera de los dos estados.

Evolución del protocolo de evaluación. El conjunto de prueba utilizado en esta sección difiere del empleado en la primera versión del capítulo. La versión anterior constaba de 101 recortes particionados por sesión de grabación; la actual consta de 152 recortes particionados por cadena de matrícula. El cambio responde a un problema de fuga de identidad: el 51 % de los recortes compartía su cadena de matrícula con otra sesión, de modo que la partición por sesión no impedía que el modelo evaluara placas que ya había visto durante el entrenamiento. Los números de ambas versiones no son comparables directamente: la tarea del test actual es estrictamente más difícil, porque ninguna placa aparece en ambos conjuntos.

Una segunda verificación detectó que cuatro placas aparecían a la vez en entrenamiento y prueba: las correcciones de etiqueta aplicadas después de la partición fusionaron cadenas que antes diferían. Los siete recortes de prueba de esas cuatro placas se excluyeron del conjunto de evaluación, que pasó de 159 a 152 recortes. La exclusión no cambia el veredicto y reduce la coincidencia exacta medida en algo menos de un punto ($0.535 \rightarrow 0.526$): mantenerlos habría inflado el resultado con siete recortes cuya placa el modelo ya había visto.

Adicionalmente, el conjunto de entrenamiento real creció en dos pasos sobre el mismo test de 152 recortes. Primero, la curación de los recortes ceñidos —29 etiquetas corregidas y 80 recortes ilegibles eliminados— fijó el entrenamiento en 889 recortes, que sobre este test alcanza 0.480 de coincidencia exacta. Segundo, la revisión de 196 recortes adicionales propuestos por el detector de placas y etiquetados manualmente por un operador llevó el entrenamiento a 1,085 recortes y la coincidencia exacta a 0.526. La comparación entre los dos estados se hace sobre el mismo test y por tanto es limpia; la cifra de 0.831 de la versión anterior del capítulo no es comparable porque proviene del protocolo con fuga.

Estabilidad y transferencia del preentrenamiento sintético

Dos resultados del procedimiento, no del modelo, condicionan la lectura de todo lo anterior. Ambos se midieron sobre el protocolo de evaluación anterior (101 recortes particionados por sesión, 395 recortes reales de entrenamiento) y no se re-ejecutaron tras ampliar el conjunto de datos; se conservan porque caracterizan el mecanismo del preentrenamiento sintético, que no depende del tamaño del conjunto real de ajuste fino. El resto de este capítulo reporta las métricas bajo el protocolo actual.

El arranque de la etapa 1 era inestable. Aplicar la tasa de aprendizaje plena desde el primer lote hacía que la cabeza recién inicializada cayera en el equilibrio de salida en blanco de CTC en aproximadamente la mitad de las corridas. Cuatro ejecuciones con idéntica configuración arrojaron, en su primer epoch:

Cuadro 3.17: Exactitud por carácter del primer epoch de la etapa 1 en ejecuciones idénticas. La diferencia no proviene de la semilla: fija los pesos iniciales, pero no la trayectoria numérica.

Configuración	**Char. accuracy (epoch 1)**	**Resultado**
Tasa plena, semilla libre	0.913	convergió
Tasa plena, semilla libre	0.615	colapso parcial
Tasa plena, semilla fija	0.112	colapso
Tasa plena, semilla fija, determinístico	0.100	colapso
Con escalado en 1,500 lotes	**0.923**	**convergió**

El escalado lineal de la tasa durante los primeros 1,500 lotes elimina la variabilidad: la

corrida reportada en este capítulo es reproducible y su primer epoch queda en 0.923. Forzar el determinismo de las rutinas de GPU sin el escalado no resuelve el problema, solo hace que el colapso sea reproducible.

Más preentrenamiento sintético no transfiere. Extender la etapa 1 de 2 a 5 épocas mejora la validación sintética y degrada el resultado real:

Cuadro 3.18: Efecto de extender la etapa 1. La etapa 2 es idéntica en ambos casos.

Etapa 1	Validación sintética	Prueba real
2 épocas	0.933	**0.831**
5 épocas	0.948	0.585

La validación sintética mide qué tan bien se ajustó el generador, no la transferencia al dominio real. Cada época adicional especializa las características en las fuentes, el antialiasing y la degradación del renderer, que no son las de una placa nocturna real. Por esa razón la etapa 1 se mantiene en dos épocas y la selección de modelos no se apoya en la exactitud sintética.

Por qué el volumen sintético no cierra la brecha de dominio

La etapa 1 dispone de 360,000 matrículas sintéticas frente a los 395 recortes reales con que contaba el ajuste fino en el momento de esa medición: una relación de casi mil a uno a favor del material generado. Si la brecha de reconocimiento fuera de volumen, esa proporción la habría cerrado. No lo hace, y la medición que lo demuestra es directa, porque la etapa 1 se evalúa simultáneamente sobre ambos dominios.

La Tabla 3.19 presenta las cinco épocas de una misma corrida. La exactitud sintética asciende de 0.923 a 0.948 y se estabiliza; la exactitud real oscila entre 0.094 y 0.133 sin tendencia, y la coincidencia exacta permanece en 0.000 en las cinco épocas.

Cuadro 3.19: Etapa 1 evaluada sobre su propio dominio y sobre el dominio real, en una misma corrida de cinco épocas. La coincidencia exacta sobre material real nunca abandona el cero.

Época	Pérdida sintética	Sintética (car.)	Real (car.)	Real (EM)
1	0.778	0.923	0.129	0.000
2	0.201	0.936	0.116	0.000
3	0.179	0.948	0.133	0.000
4	0.176	0.941	0.122	0.000
5	0.182	0.936	0.094	0.000
Media	—	**0.937**	0.119	**0.000**

La Figura 3.3 representa ambas series con ejes independientes. Dentro de la corrida, el coeficiente de correlación de Pearson entre exactitud sintética y exactitud real es $r = 0.10$: la métrica sintética **no predice** el desempeño real. Agrupando las siete mediciones disponibles —las cinco de esta corrida y las dos de la configuración reportada— se obtiene $r = -0.46$, pero ese valor no debe interpretarse como una relación causal inversa: agrupa corridas con presupuestos de entrenamiento distintos y su magnitud depende de un único punto. La lectura robusta es la que no depende de ninguna correlación: **en siete mediciones, sobre dos corridas, la coincidencia exacta sobre material real fue** 0.000.

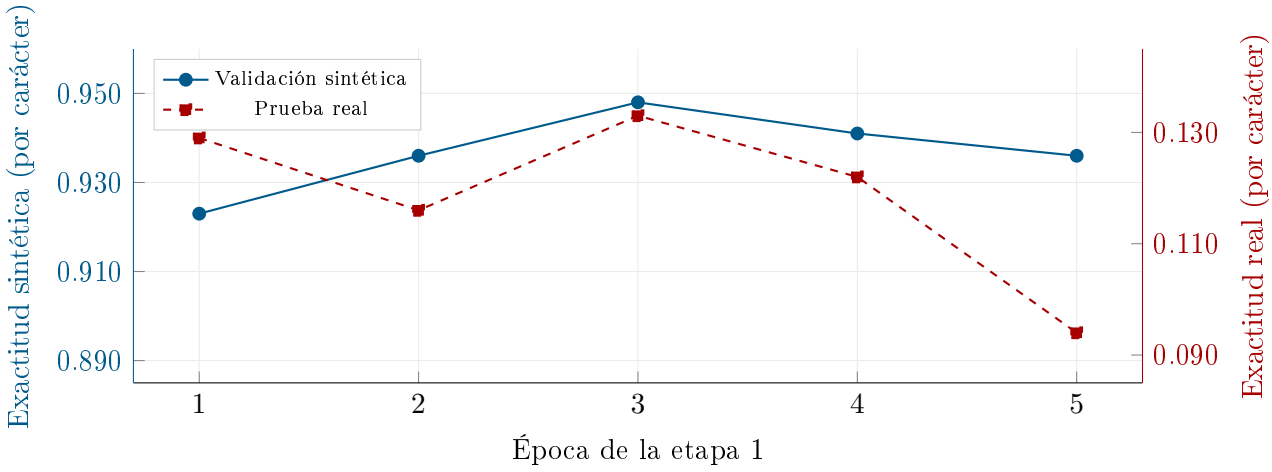


Figura 3.3: Exactitud por carácter de la etapa 1 sobre su propio dominio (eje izquierdo, línea continua) y sobre el dominio real (eje derecho, línea discontinua). La mejora sintética no se traduce en mejora real: las series están desacopladas y la coincidencia exacta real es nula en todas las épocas.

El mecanismo: se aprende el generador, no la matrícula. Un modelo no distingue entre aprender la tarea y aprender el atajo que resuelve el conjunto de entrenamiento. Con 360,000 muestras de un único generador, el atajo más corto es explotar los invariantes del renderizador, que son estables y perfectamente informativos dentro del dominio sintético y están ausentes en el real. Cuatro propiedades del pipeline sostienen ese atajo:

1. **La tipografía es sustituta, no la de la placa.** El renderizador documenta que la fuente es una condensada genérica de reserva; las caras disponibles son DejaVu, Liberation, Verdana y Noto. La red aprende los trazos de esas fuentes. Sobre el relieve real, con otra die y otra separación entre glifos, esas características no se activan.

2. **Falta el proceso físico de fabricación.** No hay relieve, holograma, lámina reflectiva ni tornillos de sujeción. El relieve produce un sombreado interno que un modelo puede usar como señal, y los tornillos producen oclusiones parciales que el generador nunca introduce. La red aprende una placa plana que no existe.

3. **La degradación se muestrea de forma independiente.** El pipeline aplica perspectiva, exposición, sombra, brillo especular, remuestreo, ruido y compresión como extracciones separadas, cada una con su probabilidad. La distribución conjunta resultante es el producto de sus marginales. La degradación real está correlacionada y es estereotípica: el desenfoque de movimiento tiene dirección y depende de la velocidad, la sombra sigue la geometría del portón, el ruido tiene patrón fijo del sensor de esa cámara y la compresión es la del grabador. El modelo aprende a ser robusto frente al promedio de todas las degradaciones posibles, que no es ninguna de las que ocurren. La diferencia de luminancia ya medida —mediana 76.6 real frente a 106.4 sintética, con desviación 20.7 frente a 34.4— es una manifestación de este mismo desajuste.

4. **El remuestreo no equivale al desenfoque óptico.** El generador dibuja un original nítido de 610×204 píxeles y lo reduce por interpolación hasta el tamaño de destino. En la captura real la información se destruye antes del recorte, en la óptica y en la compresión del grabador. El aliasing de una interpolación bilineal y el desenfoque de un sistema óptico no son la misma función de transferencia, y el modelo aprende la primera.

La coincidencia exacta nula aporta la confirmación de que el desajuste es de glifo y no de secuencia: el modelo acierta en promedio uno de cada diez caracteres sobre material real, pero no completa nunca una matrícula de siete. Un error de modelado de secuencia produciría cadenas completas con caracteres permutados; un error de representación del glifo produce exactamente lo observado.

Qué implica para el diseño experimental. La consecuencia no es aumentar el volumen: más muestras del mismo generador amplifican el atajo. La comparación al nivel de corrida lo confirma, porque extender la etapa 1 de dos a cinco épocas mejora la validación sintética ($0.933 \rightarrow 0.948$) y deteriora la prueba real ($0.831 \rightarrow 0.585$). La señal útil para el dominio real está en el material real, que llevó al reconocedor desde 0.148 —el punto de partida que dejó la etapa 1— hasta la exactitud por carácter reportada en la Subsección 3.7.6. Cerrar la brecha exige fidelidad del generador: tipografía de la placa, relieve y su sombreado, degradación correlacionada con la física del acceso, y calibración del remuestreo contra la función de transferencia de la cámara. Ese es el trabajo futuro que la Subsección 5.2.4 enumera.

3.7.7. Determinación de eventos de ingreso y salida (H1.5)

La hipótesis $H_{1.5}$ requiere una exactitud igual o superior al 90 % en la clasificación de eventos de ingreso y salida. La medición se realizó sobre una hora completa de las dos cámaras, reproducida de extremo a extremo por la cadena de operación: video grabado servido por RTSP, captura, cola de tareas, inferencia, persistencia y registro del evento.

Procedimiento

La referencia humana se construyó sobre la grabación, no sobre la reproducción: una persona contó cada cruce del portón y marcó su instante en segundos de video y su sentido. Trabajar sobre el archivo hace que la verdad no dependa del reloj de una tarde de pruebas y que pueda compararse contra cualquier reproducción posterior.

El contraste empareja un evento del sistema con uno de la referencia cuando coinciden en sentido y están a menos de diez segundos. La tolerancia es mayor que en otras métricas de este capítulo por dos razones medidas: el sistema marca el instante en que el punto de contacto entra en la zona del acceso, lo que ocurre algo después del cruce que observa la persona, y la reproducción introduce una holgura de arranque de uno a tres segundos.

La hora empleada en esta medición no intervino en el desarrollo del detector de eventos. La hora usada durante ese desarrollo se reporta aparte, al final de la subsección.

Resultado

Cuadro 3.20: Determinación de eventos sobre una hora completa de las dos cámaras, con 9 cruces de referencia humana. Intervalo de Wilson [71], [72] sobre la proporción de aciertos.

Conjunto	Referencia	Aciertos	Perdidos	Espurios	Exactitud
Ingresos	2	2	0	0	1.000
Salidas	7	6	1	0	0.857
Total	9	8	1	0	0.889

La exactitud medida es 0.889**, con un intervalo del** 95 **% de** [0.565; 0.980]**. El intervalo contiene el umbral del** 90 %**, de modo que la hipótesis no se respalda ni se refuta con esta muestra: queda *inconclusa*.**

El único cruce perdido es el último de la grabación, a los 3 597.6 segundos de un video de 3 600: la reproducción termina antes de que la captura procese ese fotograma. No es un fallo de la decisión de evento, sino del borde de la ventana reproducida.

La precisión es total: el sistema no registró ningún evento que no existiera. Los ocho aciertos incluyen los dos ingresos y seis de las siete salidas, con el sentido correcto en todos los casos.

Por qué no alcanza para pronunciarse

Nueve cruces son pocos para una proporción. Si la exactitud real fuera 0.95, harían falta 139 eventos para que el límite inferior del intervalo superara el 90 %; con 0.97 bastarían 71. El cálculo es un análisis de tamaño de muestra por precisión, no una estimación *post hoc* [73]. El recinto produce entre nueve y trece cruces por hora pareada, de modo que certificar el umbral exige del orden de diez a quince horas etiquetadas. Esa campaña no forma parte de esta entrega y se declara como trabajo inmediato.

Datos de desarrollo, no de medición

La primera hora empleada, anterior a los ajustes que surgieron de ella, arrojó una exactitud del orden de 0.7 sobre los 13 cruces de esa hora. Ese valor **no se reporta como medición**: la hora se usó para corregir el filtrado por zona, la asignación del sentido y la estabilidad del flujo de video, de modo que su resultado está contaminado por el ajuste. Se consigna aquí porque describe el punto de partida y porque la diferencia con la hora de validación muestra cuánto dependía el sistema de correcciones que no eran de método.

3.7.8. Rendimiento computacional: parcialmente evaluado

El criterio de evaluación preveía latencia promedio por evento, tasa de fotogramas, tiempo de inferencia y consumo de recursos, con una referencia operacional de un segundo por evento. De esa lista se midió la tasa de fotogramas del trabajador de inferencia (Subsección 3.7.9); el resto no se realizó. El sistema registra la latencia de inferencia por fotograma desde la implementación del trabajador, pero el banco comparativo entre formatos de modelo y variantes de escala sobre el hardware de destino no llegó a ejecutarse. **No se reporta latencia por evento**, y la reducción de huella en disco documentada en el apartado de optimización no se interpreta como una mejora de velocidad.

3.7.9. Capacidad de un trabajador de inferencia

El requerimiento RNF-07 exige incorporar nuevos puntos de acceso sin alterar el núcleo del subsistema de visión ni degradar el rendimiento. El trabajador corre con un solo proceso por una razón de correctitud, no de configuración: el estado de seguimiento vive en memoria, de modo que repartir los fotogramas de una misma cámara entre procesos haría que dos procesos confirmen la misma trayectoria y cuenten dos veces el mismo vehículo. Esa restricción convierte la exigencia en un techo medible: la cantidad de cámaras que un proceso puede atender a la frecuencia de muestreo configurada.

La medición se realiza con `tools/measure_worker_capacity.py` sobre los fotogramas de los accesos del estudio, con el detector y los clasificadores de la configuración de operación, sobre una GPU RTX 3050 para portátil. Se cronometran las tres etapas que paga cada fotograma: la decodificación del JPEG que transporta la cola, la inferencia —detección más

clasificación de atributos— y la agregación de trayectorias. La persistencia se excluye porque se paga una vez por vehículo confirmado y no una vez por fotograma. La Tabla 3.21 resume el resultado.

Cuadro 3.21: Costo por fotograma de un trabajador de inferencia y capacidad resultante. Rango sobre tres corridas y dos accesos.

Etapa	**Media**	**p95**
Decodificación del JPEG	6.8–8.2 ms	7.4 ms
Detección y clasificación (2 vehículos)	22.7–33.5 ms	46.5 ms
Agregación de trayectorias	< 0.1 ms	0.1 ms
Total por fotograma	29.5–43.4 ms	—
Fotogramas por segundo	23.0–29.7	—
Cámaras a 2 fps	11.5–14.9	—
Cámaras al 70 % de uso	8.1–10.4	—

La instalación del estudio opera dos cámaras con un fotograma cada medio segundo, es decir cuatro fotogramas por segundo. Con un rendimiento de 23 a 30 fotogramas por segundo, el trabajador queda entre el 14 % y el 17 % de su capacidad, y el margen admite entre ocho y diez accesos antes de que la cola crezca más rápido de lo que se drena. **RNF-07 se sostiene para la operación documentada y deja de sostenerse por encima de ese umbral**: superarlo exige enrutar los fotogramas de cada cámara a un proceso dedicado, o mover el estado de seguimiento a un almacén compartido. Ambas alternativas están previstas en el diseño y ninguna se implementa en esta entrega.

Queda fuera de esta medición la latencia por evento de extremo a extremo —espera en cola, persistencia y publicación—, por lo que el criterio temporal de RNF-01 continúa sin valor reportado.

3.7.10. Contraste de hipótesis

Cuadro 3.22: Estado de cada hipótesis específica frente a la evidencia medida. Los intervalos de AP y F1 macro se obtienen por remuestreo; el de exactitud por carácter, por remuestreo de las 99 placas y promedio macro por placa; los de recall, Exact Match y determinación de eventos, con la aproximación de Wilson [71], [72]. El Exact Match se calcula por recorte. Ningún intervalo incorpora la varianza entre corridas.

Hipótesis	**Criterio**	**Medido**	**IC 95 %**	**Veredicto**
$H_{1.1}$ detección de vehículos	mAP@50 ≥ 0.80	0.901	0.876–0.924	respaldada
$H_{1.2}$ tipo de vehículo	F1 macro ≥ 0.85	0.8887	0.866–0.910	respaldada
$H_{1.3}$ color predominante	F1 macro ≥ 0.85	0.8124	0.777–0.844	no respaldada
$H_{1.4}$ placas (detección)	recall ≥ 0.90	0.9714	0.936–0.988	respaldada
$H_{1.4}$ placas (reconocimiento)	char acc ≥ 0.90	0.851	0.811–0.890	no respaldada
$H_{1.4}$ placas (Exact Match)	EM ≥ 0.80	0.526	0.428–0.621	referencia no alcanzada
$H_{1.5}$ eventos ingreso/salida	exactitud ≥ 0.90	0.889	0.565–0.980	inconclusa

La columna de veredicto distingue tres estados que no son intercambiables: *respaldada* cuando el intervalo completo queda por encima del umbral, *no respaldada* cuando queda por debajo, e *inconclusa* cuando el intervalo contiene el umbral. Esa regla equivale a una prueba unilateral de intervalo: se postula $H_0 : \pi \geq \theta$ frente a $H_1 : \pi < \theta$ con $\alpha = 0.05$, de modo que un intervalo íntegramente por debajo de θ rechaza la hipótesis de investigación y uno íntegramente por encima la respalda; cuando el intervalo contiene θ no hay pronunciamiento posible. La aceptación de la hipótesis nula es, en este sentido, una decisión tomada con el riesgo declarado y no la demostración de que el valor nulo sea verdadero [8], [9], [10], [11], [74], [75]. El caso de color es el más estrecho: su intervalo superior (0.849) roza el umbral, de modo que el incumplimiento es marginal y no se usa como sostén del veredicto general. El macro sobre las once clases (0.5702) no se reporta como medición porque incluye clases con soporte insuficiente y el protocolo las excluye del promedio. Los intervalos declarados

tampoco corrigen la dependencia entre recortes de una misma placa o sesión; un intervalo que la incorporara exigiría remuestreo por conglomerados [76] y resultaría más ancho que el reportado.

3.7.11. Análisis y limitaciones

El techo del reconocimiento es el conjunto de datos

El techo no lo fija la comparación entre modelos, que el CRNN gana en ambas métricas, sino la resolución del material. Con el mismo modelo, la exactitud por carácter pasa de 0.643 en los recortes de menos de 20 píxeles de altura a 0.875 en los de 40 o más. Los recortes del primer grupo contienen siete caracteres en unos 40×13 píxeles: ningún modelo puede reconstruir información que no está en la imagen.

Con 640 placas reales —de las cuales 541 se destinan a entrenamiento— el ajuste fino dispone todavía de pocos ejemplos por carácter. La exactitud de prueba se aplana mientras la pérdida de entrenamiento continúa descendiendo: el modelo deja de aprender del dominio real y empieza a memorizar las placas vistas. El límite aparece por cantidad de datos, no por convergencia. La ampliación del conjunto de entrenamiento de 889 a 1,085 recortes —196 recortes nuevos revisados a mano— elevó la coincidencia exacta de 0.480 a 0.526 sobre el mismo test de 152 recortes, lo que confirma que la cantidad de datos sigue siendo una palanca activa y que el techo aún no se ha alcanzado. La comparación pareada sobre las mismas 152 placas da 7 aciertos solo del modelo anterior frente a 14 solo del actual; el test de McNemar [77], [78] arroja $p = 0.19$, de modo que la dirección es consistente en ambos encuadres pero el tamaño del test no alcanza para declarar la diferencia significativa.

Decisiones que se documentan por su efecto medido

Tres decisiones de implementación modificaron el resultado en un orden de magnitud mayor que el ajuste de hiperparámetros, y se reportan con su medición. Estas mediciones se tomaron sobre el protocolo anterior; se conservan porque el tamaño del efecto es lo que las hace relevantes y no depende de la partición.

- **Normalización de contraste (CLAHE):** 0.14 → 0.676 de exactitud por carácter en

la corrida de dos etapas. Sin ella, las capturas nocturnas quedan fuera de la distribución de entrenamiento.

- **Aumento geométrico de recorte:** 0.594 → 0.782 de exactitud por carácter en el sistema completo. El error de encuadre entre el detector que corta en producción y el etiquetador que cortó los datos de entrenamiento se absorbía como error de reconocimiento.
- **Selección de época por clases con soporte suficiente:** 0.7937 → 0.8124 de F1 macro en el clasificador de color, sin cambiar el modelo ni una sola imagen.

Amenazas a la validez

- **Una sola corrida por clasificador.** La varianza medida entre corridas idénticas es de 1 a 3.4 puntos de F1 macro; diferencias menores a ese rango no se interpretan como mejoras.
- **Muestra moderada en la medición de extremo a extremo.** Los 99 recortes con texto verificado implican un error estándar de 1.9 puntos en exactitud por carácter ($IC_{95} \approx \pm 3.6$) y de 5.0 puntos en coincidencia exacta ($IC_{95} \approx \pm 9.8$). La comparación con PaddleOCR en esa condición es indicativa, no concluyente.
- **Selección de la muestra de extremo a extremo.** Los 99 recortes provienen de un conjunto donde el etiquetador automático ya había propuesto una caja, por lo que la muestra no es estrictamente aleatoria respecto de la dificultad.
- **Un solo recinto y un solo conjunto de cámaras.** Todas las sesiones provienen de las mismas cámaras. La partición por bloque horario controla la fuga por identidad, y la partición por placa la elimina para el reconocimiento, pero no se controla la variabilidad entre modelos de cámara o condiciones de instalación distintas.
- **Muestra insuficiente en la determinación de eventos.** La exactitud medida (0.889) tiene un intervalo que contiene el umbral, de modo que la etapa queda inconclusa y no medida con la precisión que exige el criterio. Certificarla requiere del orden de diez a quince horas pareadas etiquetadas (Subsección 3.7.7).
- **Clases de color no evaluables.** Cinco de las once clases de color tienen menos de 40 ejemplares en prueba, con intervalos de confianza superiores a 25 puntos. Su desempeño no es interpretable con este conjunto.

3.8. Interfaces del Sistema

Las figuras de esta sección son capturas de la aplicación en operación, tomadas sobre el entorno local con la pila completa levantada —API, base de datos, cola y cliente web—, y no maquetas. Los datos que aparecen en ellas provienen de un conjunto de siembra preparado para la demostración; las imágenes de los vehículos son recortes del material real capturado en el recinto.

La comparación con los wireframes del capítulo de diseño muestra tres diferencias que conviene declarar antes de recorrer las vistas. La primera es que el panel de monitoreo no reproduce el video continuo con cajas en streaming: el sistema entrega el evento confirmado y el recorte de mayor tamaño que lo evidencia sin sobrecargar el ancho de banda. La segunda es que la vista de padrón de vehículos autorizados se implementó como vista administrativa y operativa (RF-10), complementada por la búsqueda forense de incidentes por ventana temporal. La tercera es que se añadió una vista de telemetría del modelo que no estaba en el diseño inicial, y que expone la latencia y la confianza registradas por el sistema en operación.

La aplicación ofrece tres modos de presentación —claro, oscuro y sistema—; el último sigue la preferencia del equipo. Las capturas corresponden al tema oscuro, que es el que se emplea en operación nocturna.

La navegación se organiza en dos bloques. El de operación —panel, incidentes, historial y padrón— está disponible para cualquier sesión autenticada; el de administración —cámaras, modelo y usuarios— aparece solo para el rol administrador, y el servidor aplica la misma regla sobre cada recurso. Ocultar el enlace es una cortesía de la interfaz, no el control de acceso.

3.8.1. Acceso y autenticación

La aplicación exige autenticación antes de mostrar cualquier dato. El acceso se resuelve con usuario y contraseña, y la sesión se conserva en una cookie inaccesible al código del navegador; la interfaz de programación rechaza las solicitudes de lectura que no presenten un token válido. La Figura 3.4 muestra la pantalla de acceso.

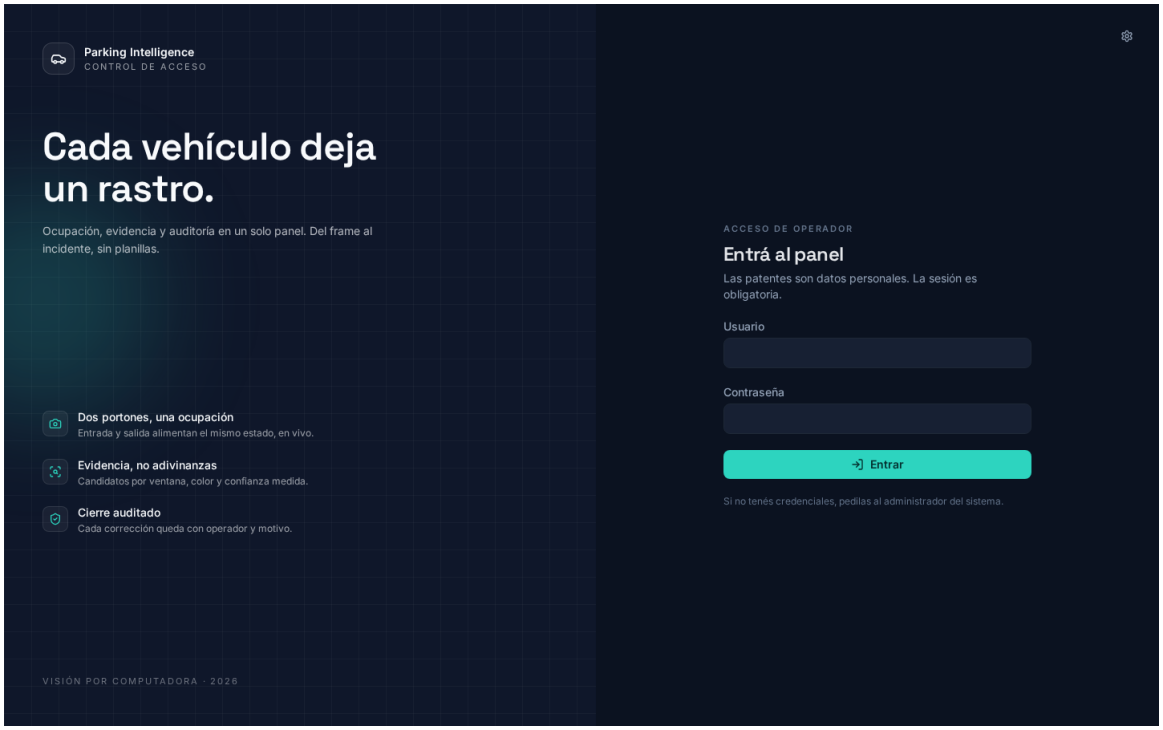


Figura 3.4: Pantalla de acceso a la aplicación.

3.8.2. Panel de monitoreo

El panel concentra el estado del recinto en una sola vista (Figura 3.5). La columna izquierda presenta la ocupación como un medidor con el número de vehículos dentro y la capacidad total; las tarjetas contiguas resumen los ingresos del último día, la estadía promedio, la latencia del percentil 95 de la inferencia y la cantidad de cámaras que reportan latido.

Debajo se listan las cámaras con su puerta asignada y el instante de la última señal, de modo que una cámara caída sea visible sin consultar registros. La lista de vehículos activos expone, por cada sesión abierta, la matrícula, el tipo, el color, la puerta de ingreso, el instante de entrada y el tiempo transcurrido, junto con la acción de cierre manual. Las sesiones cuya permanencia excede el umbral configurado se señalan con una marca de sesión huérfana: son las que quedaron sin salida registrada y que el operador debe resolver. El panel de eventos en vivo recibe las novedades por el canal persistente, y el gráfico inferior presenta los ingresos de los últimos siete días.

Respecto del wireframe de la Figura 2.5a, la diferencia principal es la ausencia del flujo de video. La decisión responde a un criterio de costo y de arquitectura: transmitir video al navegador exige un servidor de medios y ancho de banda continuo, mientras que el evento confirmado con su recorte entrega la evidencia necesaria para auditar la decisión sin ese costo.

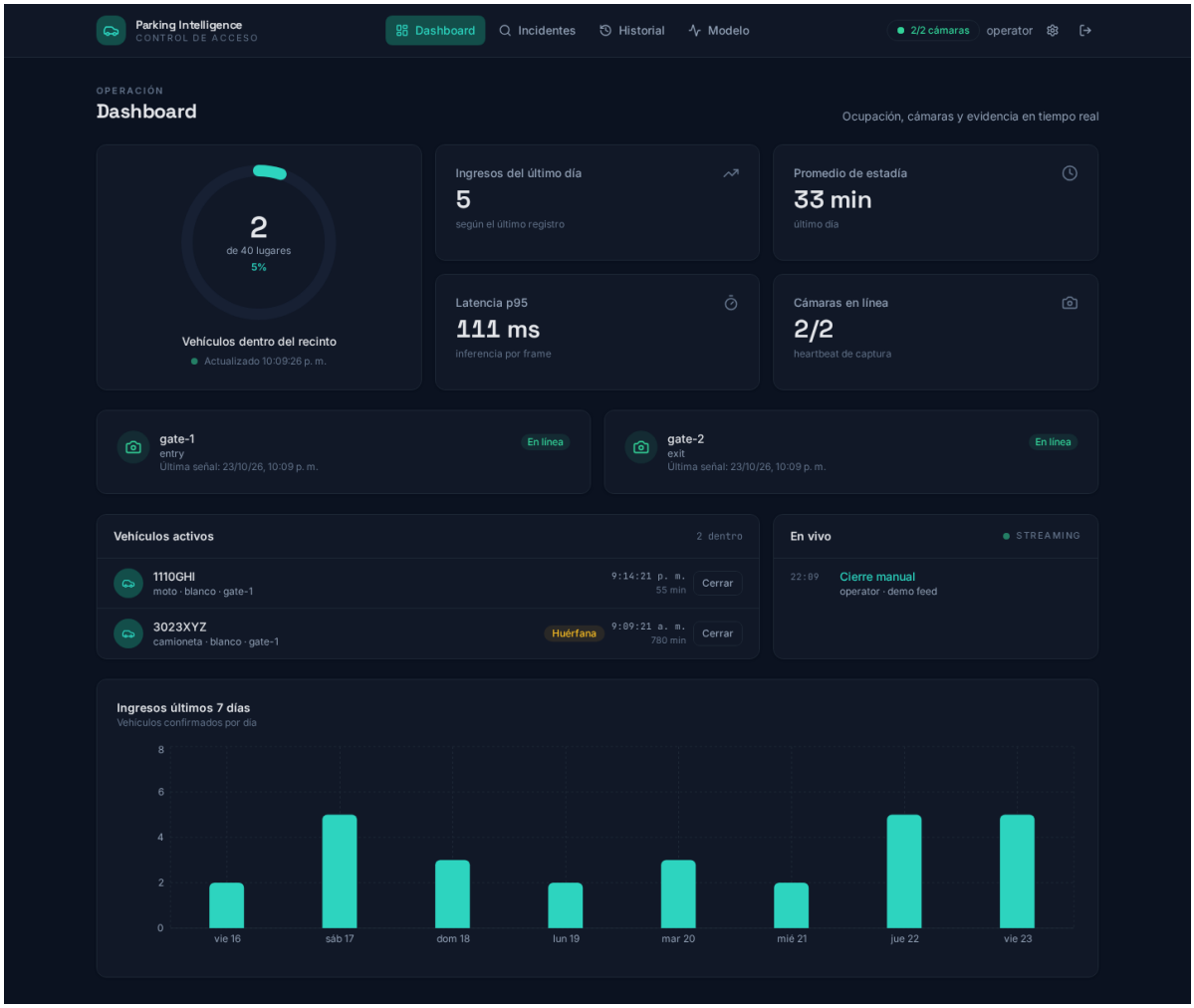


Figura 3.5: Panel de monitoreo con la ocupación, el estado de las cámaras, las sesiones activas y el registro de eventos en vivo.

La contrapartida se asume: el operador no ve el movimiento en tiempo real, sino el resultado de cada cruce.

3.8.3. Búsqueda de incidentes y detalle del vehículo

La vista de incidentes (Figura 3.6) responde a la consulta que motiva el sistema en la operación: qué vehículos pasaron por el acceso en una ventana horaria determinada. El operador fija un intervalo —por omisión, las últimas veinticuatro horas— y puede restringir por color. El filtro de color es deliberadamente blando: incluye los vehículos cuya clasificación está próxima al color pedido, porque el clasificador confunde tonos acromáticos y un filtro estricto ocultaría el candidato buscado. El vecindario no es arbitrario: se midió que *plata* y *gris* comparten cerca del 12 % de sus predicciones, y *gris* y *negro* entre 10 y 12 %; por eso la consulta de *plata* incluye *gris* y *blanco*, y los aciertos exactos se ordenan primero con la etiqueta de coincidencia exacta o vecina.

El resultado se presenta como una grilla de candidatos ordenada por cercanía al instante consultado. Cada tarjeta muestra el recorte del vehículo, la matrícula reconocida o la indicación de que no pudo leerse, el tipo y el color con su confianza, y el instante de la observación. Los casos sin matrícula se conservan en la lista: son precisamente los que el operador necesita revisar con la imagen. Seleccionar una tarjeta abre el detalle del vehículo (Figura 3.7), con el recorte de mayor tamaño disponible, los atributos con su confianza, el identificador interno y la tabla de sesiones con ingreso, salida, duración, estado y responsable del cierre.

En el recorte del detalle se observa la información que el grabador superpone a la imagen —identificador de canal, marca temporal y contador de fotogramas—. Esa superposición es la que el sistema anula antes de invocar al reconocimiento de texto (Sección 3.4.5): sin el enmascarado, el motor intentaría leer caracteres que no pertenecen a la matrícula y produciría lecturas espurias.

3.8.4. Historial de sesiones

La vista de historial presenta una tabla con las sesiones registradas, ordenada de la más reciente a la más antigua. Admite búsqueda por matrícula, tipo o color y filtros por punto de acceso y por estado de la sesión. Cada fila expone la matrícula, el tipo, el color, el

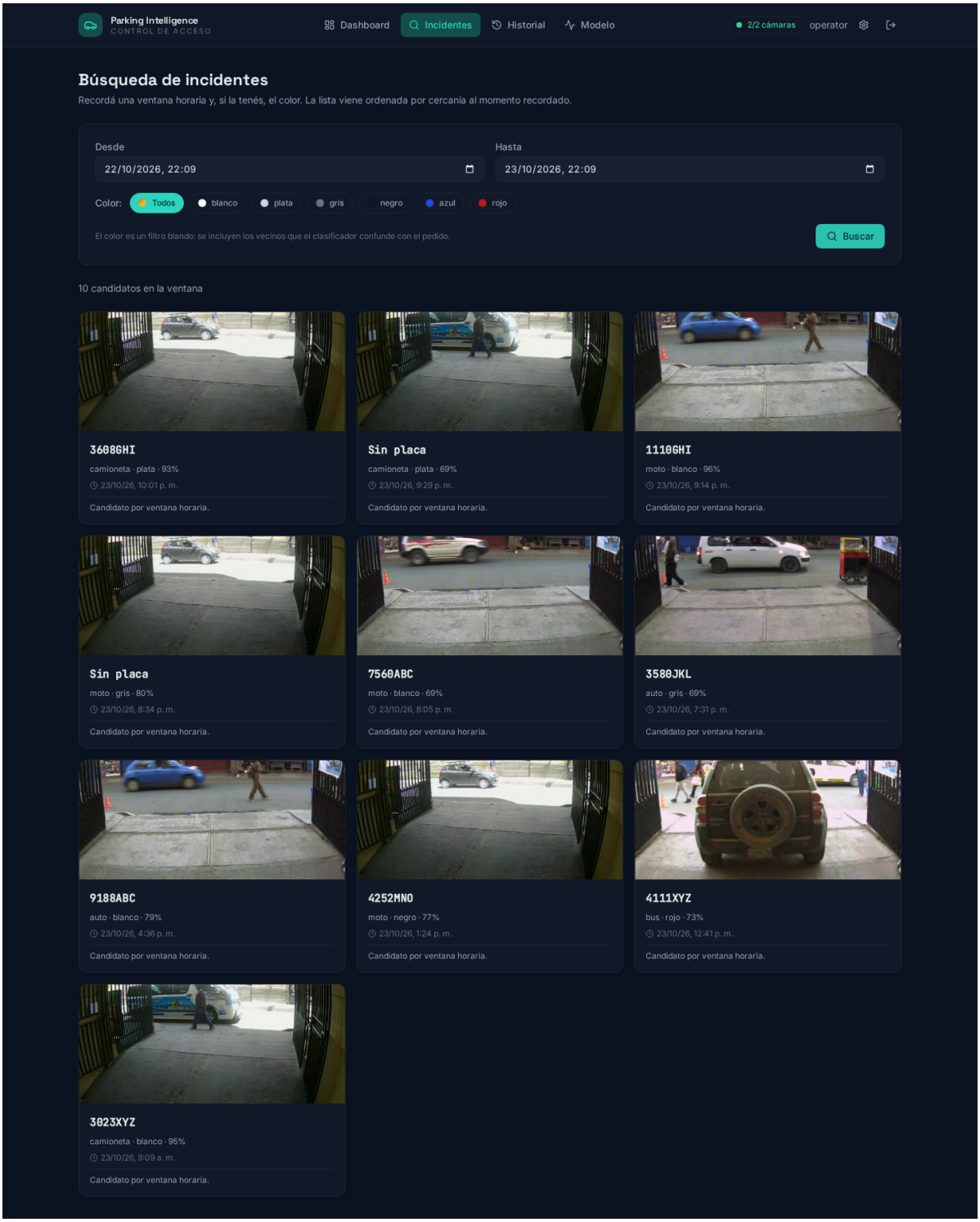


Figura 3.6: Búsqueda de incidentes por ventana temporal y color, con los candidatos y su evidencia visual.

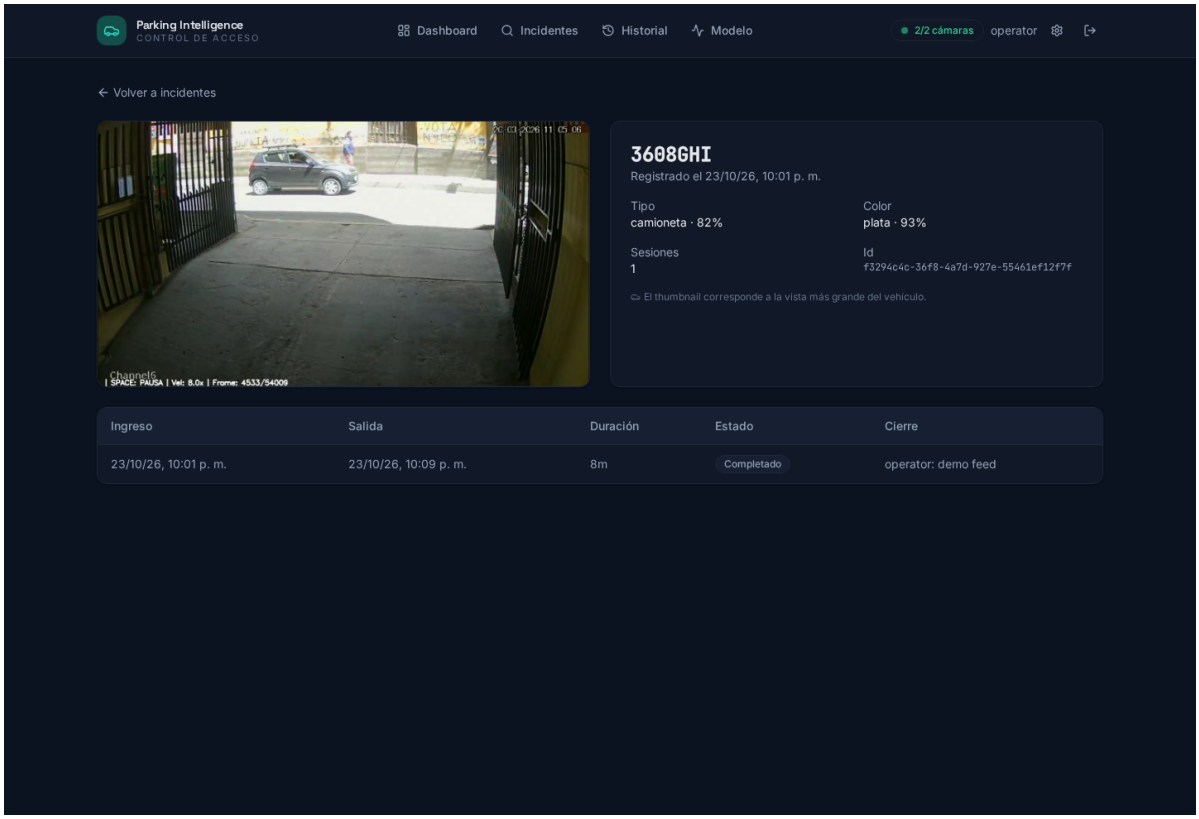


Figura 3.7: Detalle de un vehículo con su recorte, sus atributos y el historial de sesiones.

ingreso, la salida, la duración, el estado de la sesión y el responsable del cierre. La consulta subyacente está paginada por límite y desplazamiento, con un máximo de quinientos registros por solicitud; la vista solicita veinte por página y muestra el total de coincidencias junto con los controles de navegación. Su contenido es tabular y coincide en columnas con el bloque inferior de la Figura 3.7, por lo que no se reproduce como figura independiente.

3.8.5. Telemetría del modelo

La vista de desempeño (Figura 3.8) no estaba prevista en el diseño y se incorporó porque el sistema registra la latencia y la confianza de cada evento confirmado. Presenta el total de eventos de la ventana, los modelos que reportaron, el peor percentil 95 de latencia y una tabla por modelo con la confianza media y los percentiles 50 y 95 de la inferencia.

Su utilidad es diagnóstica: permite detectar una degradación de latencia o de confianza sin recurrir a la base de datos. Conviene precisar su alcance para no confundirla con una medición experimental: se alimenta de la telemetría de la operación y está sujeta a la política de retención del sistema, de modo que describe el comportamiento observado y no sustituye

el banco comparativo por formato de modelo que se declara pendiente en la Sección 3.5.6. Por esa misma razón la vista se reserva al rol de administrador, y el recurso que la alimenta exige ese rol en el servidor: la lectura del dueño es la operación del recinto, no el desempeño del detector.

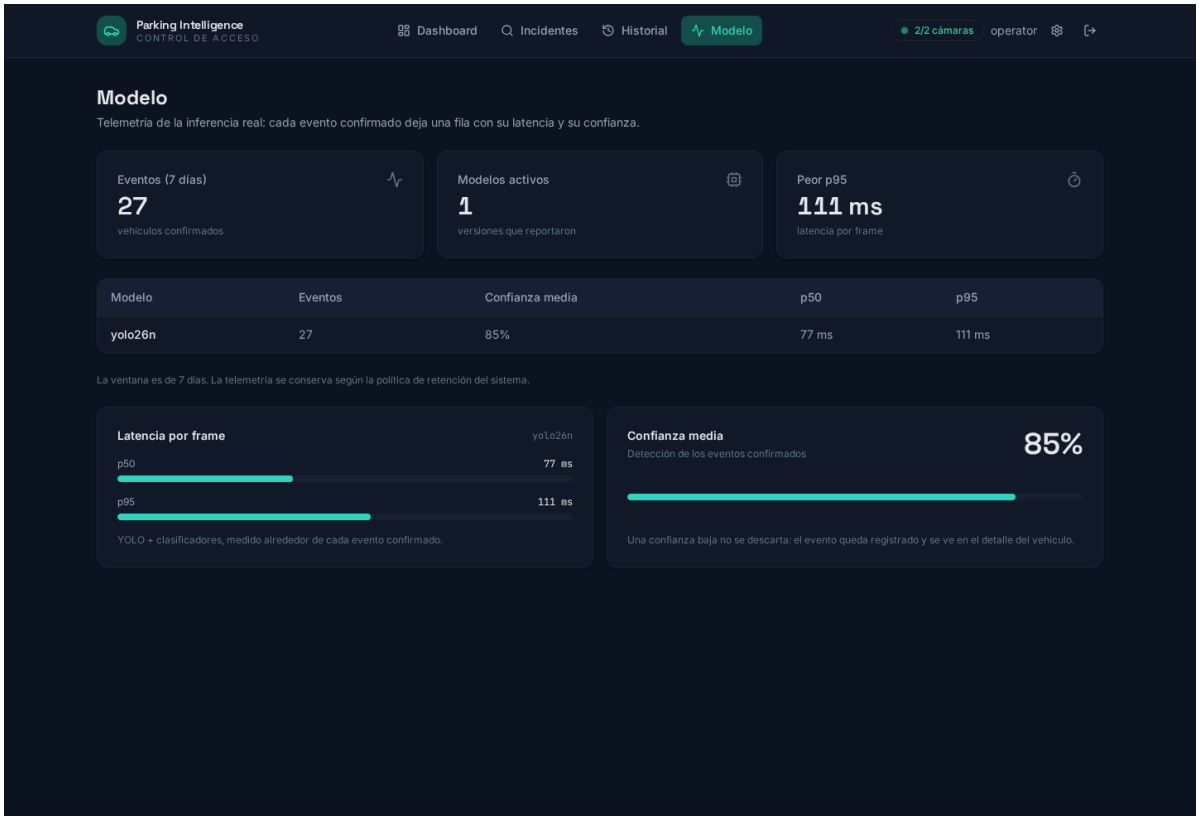


Figura 3.8: Telemetría del modelo: eventos, confianza media y latencia por percentil.

3.8.6. Vistas de administración

Las vistas de administración se reservan a los roles autorizados. El **padrón de vehículos** (RF-10) está disponible para administrador y operador: permite registrar matrículas autorizadas asignándoles titular y notas, alternar su estado activo/inactivo y eliminar registros; el motor de interfaz en tiempo real utiliza este padrón para alertar ante accesos no habilitados. Las vistas de **usuarios** y de **cámaras** quedan restringidas al administrador. La de usuarios permite listar, crear y modificar cuentas, asignar uno de los tres roles —administrador, operador o dueño— y activar o desactivar una cuenta; reproduce las reglas de negocio impidiendo eliminar al último administrador activo o a la propia cuenta. La vista de **cámaras** combina el registro persistente con el latido del capturador: muestra los puntos de acceso configurados, identifica cámaras caídas y permite dar de alta o retirar accesos.

3.8.7. Documentación de la interfaz de programación

La especificación de la interfaz se genera de forma automática a partir de las declaraciones de tipos de la aplicación y se publica en los formatos estándar de OpenAPI. El documento resultante describe las rutas y los esquemas de datos implementados, e incluye para cada operación sus parámetros, su cuerpo de solicitud, sus respuestas y los códigos de error previstos. Al derivarse del mismo código que se ejecuta, no puede desincronizarse de la implementación: un cambio en la firma de un recurso se refleja en la especificación en el siguiente arranque.

La especificación no se reproduce como figura porque su valor probatorio está en el documento generado y no en su representación visual. La Tabla 3.7 enumera las rutas, y el documento completo se obtiene desde el propio servicio, junto con una interfaz de exploración que permite ejecutar cada operación contra el sistema en funcionamiento.

Capítulo IV

Estimación de Costos

4.1. Introducción

El presente capítulo estima el costo de desarrollo e implementación del sistema, con el propósito de valorar económicamente el esfuerzo invertido y de determinar la inversión requerida para su puesta en operación. La estimación se realiza sobre artefactos verificables —el software efectivamente construido y el equipamiento efectivamente empleado— y no sobre proyecciones abstractas.

El costo se descompone en dos partidas. La primera corresponde al **desarrollo de software**, valorado a partir del tamaño funcional del sistema según el método de puntos de función, contrastado con el tamaño del código fuente medido directamente. La segunda corresponde al **equipamiento** necesario para la operación, considerando que el proyecto reutiliza la infraestructura de captura existente en el recinto y que, por consiguiente, la inversión se concentra en la capacidad de cómputo.

Quedan explícitamente fuera del alcance de esta estimación los costos de operación y mantenimiento posteriores a la puesta en marcha —consumo eléctrico, conectividad, soporte y conservación de los equipos—, así como el costo de la infraestructura de captura preexistente, que no representa un desembolso atribuible al proyecto.

Todas las valoraciones monetarias de este capítulo se expresan en bolivianos y se declaran como parámetros de valoración ajustables, dado que dependen de condiciones de

mercado locales que exceden el alcance técnico del proyecto. La metodología empleada permite recalcular la estimación ante una variación de dichos parámetros sin alterar el análisis.

4.2. Estimación del Costo de Software

La valoración del software construido se apoya en el **análisis de puntos de función** [79], [80], método estandarizado que mide el tamaño de un sistema a partir de las funciones que ofrece al usuario y no de su implementación. Esta propiedad es la que lo hace adecuado para el presente caso: permite valorar el sistema a partir de los requerimientos y de la arquitectura documentados en los capítulos precedentes, con independencia del lenguaje empleado y del estilo de codificación del desarrollador.

Se descarta explícitamente el uso de modelos paramétricos de estimación [81] construidos para proyectos industriales. Esos modelos se calibran sobre equipos de desarrollo, ciclos de vida prolongados y procesos organizacionales formalizados, condiciones que no se verifican en un proyecto de grado de desarrollo individual. Aplicarlos produciría una sobreestimación sin sustento; en su lugar se emplea un método de medición funcional cuyo resultado es verificable.

4.2.1. Tamaño funcional del sistema

El recuento clasifica cada función en una de cinco categorías —archivos lógicos internos, archivos de interfaz externa, entradas externas, salidas externas y consultas externas— y le asigna una complejidad según la cantidad de tipos de registro y de elementos de dato que manipula. La Tabla 4.1 presenta el recuento resultante.

Cuadro 4.1: Recuento de puntos de función sin ajustar

Tipo	Función	Complejidad	Puntos
Archivo lógico	Vehículos observados: atributos identificados y matrícula	Baja	7

Continuación de la Tabla 4.1

Tipo	Función	Complejidad	Puntos
Archivo lógico	Sesiones de estacionamiento: ingreso, salida y permanencia	Media	10
Archivo lógico	Detecciones: bitácora de inferencia	Media	10
Entrada externa	Registro de ingreso en el punto de acceso	Media	4
Entrada externa	Registro de salida en el punto de acceso	Baja	3
Entrada externa	Procesamiento del video archivado por el extractor	Media	4
Salida externa	Agregados diarios de ocupación y permanencia	Media	5
Salida externa	Notificación de variaciones de ocupación	Baja	4
Salida externa	Generación del conjunto de datos anotado	Media	5
Consulta externa	Sesiones actualmente activas	Baja	3
Consulta externa	Historial de sesiones con paginación	Media	4
Consulta externa	Nivel de ocupación del recinto	Baja	3
		Total sin ajustar	**62**

Fuente: Elaboración propia

La complejidad se asignó según el número de tipos de registro y de elementos de da-

to que cada función manipula. Se clasificaron como de complejidad baja las funciones que operan sobre una única estructura de datos con un número reducido de campos, y como de complejidad media aquellas que involucran una relación entre dos entidades o que devuelven información derivada de un cálculo agregado. No se clasificó ninguna función como de complejidad alta, dado que la estructura de datos del sistema es acotada y no contiene relaciones de tercer nivel ni agrupaciones jerárquicas complejas.

Al recuento sin ajustar se aplica el factor de ajuste que pondera catorce características generales del sistema. Su valor se adopta como la unidad por las siguientes razones: las características que incrementarían el factor —exigencia de desempeño, procesamiento complejo derivado del tratamiento de imágenes, y facilidad de operación e instalación— se compensan con aquellas que lo reducirían —ausencia de procesamiento distribuido, de operación en múltiples sitios y de necesidad de integración con sistemas externos—. El tamaño funcional ajustado resulta, en consecuencia, de **62 puntos de función**.

4.2.2. Contraste con el tamaño medido del código

Una verificación independiente del recuento anterior se obtiene midiendo el código fuente efectivamente escrito. La Tabla 4.2 presenta el resultado del conteo por área.

Cuadro 4.2: Tamaño del código fuente por área

Área	**Archivos**	**Líneas**
Dominio (reglas de negocio y puertos)	10	615
Aplicación (casos de uso)	2	182
Adaptadores (base de datos, cola, bus y modelos)	18	1,234
Entrada HTTP (rutas, esquemas y configuración)	18	1,408
Trabajador de inferencia	5	349
Migraciones del esquema	4	236
Captura de flujos de video	2	361
Cliente web	29	2,561

Continuación de la Tabla 4.2

Área	Archivos	Líneas
Subtotal del producto	**88**	**6,946**
Pruebas del servicio y de las herramientas	40	3,537
Herramientas de datos y entrenamiento	31	6,225
Total entregado	**159**	**16,708**

Fuente: Elaboración propia

El código se mide sobre los archivos fuente efectivamente entregados, excluyendo las dependencias de terceros, los artefactos generados y los archivos de inicialización de paquete, que no contienen lógica. El conteo distingue dos naturalezas: el **producto**, que implementa las funciones del sistema, y el material de **verificación y de construcción de datos**, que sostiene el trabajo pero no forma parte de la interfaz ofrecida al usuario.

Ambas mediciones son independientes entre sí: el tamaño funcional se derivó de las funciones documentadas y el tamaño del código, de los archivos fuente. La relación obtenida sobre el producto —6,946 líneas para 62 puntos de función, es decir aproximadamente 112 líneas por punto de función— constituye una verificación de consistencia interna del análisis. Las pruebas y las herramientas de datos se informan por transparencia, pero se excluyen de esa relación: contarlas elevaría el cociente a cerca de 270 líneas por punto de función y describiría el esfuerzo de construcción del conjunto de datos, no el tamaño funcional del sistema entregado.

4.2.3. Esfuerzo y costo de desarrollo

El esfuerzo de desarrollo se obtiene a partir del tamaño funcional y de una productividad de referencia, y el costo, a partir del esfuerzo y de una tarifa:

$$E = TFA \cdot P \qquad C_{\text{software}} = E \cdot T \tag{4.1}$$

donde TFA es el tamaño funcional ajustado —62 puntos de función—, P la productividad expresada en horas por punto de función, E el esfuerzo en horas, C_{software} el costo de desarrollo y T la tarifa horaria.

Los dos parámetros de valoración —productividad y tarifa— dependen de condiciones locales de mercado y de la experiencia del desarrollador, y no son derivables del análisis técnico. Se declaran de forma explícita en la Tabla 4.3 para que la estimación pueda reproducirse y recalcularse.

Cuadro 4.3: Parámetros de valoración del costo de software

Parámetro	**Símbolo**	**Valor**	**Procedencia**
Tamaño funcional ajustado	TFA	62 puntos	Calculado en el apartado anterior.
Productividad	P	12 horas por punto	Parámetro declarado.
Esfuerzo	E	744 horas	$TFA \cdot P$.
Tarifa horaria	T	Bs 30	Parámetro declarado.
Costo de desarrollo	C_{software}	Bs 22,320	$E \cdot T$.

Fuente: Elaboración propia

Los valores adoptados se declaran de la siguiente forma. La productividad se fija en 12 horas por punto de función, equivalentes a 744 horas de dedicación para el tamaño calculado; el valor se adopta por analogía con proyectos de magnitud comparable desarrollados en lenguajes de alto nivel, y es coherente con un desarrollo individual de varios meses. La tarifa se fija en Bs 30 por hora, que corresponde a un salario mensual de referencia de Bs 4,800 sobre una jornada de 160 horas. Con ambos parámetros, el costo de desarrollo resulta de Bs 22,320.

La estimación es sensible a la tarifa y no al tamaño: con una tarifa de Bs 20 el costo de software desciende a Bs 14,880, y con Bs 40 asciende a Bs 29,760, mientras que el tamaño funcional permanece fijo porque se deriva de las funciones documentadas y no del esfuerzo invertido. Los dos parámetros son de mercado y deben reemplazarse por la cotización vigente al momento de la presentación; la estructura del cálculo no cambia al hacerlo.

4.3. Estimación del Equipamiento

La estimación del equipamiento parte de una condición favorable del proyecto: la infraestructura de captura ya existe en el recinto y se reutiliza. El sistema no requiere la instalación de cámaras nuevas, sino que consume el flujo de video producido por el grabador digital que opera el parqueadero. En consecuencia, la inversión se concentra en la capacidad de cómputo necesaria para ejecutar la inferencia y en el almacenamiento requerido para el material de video y los conjuntos de datos derivados.

La Tabla 4.4 enumera los elementos considerados. Se distingue entre los elementos que representan un desembolso atribuible al proyecto y aquellos que se reutilizan de la infraestructura existente y que, por tanto, no se computan como inversión.

Cuadro 4.4: Equipamiento requerido por el proyecto

Elemento	**Especificación**	**Cantidad**	**Costo**
Equipo de cómputo con acelerador gráfico	GPU NVIDIA GeForce RTX 3050 (4 GB de memoria), procesador de 12 núcleos, 15 GiB de memoria principal	1	Bs 10,000
Almacenamiento externo	Unidad de 1.9 TB destinada al material de video y a los conjuntos de datos	1	Bs 850
Infraestructura de captura	Grabador digital y cámaras de los puntos de acceso, preexistentes en el recinto	—	Sin desembolso
Red y energía	Infraestructura de red local y suministro eléctrico del recinto	—	Sin desembolso
Licencias de software	Totalidad del componente de software bajo licencias de código abierto	—	Sin desembolso
		Total de equipamiento	**Bs 10,850**

Fuente: Elaboración propia

El dimensionamiento del almacenamiento se deriva del volumen efectivo del material procesado. La captura del recinto se realiza a una resolución de 1280×720 píxeles, a una tasa de quince fotogramas por segundo y con codificación de alta eficiencia, lo que produce aproximadamente 6.7 GB por cada treinta horas de registro. La generación de los conjuntos de datos añade, a su vez, poco más de 3 GB derivados del mismo material. Estas cifras sitúan los requerimientos de almacenamiento muy por debajo de la capacidad disponible, pero justifican su consideración explícita: el proceso de extracción recorre repetidamente el archivo de video y requiere conservar simultáneamente las muestras emitidas.

El equipo de cómputo merece una precisión. El acelerador gráfico empleado dispone de apenas 4 GB de memoria, un orden de magnitud inferior al de los equipos habitualmente asociados al entrenamiento de modelos de detección. Esta restricción no es un inconveniente del proyecto sino su condición de contorno: constituye la evidencia de que el sistema puede operar sobre hardware de gama moderada, que es precisamente el escenario de despliegue para el que se diseñó la estrategia de optimización descrita en el capítulo anterior. El mismo equipo ejecuta el servicio de gestión, el trabajador de inferencia y la base de datos como contenedores, sin requerir un servidor dedicado, lo que elimina una partida de inversión que el diseño evita por construcción.

Respecto de las licencias, la totalidad de los componentes empleados —lenguaje, marco de trabajo web, motor de base de datos, cola de tareas, bibliotecas de aprendizaje profundo, motor de reconocimiento de texto y biblioteca de interfaz— se distribuye bajo licencias de código abierto, de modo que no generan costo de adquisición. Se deja constancia, no obstante, de que la biblioteca de detección empleada se distribuye bajo una licencia con obligaciones de reciprocidad que condicionan su uso en un producto comercial cerrado; esta consideración no afecta al presente proyecto, de carácter académico, pero resulta relevante para una eventual explotación comercial del sistema.

Los precios unitarios se declaran a la fecha de cierre del documento: Bs 10,000 para el equipo de cómputo y Bs 850 para la unidad de almacenamiento, lo que arroja un total de Bs 10,850. Al igual que la tarifa horaria, son parámetros de mercado sujetos a variación y deben reemplazarse por cotización formal antes de la presentación. La partida es pequeña frente al costo de desarrollo, de modo que una variación de precios en el equipamiento no altera la conclusión del análisis económico.

4.4. Costo Total Estimado

El costo total del proyecto resulta de la suma de las dos partidas estimadas:

$$C_{\text{total}} = C_{\text{software}} + C_{\text{equipamiento}} \tag{4.2}$$

La Tabla 4.5 presenta la estructura de la estimación.

Cuadro 4.5: Estructura del costo total estimado

Partida	**Composición**	**Valor**
Costo de software	62 puntos de función × 12 horas por punto × Bs 30 por hora	Bs 22,320
Costo de equipamiento	Equipo de cómputo con acelerador gráfico y almacenamiento externo	Bs 10,850
	Costo total del proyecto	**Bs 33,170**

Fuente: Elaboración propia

Dos observaciones corresponden a esta estructura. La primera es que la partida de software domina la estimación —cerca del 67 % del total— porque el proyecto no requiere adquirir infraestructura de captura ni licencias, de manera que el valor del desarrollo concentra la mayor parte del costo. La segunda es que la estimación se sostiene sobre **parámetros declarados** —productividad, tarifa horaria y precios de equipo— y no sobre supuestos implícitos, lo que permite recalcularla ante una variación de las condiciones de mercado sin modificar el análisis técnico.

La valoración monetaria queda así cerrada con los parámetros declarados: Bs 33,170 como costo total, de los cuales Bs 22,320 corresponden al desarrollo y Bs 10,850 al equipamiento. Desde el punto de vista metodológico, la estimación establece un tamaño funcional verificable —62 puntos de función— corroborado por una medición independiente del código —6,946 líneas de producto en 88 archivos—, que constituye la base trazable de cualquier valoración posterior.

Capítulo V

Conclusiones y Recomendaciones

5.1. Introducción

El presente capítulo cierra el trabajo contrastando lo construido con lo comprometido. No se limita a resumir los capítulos precedentes: recorre uno por uno los objetivos específicos y las hipótesis declarados en el perfil, y para cada uno establece si la evidencia lo respalda, lo refuta o no alcanza a pronunciarse. Esa distinción es el aporte central de este capítulo, porque un sistema complejo rara vez se comporta de forma uniforme: algunos componentes superan el umbral exigido, otros quedan por debajo y otros no llegaron a medirse.

El capítulo se organiza en tres partes. La primera sintetiza el sistema efectivamente construido y delimita qué puede afirmarse de él. La segunda contrasta la hipótesis general y cada hipótesis específica, y dedica un apartado propio a la brecha del reconocimiento de matrículas, por ser el componente donde la distancia entre el criterio comprometido y el resultado medido es mayor. La tercera formula las recomendaciones que se desprenden de esas mediciones, distinguiendo las que corresponden a la continuidad inmediata del sistema de aquellas que constituyen líneas de trabajo posteriores.

Se mantiene a lo largo del capítulo una regla de escritura: ningún resultado se presenta sin la condición que lo produjo, y ninguna limitación se enuncia sin indicar qué la causa y qué la removería. Un trabajo que declara sus brechas con precisión es más útil que uno que las disimula, porque permite que el siguiente investigador sepa exactamente dónde estaba el límite y por qué.

5.2. Conclusiones

5.2.1. Síntesis del sistema construido

El trabajo entrega un sistema de gestión de acceso vehicular ensamblado a partir de dos subsistemas con responsabilidades separadas. El subsistema de visión encadena detección de vehículos sobre pesos preentrenados, seguimiento multiobjeto, un detector de placas entrenado para este dominio, un reconocedor de matrículas con arquitectura ResNet18–BiLSTM–CTC y dos clasificadores de atributos. El subsistema de gestión expone las rutas HTTP operativas, de consulta, de autenticación y de diagnóstico, junto con un canal persistente de notificación sobre FastAPI, PostgreSQL y una cola de tareas, y ofrece un cliente web con seis vistas. El despliegue se resuelve con contenedores, exportación a formatos intermedios y compilación de motores especializados para el acelerador de destino.

La magnitud del artefacto es verificable y no estimada: el sistema comprende 62 puntos de función y 6,946 líneas de producto distribuidas en 88 archivos, a las que se suman 3,537 líneas de pruebas y 6,225 de herramientas de datos. El conjunto de datos que lo sustenta también es verificable: 640 matrículas reales distintas confirmadas por un operador, 360,000 matrículas sintéticas con formato boliviano, 4,006 recortes de vehículo y 1,237 recortes de placa reales particionados por cadena de matrícula.

Esa verificabilidad alcanza al software. El subsistema de gestión se encuentra integrado con el motor de visión: la tarea de inferencia decodifica el fotograma, invoca al motor, agrega las detecciones por trayectoria y persiste el vehículo, la sesión y la observación de forma atómica, con la latencia de inferencia registrada por fotograma. El esquema se materializa en seis revisiones de migración que se aplican en orden, el cierre de sesión opera por emparejamiento exacto de matrícula con umbral de confianza y admite el cierre manual auditado, y la aplicación web ofrece nueve vistas operativas y administrativas. El control de autorización (RF-10) se integró exitosamente mediante un padrón institucional de vehículos habilitados, y la auditoría histórica (RF-11) dispone de filtrado multicriterio y exportación. La verificación automática comprende 156 pruebas unitarias y de integración que se ejecutan contra una base PostgreSQL real con las migraciones aplicadas, con una cobertura del 83 % sobre `backend/app`.

El límite que corresponde declarar ya no es la validación de campo: esa cadena se ejecutó de extremo a extremo sobre una hora completa de las dos cámaras, con verdad de

referencia humana y 0.889 de exactitud en la determinación de eventos (Subsección 3.7.7). Lo que queda pendiente es el tamaño de la muestra: nueve cruces no permiten pronunciarse sobre un umbral del 90 %, y certificarlo exige del orden de diez a quince horas pareadas etiquetadas. Los resultados se atribuyen a los componentes ejecutados y a las condiciones de instalación del recinto, no a una operación en condiciones distintas de las medidas.

5.2.2. Contrastación de la hipótesis general

La hipótesis general exige que el sistema alcance simultáneamente los criterios de desempeño establecidos para la detección e identificación de vehículos, el reconocimiento de placas y la determinación de eventos. Por su forma conjuntiva, basta que un solo término no se cumpla para que la hipótesis no se respalde, y basta que un término no se haya medido para que no pueda declararse verificada.

El estado de cada indicador se resume en la Tabla 3.22: tres criterios se cumplen (detección de vehículos, identificación de tipo y detección de placas), el F1 macro de color y la exactitud por carácter quedan por debajo de su umbral, la coincidencia exacta no alcanza la referencia declarada y la determinación de eventos queda inconclusa porque su intervalo contiene el umbral.

H_1 se rechaza y H_0 se acepta según la regla de decisión adoptada. El sistema, en su estado actual, no satisface el conjunto de criterios que la hipótesis exige: el F1 macro de color queda por debajo de su umbral con un intervalo que excluye el 0.85, y el componente de reconocimiento no alcanza la coincidencia exacta declarada como referencia de aceptación, que dista más de veinte puntos. Por la forma conjuntiva de H_1, cualquiera de los dos basta para refutarla. La regla se formuló como prueba unilateral de intervalo: $H_0 : \pi \geq \theta$ frente a $H_1 : \pi < \theta$, con $\alpha = 0.05$ [8], [9]. El F1 de color y la exactitud por carácter tienen su intervalo del 95 % íntegramente por debajo del umbral, y la referencia de coincidencia exacta queda fuera por un margen que ningún método de intervalo discute. La aceptación de H_0 se apoya además en la regla adoptada en el perfil y en la guía de la FNI, que la considera válida cuando la de investigación es descartada.

La formulación importa: no se afirma que H_0 sea verdadera —ninguna prueba lo permite [11]— sino que H_1 queda refutada bajo las condiciones evaluadas [82] y que la decisión se toma con el riesgo α declarado [10]. Por eso la conclusión se limita a las condiciones de captura y operación del estudio, y por eso cada contraste reporta el tamaño de su conjunto

de prueba y su intervalo de confianza, del cual se deriva la precisión alcanzada; sin ellos, aceptar H_0 sería confundir ausencia de evidencia con evidencia de ausencia [83].

El sostén más firme del veredicto es la coincidencia exacta: su intervalo ([0.428; 0.621]) queda más de veinte puntos por debajo de la referencia de 0.80 y esa distancia excede cualquier fuente de error considerada. La exactitud por carácter, medida por remuestreo de placas, excluye también el umbral ([0.811; 0.890]), con un margen menor pero estable frente a la semilla de remuestreo y al checkpoint. El F1 de color acompaña el veredicto, aunque su margen (0.6 puntos) es menor que la varianza medida entre corridas de entrenamiento, de modo que se reporta como confirmación y no como fundamento único.

La lectura de extremo a extremo sobre el encuadre servido (0.867 con promedio por placa, intervalo [0.818; 0.910]) contiene el umbral en todo su recorrido, de modo que tampoco allí el término se cumple con certeza; y la coincidencia exacta del sistema ensamblado (0.539 por placa, intervalo [0.429; 0.653]) sigue lejos del 0.80.

La conclusión debe formularse con una precisión adicional sobre su alcance: H_0 se sostiene por los términos medidos, no por los ausentes. La hipótesis específica $H_{0.1}$ (detección de vehículos) queda descartada por la medición: el detector supera el umbral comprometido con un límite inferior de intervalo de 0.876. La hipótesis $H_{0.5}$ (determinación de eventos) queda igualmente sin sostén, pero por un motivo distinto: su exactitud medida (0.889) tiene un intervalo de confianza que contiene el umbral del 90 %. No se declara que el sistema falle en esa etapa ni que acierte; se declara que la muestra disponible no permite pronunciarse. Un juicio en cualquiera de las dos direcciones carecería de respaldo experimental.

5.2.3. Cumplimiento de los objetivos específicos

La Tabla 5.1 contrasta cada objetivo específico con la evidencia producida y con el grado de cumplimiento que esa evidencia sostiene. Se emplean tres estados: *cumplido* cuando el objetivo se alcanzó y su resultado es verificable; *parcial* cuando el artefacto existe y fue evaluado, pero algún criterio asociado no se alcanzó o alguna medición quedó pendiente; y *no cumplido* cuando no existe evidencia que lo respalde. En este trabajo no se registra ningún objetivo en el tercer estado.

Cuadro 5.1: Cumplimiento de los objetivos específicos

Objetivo específico	**Estado**	**Evidencia y observación**
Analizar las condiciones de operación, los puntos de acceso y los requerimientos técnicos y funcionales.	Cumplido	Requerimientos funcionales, no funcionales y de infraestructura especificados con identificadores trazables; condiciones de captura, iluminación y ángulo caracterizadas a partir del material recolectado.
Construir y preparar un conjunto de datos representativo de las condiciones reales.	Cumplido	640 matrículas reales verificadas por operador, 360,000 sintéticas, 4,006 recortes de vehículo y 1,304 recortes de placa reales, de los cuales 196 se incorporaron tras revisión manual. Partición por sesión y por bloque horario para atributos y detección, y por cadena de matrícula para reconocimiento, sin placas compartidas entre entrenamiento y prueba. El volumen real es la restricción dominante del módulo de reconocimiento y así se declara.
Adaptar y entrenar modelos de detección e identificación de tipo y color, y de localización de placas.	Parcial	Tipo de vehículo: F1 macro 0.8887 sobre el umbral de 0.85. Color: F1 macro 0.8124 bajo el umbral. Localización de placas: exhaustividad 0.9714 y mAP@50 0.9898. Detección de vehículos: el detector de operación opera con pesos preentrenados y alcanza AP@50 0.901 (intervalo [0.876; 0.924]) sobre 337 fotogramas y 304 cajas verificadas por una persona.

Continuación de la Tabla 5.1

Objetivo específico	Estado	Evidencia y observación
Desarrollar y entrenar un modelo de reconocimiento óptico de caracteres para placas vehiculares y evaluarlo bajo distintas condiciones de visibilidad.	Parcial	Modelo construido y entrenado en dos etapas, evaluado sobre 152 recortes de placa (99 placas distintas) y sobre 99 recortes de vehículo con texto verificado. Exactitud por carácter 0.851 y 0.882; coincidencia exacta 0.526 y 0.566. El criterio de 0.90 de exactitud por carácter no se alcanza en el módulo de reconocimiento, y la coincidencia exacta queda lejos del 0.80 de referencia; la causa se analiza en la Subsección 5.2.4.
Integrar los modelos en un sistema de gestión con registro y consulta de eventos.	Cumplido	Modelo de datos con restricciones e índices, seis revisiones de migración, rutas HTTP operativas, de consulta, de autenticación, de administración y de diagnóstico, canal persistente de notificación, cliente web con nueve vistas y emparejamiento de salida con abstención. Verificado por 156 pruebas unitarias y de integración contra una base real, con una cobertura del 83 % sobre `backend/app`. La validación de extremo a extremo con las cámaras del recinto corresponde al objetivo de evaluación, no a este.

Continuación de la Tabla 5.1

Objetivo específico	Estado	Evidencia y observación
Evaluar el desempeño del sistema y el efecto de las técnicas de optimización.	Parcial	Cinco de los siete indicadores de contraste fueron medidos sobre conjuntos de prueba independientes. La optimización fue aplicada y su efecto sobre la huella en disco está cuantificado (reducción del 75 % entre el grafo en precisión simple con pesos externos y el artefacto en media precisión), pero la latencia, la tasa de fotogramas y la degradación de exactitud atribuible a la reducción de precisión no llegaron a medirse.

Fuente: Elaboración propia

Los tres objetivos parciales comparten un mismo patrón: el artefacto se construyó, se ejecutó y se midió, pero la medición no alcanzó el umbral comprometido o no cubrió todos los términos previstos. Ninguno de los tres constituye un objetivo abandonado. Se trata de objetivos cuyo cumplimiento está acotado por condiciones que este apartado identifica y que las recomendaciones traducen en trabajo concreto.

5.2.4. La brecha del reconocimiento de matrículas

Es el componente de mayor distancia entre lo comprometido y lo obtenido, y por eso recibe un tratamiento propio. El criterio exigía, de forma simultánea, exhaustividad de detección igual o superior al 90 % y exactitud por carácter igual o superior al 90 %, con una coincidencia exacta adicional del 80 %. El componente de detección se cumplió con holgura (0.9714 de exhaustividad y 0.9898 de mAP@50). El déficit es íntegramente de reconocimiento: 0.851 de exactitud por carácter sobre recortes aislados y 0.882 sobre el sistema ensamblado, con coincidencia exacta de 0.526 y 0.566 respectivamente. El avance respecto de la versión anterior del capítulo —0.831 y 0.782 de exactitud por carácter— es real, pero se midió sobre un conjunto de prueba estrictamente más difícil, sin placas compartidas con el entrenamiento.

La conclusión que sostiene la evidencia es que **la restricción dominante no es la**

arquitectura del modelo, sino la calidad y el volumen del material disponible. Cinco mediciones independientes convergen en esa dirección.

Primera causa: la resolución física del recorte. Un recorte de 40×13 píxeles que contiene siete caracteres dispone de aproximadamente cinco píxeles de ancho por glifo. A esa escala la información no está presente en la imagen: ningún modelo puede reconstruir un trazo que no fue capturado. La degradación es monótona con la altura del recorte —0.643 por debajo de 20 píxeles, 0.781 entre 20 y 30, 0.841 entre 30 y 40 y 0.872 por encima de 40—, lo que descarta una explicación por variabilidad muestral y confirma una relación causal con la resolución.

Este dato tiene una consecuencia decisiva para interpretar la brecha: aun eliminando por completo los recortes de baja resolución y evaluando solo el mejor tramo, el sistema alcanza 0.872, todavía por debajo del 0.90 exigido. La infraestructura de captura disponible no produce, en la generalidad de los casos, la calidad de imagen que el criterio supone.

Segunda causa: el volumen de datos reales. El ajuste fino dispone de 1,085 recortes reales de 541 placas distintas para un alfabeto de 37 símbolos, todavía pocos ejemplos por carácter. La ampliación del conjunto de entrenamiento de 889 a 1,085 recortes, medida sobre el mismo test, elevó la coincidencia exacta de 0.480 a 0.526: la cantidad de datos sigue siendo una palanca activa y el techo aún no se ha agotado por esa vía. El preentrenamiento sobre 360,000 matrículas sintéticas elevó el piso del desempeño, pero no su techo. La calibración del generador contra los recortes reales igualó el tono —brillo medio 110 sintético frente a 101 real, contraste 37 frente a 38, relación rojo/azul 1.49 frente a 1.55—, de modo que la diferencia de dominio que persiste ya no es de iluminación sino de fidelidad del glifo: tipografía, relieve y oclusiones por sujeción.

Tercera causa: la geometría del recorte. La comparación entre evaluar con la caja provista por un operador y con la caja producida por el detector propio aísla el error atribuible al encuadre. Con el detector entrenado para este dominio, la caja automática ya no penaliza al reconocedor: sobre los 99 recortes ensamblados rinde 0.882 de exactitud por carácter frente a 0.854 de la caja humana, porque adapta el encuadre a cada imagen mientras el recorte de referencia usa márgenes fijos. El costo de la localización, en cambio, sí aparece con un localizador genérico: de las 700 propuestas del etiquetador automático que correspondían a una placa, solo 183 (26.1 %) resultaron utilizables sin corrección. La geometría del recorte

condiciona la lectura, y por eso el detector propio es parte de la solución y no solo del pipeline.

Cuarta causa: el formato impide corregir por gramática. Se implementó y se midió un post-procesamiento determinista basado en la gramática oficial de las matrículas bolivianas. Su efecto fue contrario al esperado: redujo ligeramente la exactitud por carácter en la medición original ($0.831 \rightarrow 0.826$), porque el formato admite letras dentro del bloque numérico y la regla convierte lecturas correctas en incorrectas. El mecanismo se descartó como criterio de producción y se conserva únicamente como referencia sintáctica. La lección es que el formato boliviano ofrece menos restricciones explotables que otros formatos, y que la corrección sintáctica no sustituye la calidad de captura.

Por qué no se trata de un problema de ajuste. Tres observaciones descartan que la brecha se cierre con calibración de hiperparámetros. Primero, la degradación con la resolución es la misma en dos arquitecturas sin nada en común: el CRNN pasa de 0.643 a 0.872 de exactitud por carácter entre los recortes de menos de 20 píxeles y los de 40 o más, mientras que PaddleOCR pasa de 0.403 a 0.663 en los mismos tramos de la medición original. Que ambos fallen en el mismo tipo de recorte apunta a un límite del material y no del método. Segundo, la diferencia entre la mejor época (0.851) y la última (0.850) es inferior al error estándar del conjunto de prueba, estimado en 3.7 puntos con $n = 152$ ($\mathrm{IC}_{95} \approx \pm 7$ sobre la probabilidad de coincidencia exacta); elegir el máximo no aporta una mejora que el test pueda resolver. Tercero, las intervenciones que sí produjeron cambios de magnitud fueron todas de preprocesamiento y de datos, no de arquitectura: la normalización de contraste llevó la exactitud de 0.14 a 0.676, el aumento geométrico de recorte de 0.594 a 0.782, y la corrección del criterio de selección de época de 0.7937 a 0.8124 en el clasificador de color, sin modificar el modelo ni una sola imagen.

Quinta causa: el preentrenamiento sintético no transfiere. A las cuatro causas anteriores se suma una quinta, medida de forma directa en la Subsección 3.7.6: la etapa 1 se evalúa sobre ambos dominios en cada época y el resultado es concluyente. Mientras la exactitud sintética asciende de 0.923 a 0.948, la real oscila entre 0.094 y 0.133 sin tendencia y la coincidencia exacta permanece en 0.000 en las siete mediciones disponibles. La relación no es de volumen sino de fidelidad del generador: el modelo converge en su propio dominio y no adquiere la representación del glifo real, porque el renderizador emplea tipografía sustituta, omite el relieve de la placa y muestrea la degradación de forma independiente en lugar de reproducir la física del acceso. El preentrenamiento elevó el piso —el reconocedor arranca su

ajuste fino en 0.148— pero no el techo, y por eso el resultado final depende del material real disponible.

Consecuencia operativa. El reconocedor emite siempre una cadena plausible y acumula aciertos parciales: por eso supera al motor genérico en exactitud por carácter y queda por debajo en coincidencia exacta, dado que el motor genérico devuelve vacío cuando no alcanza su umbral de confianza. Para el control de acceso, esa diferencia no es neutra: una matrícula incorrecta registrada es peor que una matrícula no leída, porque traslada el error a la base de datos y a la trazabilidad del vehículo. De aquí se desprende una conclusión de diseño, y el sistema la implementa: el emparejamiento entre ingreso y salida exige coincidencia exacta de matrícula y un umbral de confianza de 0.6, y cuando no lo alcanza deja la sesión abierta en lugar de cerrarla con la mejor aproximación. La alternativa —usar el extractor del clasificador de tipo como señal de identidad— se midió y se descartó: sobre 400 recortes, el 19.9 % de los pares de vehículos distintos superó el umbral de similitud de 0.95, de modo que ningún punto de corte los separa. La consecuencia operativa es que el sistema no opera de forma autónoma frente a una lectura dudosa: la deja visible y la resuelve un operador con registro de motivo y responsable. Esa es la respuesta correcta a un techo de reconocimiento medido, y no un rodeo.

Qué haría falta para cerrar la brecha. Las mediciones permiten formular la condición necesaria con una cifra: una altura de placa igual o superior a 40 píxeles en el punto de lectura, combinada con un volumen de datos reales un orden de magnitud superior al actual. En esas condiciones el sistema ya alcanza 0.872 de exactitud por carácter sobre el tramo de mayor resolución, y la brecha restante es abordable con las técnicas que se recomiendan en el apartado siguiente —votación sobre múltiples fotogramas de la trayectoria, hoy limitada al mejor fotograma, y decodificación restringida solo en los bloques donde la gramática no es ambigua—. Ninguna de esas técnicas fue ejecutada en este trabajo: se enuncian como trabajo futuro y no como resultado. El aporte de esta sección es haber establecido, con evidencia, cuál es la condición de captura por debajo de la cual el criterio del 90 % es inalcanzable con cualquier modelo.

5.2.5. Alcance del aporte

La hipótesis general se rechaza, y conviene decir con precisión qué significa eso y qué no. No significa que el sistema no funcione: significa que no satisface simultáneamente los cinco criterios que el perfil comprometió. Leída por componente, la evidencia sostiene dos usos operativos y no sostiene otros dos.

Cuadro 5.2: Qué sostiene la evidencia y qué no, por componente.

Componente	**Medido**	**Veredicto**	**Lectura operativa**
Detección de vehículos	0.901	respaldada	Un vehículo que cruza queda registrado con su instante, sentido y recorte
Tipo de vehículo	0.889	respaldada	El atributo sirve para filtrar y para describir el vehículo al operador
Detección de placas	0.971	respaldada	La placa se localiza dentro del vehículo; el recorte sirve como evidencia
Color predominante	0.812	no respaldada	Sirve para acotar una búsqueda, no para identificar
Lectura de matrícula	0.851	no respaldada	No alcanza para cerrar una sesión sin intervención humana
Determinación de eventos	0.889	inconclusa	Sin eventos falsos en la hora medida; falta muestra para afirmar el umbral

El sistema es desplegable para registrar y buscar. Cada cruce queda documentado con su evidencia visual, y la búsqueda por ventana temporal y color resuelve el caso que motivó el trabajo: encontrar el vehículo que entró en un intervalo recordado. Ese uso no depende de la lectura de matrícula ni del conteo automático, que son los dos componentes que no alcanzan.

El sistema no es desplegable como automatismo completo. La lectura de matrícula acierta la mitad de las veces de forma exacta, de modo que el cierre de sesión debe quedar a cargo del operador. Y la determinación de eventos, aunque no produjo ningún evento falso

en la hora medida, tiene un intervalo demasiado ancho para afirmar que alcanza el 90 %. Un producto que prometa ambas cosas con esta evidencia estaría prometiendo más de lo medido.

Tampoco autoriza a concluir que el enfoque sea inviable. Los incumplimientos tienen causa identificada —la resolución de captura gobierna el reconocimiento, la asociación de trayectorias gobierna la determinación de eventos— y ambas son condiciones de instalación y de algoritmo, no límites del método. El trabajo establece dónde queda el techo con este material y esta configuración; no establece dónde queda el techo del enfoque.

5.2.6. Aportes verificables

Aporte teórico. La sistematización de fundamentos se complementa con un resultado empírico que no estaba en el perfil: la demostración de que, en reconocimiento de matrículas, el techo de desempeño está gobernado por la resolución de captura y el volumen de datos antes que por la capacidad del modelo, y la estimación de la condición de captura mínima compatible con un umbral del 90 %.

Aporte práctico. El pipeline integrado, el conjunto de datos construido y verificado, el sistema de gestión con su interfaz de programación y su cliente web, y los protocolos de evaluación quedan disponibles como artefactos reutilizables. El conjunto de datos es el activo de mayor valor: 640 matrículas reales con texto confirmado por operador y 1,237 recortes de placa reales, de los cuales 196 se obtuvieron revisando manualmente propuestas del detector, no se obtienen sin el trabajo de inspección que aquí se documenta.

Aporte académico. El trabajo deja registro de resultados negativos que rara vez se publican: el post-procesamiento sintáctico medido y desactivado, las configuraciones de entrenamiento descartadas con su efecto cuantificado, los dos componentes que no alcanzan su umbral con la causa de cada uno, y la hipótesis que quedó inconclusa con el tamaño de muestra que haría falta para resolverla. Ese registro evita que un trabajo posterior repita el mismo camino y permite discutir el desempeño con datos en lugar de impresiones.

5.3. Recomendaciones

Las recomendaciones se ordenan por aquello que las origina: primero las que habilitan el funcionamiento del sistema como un todo, luego las que atacan la causa de la brecha de reconocimiento, después las que completan la medición pendiente y, finalmente, las que desbordan el alcance de este trabajo y constituyen líneas de continuidad.

5.3.1. Validación de extremo a extremo y puesta en operación

1. **Ejecutar la simulación de extremo a extremo con las dos cámaras.** La cadena técnica está integrada y probada por componente, pero el sistema no fue ejercitado sobre una secuencia completa de ingresos y salidas con las dos fuentes sincronizadas. Esa corrida es la que sostiene la medición de $H_{1.5}$ y la que permite afirmar que el conjunto opera como un todo continuo. La infraestructura de simulación ya existe y opera sobre el material grabado; lo pendiente es su ejecución con registro de resultados.

2. **Validar en campo contra la infraestructura del recinto.** La simulación no reproduce la negociación con el grabador real ni el comportamiento de la red local. Un despliegue piloto en un punto de acceso, con supervisión y sin sustituir el control manual, es la instancia que expone las diferencias entre el laboratorio y la operación.

3. **Completar el banco comparativo de latencia y tasa de fotogramas.** El sistema ya registra la latencia de inferencia por fotograma, de modo que el dato se acumula desde la primera corrida. Lo que resta es la comparación controlada entre formatos de modelo y variantes de escala sobre el hardware de destino, con la degradación de exactitud atribuible a la media precisión.

4. **Calibrar el umbral de confianza del emparejamiento.** El valor de 0.6 se adoptó como política de rechazo y no como resultado de una curva. Calibrarlo sobre el conjunto de prueba congelado permite fijarlo con una tasa de error conocida, y revisar el criterio de abstención con datos en lugar de con criterio.

5. **Cubrir la ruta de captura con pruebas automatizadas.** La verificación alcanza el dominio, los recursos y el ciclo de persistencia. El lector RTSP y el muestreo de fotogramas operan como proceso separado y no disponen de pruebas propias: es la única pieza del camino de escritura que se apoya en verificación manual.

6. **Revalidar la valoración económica.** La estimación quedó cerrada con parámetros declarados —62 puntos de función, 12 horas por punto y una tarifa de Bs 30 por hora—

que arrojan Bs 33,170. Los tres valores son de mercado: conviene reemplazarlos por la cotización vigente al momento de la presentación, sin rehacer el análisis.

5.3.2. Condiciones de captura

Es la recomendación de mayor impacto y la de menor costo relativo, porque no requiere modificar el software. La evidencia de la Subsección 5.2.4 fija una condición concreta: para que el reconocimiento alcance el umbral comprometido, la placa debe ocupar al menos 40 píxeles de altura en el punto de lectura. Por debajo de ese valor el sistema opera en un régimen donde la información se pierde antes de llegar al modelo.

En términos de instalación, esa condición se traduce en cuatro decisiones:

- **Cámara dedicada al punto de acceso**, en lugar de reutilizar la toma general del grabador. La distancia y el ángulo deben elegirse para que la placa ocupe la altura mínima indicada en el instante de lectura, no en el mejor fotograma de la trayectoria.
- **Iluminación controlada en el punto de lectura.** Las capturas disponibles son nocturnas, con luminancia mediana de 76.6 y desviación estándar de 20.7; esa dispersión es la que obliga a normalizar contraste y la que degrada la lectura. Una fuente de iluminación fija, con filtro infrarrojo si la cámara lo admite, reduce la variabilidad de entrada.
- **Exposición y balance de blancos fijos.** El mismo vehículo gris claro aparece casi blanco bajo iluminación directa y casi negro a la sombra. Sin una referencia de exposición absoluta, la separación entre blanco, plata, gris y negro —que concentra todo el error del clasificador de color— no es resoluble por el modelo.
- **Evitar el ángulo oblicuo pronunciado.** La deformación perspectiva del recorte reduce la altura efectiva de los glifos y traslada el problema a la etapa de reconocimiento, donde ya no puede corregirse.

5.3.3. Reconocimiento de matrículas

Las acciones se ordenan por relación entre esfuerzo y efecto esperado, según lo medido:

1. **Escalar el conjunto de datos reales.** Es la intervención con mayor efecto potencial y la única que ataca la causa de fondo. La infraestructura de extracción y los protocolos de evaluación ya están construidos para escalar sin modificar el código; la tarea pendiente es de recolección y verificación, no de desarrollo. El objetivo debería fijarse en un orden de magnitud superior al volumen actual, con verificación humana mantenida en cada lote.

2. **Votar sobre múltiples fotogramas de la trayectoria.** Hoy solo el fotograma mejor puntuado de cada vehículo alcanza la etapa de reconocimiento. El seguimiento ya produce decenas de observaciones por trayectoria; consolidarlas por votación por posición o por consenso de secuencia aprovecha información que el sistema ya captura y descarta. Esta es la técnica con mejor relación entre costo de implementación y ganancia esperada en coincidencia exacta.

3. **Restringir la decodificación solo donde la gramática no es ambigua.** El postprocesamiento global falló porque el formato boliviano admite letras en el bloque numérico. Una restricción aplicada únicamente a las posiciones que el formato fija como numéricas, y desactivada en las demás, evita el efecto contrario que se midió.

4. **Calibrar la confianza y publicar la incertidumbre.** La coincidencia exacta de 0.566 en el sistema ensamblado indica que todavía una fracción apreciable de las lecturas contiene al menos un carácter erróneo. Reportar la confianza por carácter permite que el operador distinga entre una lectura utilizable y una que requiere confirmación.

5. **Mantener la revisión humana como parte del ciclo.** El pseudo-etiquetado automático fue útil como propuesta, pero solo el 26.1 % de sus cajas resultó utilizable sin corrección. La revisión humana no es una etapa transitoria: es el mecanismo que garantiza la calidad del conjunto y debe presupuestarse como tal.

5.3.4. Medición pendiente por muestra insuficiente

- **Determinación de eventos ($H_{1.5}$).** La cadena de extremo a extremo ya está implementada y medida sobre una hora pareada, con una exactitud de 0.889 cuyo intervalo contiene el umbral (Subsección 3.7.7). Lo que falta no es infraestructura sino muestra: con una exactitud real de 0.95 se requieren 139 cruces, y el recinto produce entre nueve y trece por hora pareada. La campaña consiste en etiquetar las horas restantes con el mismo procedimiento, sin volver a ajustar el sistema contra ellas.

- **Estabilidad de la detección medida.** La detección de vehículos quedó medida sobre

dos cámaras y un recinto (Subsección 3.7.2). Repetir el conjunto sobre un tercer punto de acceso, o sobre el mismo con otra altura de montaje, diría si el resultado se sostiene fuera de estas condiciones de instalación.

- **Rendimiento y optimización.** Las transformaciones aplicadas —exportación a formatos intermedios, media precisión y compilación de motores— reducen la huella del artefacto exportado en un 75 %, pero su efecto sobre latencia, tasa de fotogramas y exactitud no fue medido. Corresponde completar el banco de pruebas por formato y por variante de escala sobre el hardware de despliegue, e incluir la degradación de exactitud atribuible a la reducción de precisión, que es el dato que permite decidir si el intercambio es aceptable.

5.3.5. Líneas de continuidad

1. **Integración del padrón con barreras físicas y mecanismos RFID.** Una vez implementado el padrón lógico de vehículos autorizados (RF-10) en software, la línea de evolución natural consiste en la integración con actuadores físicos (barreras vehiculares, semáforos de acceso o control de apertura automática) o sistemas complementarios de radiofrecuencia (RFID).

2. **Clasificación de color con referencia de exposición.** El error se concentra en el bloque acromático y no es de tono sino de nivel de brillo. Un modelo que reciba una referencia de exposición —una región de calibración en la escena o una tarjeta de grises— podría separar blanco, plata, gris y negro con un criterio distinto al actual.

3. **Reidentificación y seguimiento entre cámaras.** La arquitectura prevé dos puntos de acceso. La asociación de una misma visita entre cámaras permitiría determinar eventos sin depender exclusivamente de la lectura de matrícula, lo que reduce la exposición a la brecha de reconocimiento.

4. **Despliegue en hardware de borde.** Los motores compilados son específicos del dispositivo y del controlador con que se generaron. Un procedimiento reproducible de compilación por equipo de destino es condición para distribuir el sistema más allá del equipo de desarrollo.

5. **Tratamiento de datos personales.** Las matrículas y los registros asociados constituyen datos personales. El sistema aplica una retención de 90 días para la telemetría y los recortes, y conserva el vehículo y la sesión para la trazabilidad. Antes de una operación real debe definirse la base de licitud, el plazo definitivo de conservación, quién accede

a la información y cómo se atiende una solicitud de supresión; la política de retención es, en último término, una decisión institucional que el software no resuelve por sí solo.

6. **Extensión del conjunto de datos a otros recintos.** Todas las sesiones provienen de las mismas cámaras. La partición por bloque horario controla la fuga por identidad, pero no la variabilidad entre modelos de cámara o condiciones de instalación distintas. Un conjunto con material de un segundo recinto permitiría medir la generalización, que es la pregunta que un despliegue real plantea de inmediato.

5.3.6. Consideración final

El trabajo deja un sistema construido, un conjunto de datos verificado y un conjunto de mediciones que delimitan con precisión qué sabe hacer el sistema y qué no. De los siete indicadores de contraste, dos se respaldan, tres quedan por debajo del umbral o de la referencia y dos no se contrastan; de los seis objetivos específicos, tres se cumplen y tres se cumplen parcialmente. Esa proporción no es un resultado adverso: es el mapa exacto del estado del arte alcanzado, y es la información que permite que cualquier trabajo posterior se apoye en lo construido sin repetir lo ya medido.

Bibliografía

[1] H. Deshpande, A. Shing y H. Herunde, "Comparative Analysis on YOLO Object Detection With OpenCV," *RIE Journal*, 2020. dirección: https://www.riejournal.com

[2] Z. Tang et al., "CityFlow: A City-Scale Benchmark for Multi-Target Multi-Camera Vehicle Tracking and Re-Identification," en *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, jun. de 2019, págs. 8797-8806. DOI: 10.1109/CVPR.2019.00900 dirección: https://doi.org/10.1109/CVPR.2019.00900

[3] H. Liu et al., "Camera-Based Smart Parking System Using Perspective Transformation," *Smart Cities*, vol. 6, n.º 2, págs. 1168-1184, 2023. DOI: 10.3390/smartcities6020056 dirección: https://doi.org/10.3390/smartcities6020056

[4] M. A. Alvares, "Análisis, diseño e implementación de un sistema de control de ingreso de vehículos basado en visión artificial y reconocimiento de placas en el parqueadero de la Universidad Politécnica Salesiana - sede Cuenca," Tesis de mtría., Universidad Politécnica Salesiana, Cuenca, Ecuador, 2014. dirección: https://dspace.ups.edu.ec/handle/123456789/7060

[5] A. J. Martines Samper, "Diseño e implementación de una metodología basada en inteligencia artificial capaz de contar y clasificar vehículos en los accesos de aeropuertos y de reconocer vehículos aeroportuarios," Trabajo Académico, Universitat Politècnica de València, 2022. dirección: http://hdl.handle.net/10251/184351

[6] J. Gonzalez Caballero, *Sistema inteligente de gestión de tráfico*, 2019. dirección: http://uvadoc.uva.es/handle/10324/37784

[7] Instituto Nacional de Estadística, *Estadísticas del Parque Automotor, 2003–2025*, Con base en el Registro Único para la Administración Tributaria Municipal (RUAT). Acceso: 4 de noviembre de 2026, La Paz, Bolivia, 2026. dirección: https://www.ine.gob.bo/index.php/estadisticas-economicas/transportes/parque-automotor-cuadros-estadisticos/

[8] D. J. Schuirmann, "A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, n.º 6, págs. 657-680, 1987. DOI: 10.1007/BF01068419

[9] D. Lakens, "Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses," *Social Psychological and Personality Science*, vol. 8, n.º 4, págs. 355-362, 2017. DOI: 10.1177/1948550617697177

[10] J. Neyman y E. S. Pearson, "On the Problem of the Most Efficient Tests of Statistical Hypotheses," *Philosophical Transactions of the Royal Society of London. Series A*, vol. 231, págs. 289-337, 1933. DOI: 10.1098/rsta.1933.0009

[11] R. A. Fisher, *The Design of Experiments*. Edinburgh: Oliver y Boyd, 1935.

[12] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. Pearson, 2018, ISBN: 9780133356724.

[13] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2.ª ed. Springer, 2022. DOI: 10.1007/978-3-030-34372-9 dirección: https://doi.org/10.1007/978-3-030-34372-9

[14] F. Liu, Z. Zeng y R. Jiang, "A Video-Based Real-Time Adaptive Vehicle-Counting System for Urban Roads," *PLOS ONE*, vol. 12, n.º 11, e0186098, 2017. DOI: 10.1371/journal.pone.0186098 dirección: https://doi.org/10.1371/journal.pone.0186098

[15] G. J. Auza Banegas, D. A. Calvimontes Vera, S. S. Castro Sandoval, N. L. Condori Peredo y E. Salcedo, *BLPR: Robust License Plate Recognition under Viewpoint and Illumination Variations via Confidence-Driven VLM Fallback*, 2026. DOI: 10.48550/arXiv.2604.09927 arXiv: 2604.09927 [cs.CV]. dirección: https://arxiv.org/abs/2604.09927

[16] M. S. Nixon y A. S. Aguado, *Feature Extraction & Image Processing for Computer Vision*, 3.ª ed. Academic Press, 2013, ISBN: 9780123965493. dirección: https://www.sciencedirect.com/book/9780123965493/feature-extraction-and-image-processing-for-computer-vision

[17] H. J. Trussell y M. J. Vrhel, *Fundamentals of Digital Imaging*. Cambridge University Press, 2008. DOI: 10.1017/CBO9780511754555 dirección: https://doi.org/10.1017/CBO9780511754555

[18] P. Soille, *Morphological Image Analysis: Principles and Applications*, 2.ª ed. Berlin, Heidelberg: Springer, 2003, ISBN: 978-3-540-42988-3. DOI: 10.1007/978-3-662-05088-0

[19] C.-N. E. Anagnostopoulos, I. E. Anagnostopoulos, I. D. Psoroulas, V. Loumos y E. Kayafas, "License Plate Recognition From Still Images and Video Sequences: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 9, n.º 3, págs. 377-391, 2008. DOI: 10.1109/TITS.2008.922938 dirección: https://doi.org/10.1109/TITS.2008.922938

[20] P. Tătulea, F. Călin, R. Brad, L. Brâncovean y M. Greavu, "An Image Feature-Based Method for Parking Lot Occupancy," *Acta Technica Napocensis - Electronics and Telecommunications*, vol. 60, n.º 1, págs. 21-26, 2019. DOI: 10.3390/fi11080169 dirección: https://doi.org/10.3390/fi11080169

[21] S. Chandrasekaran et al., *Computer Vision Based Parking Optimization System*, 2022. DOI: 10.48550/arXiv.2201.00095 arXiv: 2201.00095 [cs.CV]. dirección: https://arxiv.org/abs/2201.00095

[22] H. Lee et al., "A Systematic Review of Computer Vision and AI in Parking Space Allocation in a Seaport," *Applied Sciences*, vol. 13, n.º 18, 2023. DOI: 10.3390/app131810254

[23] A. Río-Álvarez, J. Andrés-Suárez, M. González-Rodríguez, D. Fernández-Lanvín y B. L. Pérez, "Effects of Challenging Weather and Illumination on Learning-Based License Plate Detection in Noncontrolled Environments," *Scientific Programming*, vol. 2019, págs. 1-16, 2019. DOI: 10.1155/2019/6897345 dirección: https://doi.org/10.1155/2019/6897345

[24] A. Voulodimos, N. Doulamis, A. Doulamis y E. Protopapadakis, *Deep Learning for Computer Vision: A Brief Review*, 2018. DOI: 10.1155/2018/7068349 dirección: https://doi.org/10.1155/2018/7068349

[25] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach* (Pearson Series in Artificial Intelligence), 4.ª ed. Hoboken, NJ: Pearson, 2020, ISBN: 9780134610993.

[26] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2.ª ed. New York, NY: Springer, 2009, ISBN: 978-0-387-84857-0. DOI: 10.1007/978-0-387-84858-7

[27] M. Mohri, A. Rostamizadeh y A. Talwalkar, *Foundations of Machine Learning*, 2.ª ed. Cambridge, MA: MIT Press, 2018, ISBN: 978-0-262-03940-6. dirección: https://cs.nyu.edu/~mohri/mlbook/

[28] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016, ISBN: 978-0-262-03561-3. dirección: https://www.deeplearningbook.org/

[29] C. M. Bishop y H. Bishop, *Deep Learning: Foundations and Concepts*. Springer, 2024, ISBN: 9783031454677. DOI: 10.1007/978-3-031-45468-4 dirección: https://doi.org/10.1007/978-3-031-45468-4

[30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, n.º 56, págs. 1929-1958, 2014. dirección: https://www.jmlr.org/papers/v15/srivastava14a.html

[31] Y. LeCun, Y. Bengio y G. Hinton, "Deep learning," *Nature*, vol. 521, n.º 7553, págs. 436-444, 2015. DOI: 10.1038/nature14539 dirección: https://doi.org/10.1038/nature14539

[32] K. Fukushima, "Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position," *Biological Cybernetics*, vol. 36, n.º 4, págs. 193-202, 1980.

[33] S. J. Pan y Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, n.º 10, págs. 1345-1359, 2010. DOI: 10.1109/TKDE.2009.191

[34] J. Yosinski, J. Clune, Y. Bengio y H. Lipson, "How Transferable Are Features in Deep Neural Networks?" En *Advances in Neural Information Processing Systems*, vol. 27, 2014. dirección: https://papers.nips.cc/paper/2014/hash/532a2f85b6977104bc93f8580abbb330-Abstract.html

[35] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 779-788. DOI: 10.1109/CVPR.2016.91 dirección: https://doi.org/10.1109/CVPR.2016.91

[36] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid y S. Savarese, "Generalized Intersection over Union: A Metric and a Loss for Bounding Box Regression," en *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, págs. 658-666. DOI: 10.1109/CVPR.2019.00075 dirección: https://doi.org/10.1109/CVPR.2019.00075

[37] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye y D. Ren, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," en *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, págs. 12 993-13 000. DOI: 10.1609/aaai.v34i07.6999 dirección: https://doi.org/10.1609/aaai.v34i07.6999

[38] S. Ren, K. He, R. Girshick y J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," en *Advances in Neural Information Processing Systems*, vol. 28, 2015, págs. 91-99. dirección: https://proceedings.neurips.cc/paper/2015/hash/14bfa6bb14875e45bba028a21ed38046-Abstract.html

[39] W. Liu et al., "SSD: Single Shot MultiBox Detector," en *European Conference on Computer Vision (ECCV)*, Springer, 2016, págs. 21-37. DOI: 10.1007/978-3-319-46448-0_2 dirección: https://doi.org/10.1007/978-3-319-46448-0_2

[40] C.-Y. Wang y H.-Y. M. Liao, "YOLOv1 to YOLOv10: The Fastest and Most Accurate Real-time Object Detection Systems," *APSIPA Transactions on Signal and Information Processing*, vol. 13, n.º 1, págs. 1-38, 2024. DOI: 10.1561/116.20240058 dirección: https://doi.org/10.1561/116.20240058

[41] Z. Tian, C. Shen, H. Chen y T. He, "FCOS: Fully Convolutional One-Stage Object Detection," en *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, págs. 9627-9636. DOI: 10.1109/ICCV.2019.00972 dirección: https://doi.org/10.1109/ICCV.2019.00972

[42] G. Jocher y J. Qiu, *Ultralytics YOLO11*, ver. 11.0.0, 2024. dirección: https://github.com/ultralytics/ultralytics

[43] R. Sapkota, R. H. Cheppally, A. Sharda y M. Karkee, *YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection*, 2026. DOI: 10.48550/arXiv.2509.25164 arXiv: 2509.25164 [cs.CV]. dirección: https://doi.org/10.48550/arXiv.2509.25164

[44] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon y M. E. Kalfaoglu, *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*, 2026. DOI: 10.48550/arXiv.2606.03748 arXiv: 2606.03748 [cs.CV]. dirección: https://arxiv.org/abs/2606.03748

[45] T.-Y. Lin, P. Goyal, R. Girshick, K. He y P. Dollár, "Focal Loss for Dense Object Detection," en *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, págs. 2999-3007. DOI: 10.1109/ICCV.2017.324 dirección: https://doi.org/10.1109/ICCV.2017.324

[46] N. Bodla, B. Singh, R. Chellappa y L. S. Davis, "Soft-NMS – Improving Object Detection With One Line of Code," en *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, págs. 5561-5569. DOI: 10.1109/ICCV.2017.593 dirección: https://doi.org/10.1109/ICCV.2017.593

[47] Y. Tian, Q. Ye y D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," *arXiv preprint arXiv:2502.12524*, 2025. arXiv: 2502.12524 [cs.CV]. dirección: https://arxiv.org/abs/2502.12524

[48] M. Lei et al., *YOLOv13: Real-Time Object Detection with Hypergraph-Enhanced Adaptive Visual Perception*, 2025. arXiv: 2506.17733 [cs.CV]. dirección: https://arxiv.org/abs/2506.17733

[49] G. E. Lima, R. Laroca, E. Santos, E. Nascimento Jr. y D. Menotti, *Toward Enhancing Vehicle Color Recognition in Adverse Conditions: A Dataset and Benchmark*, 2024. DOI: 10.48550/arXiv.2408.11589 arXiv: 2408.11589 [cs.CV]. dirección: https://arxiv.org/abs/2408.11589

[50] X. Ni y H. Huttunen, "Vehicle Attribute Recognition by Appearance: Computer Vision Methods for Vehicle Type, Make and Model Classification," *Journal of Signal Processing Systems*, vol. 93, n.º 4, págs. 357-368, 2021. DOI: 10.1007/s11265-020-01567-6 dirección: https://doi.org/10.1007/s11265-020-01567-6

[51] R. Laroca et al., "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," en *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2018, págs. 1-10. DOI: 10.1109/IJCNN.2018.8489629 dirección: https://doi.org/10.1109/IJCNN.2018.8489629

[52] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan y S. Belongie, "Feature Pyramid Networks for Object Detection," en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, págs. 2117-2125. DOI: 10.1109/CVPR.2017.106 dirección: https://doi.org/10.1109/CVPR.2017.106

[53] J. Shashirangana, H. Padmasiri, D. Meedeniya y C. Perera, "Automated License Plate Recognition: A Survey on Methods and Techniques," *IEEE Access*, vol. 9, págs. 11 203-11 225, 2021. DOI: 10.1109/ACCESS.2020.3047929 dirección: https://doi.org/10.1109/ACCESS.2020.3047929

[54] B. Shi, X. Bai y C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, n.º 11, págs. 2298-2304, 2017. DOI: 10.1109/TPAMI.2016.2646371 dirección: https://doi.org/10.1109/TPAMI.2016.2646371

[55] H. Li, P. Wang, M. You y C. Shen, "Reading Car License Plates Using Deep Neural Networks," *Image and Vision Computing*, vol. 72, págs. 14-23, 2018. DOI: 10.1016/j.imavis.2018.02.002 dirección: https://doi.org/10.1016/j.imavis.2018.02.002

[56] S. Hochreiter y J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, n.º 8, págs. 1735-1780, 1997. DOI: 10.1162/neco.1997.9.8.1735 dirección: https://doi.org/10.1162/neco.1997.9.8.1735

[57] A. Graves, S. Fernández, F. Gomez y J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," en *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, págs. 369-376. DOI: 10.1145/1143844.1143891 dirección: https://doi.org/10.1145/1143844.1143891

[58] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu y T.-K. Kim, "Multiple Object Tracking: A Literature Review," *Artificial Intelligence*, vol. 293, pág. 103 448, 2021. DOI: 10.1016/j.artint.2020.103448 dirección: https://doi.org/10.1016/j.artint.2020.103448

[59] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, n.º 1, págs. 35-45, 1960. DOI: 10.1115/1.3662552 dirección: https://doi.org/10.1115/1.3662552

[60] A. Bewley, Z. Ge, L. Ott, F. Ramos y B. Upcroft, "Simple Online and Realtime Tracking," en *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016, págs. 3464-3468. DOI: 10.1109/ICIP.2016.7533003 dirección: https://doi.org/10.1109/ICIP.2016.7533003

[61] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, vol. 2, n.º 1–2, págs. 83-97, 1955. DOI: 10.1002/nav.3800020109 dirección: https://doi.org/10.1002/nav.3800020109

[62] A. Karbalaie, F. Abtahi y M. Sjöström, "Event Detection in Surveillance Videos: A Review," *Multimedia Tools and Applications*, vol. 81, págs. 35 463-35 501, 2022. DOI: 10.1007/s11042-021-11864-2 dirección: https://doi.org/10.1007/s11042-021-11864-2

[63] M. Sokolova y G. Lapalme, "A Systematic Analysis of Performance Measures for Classification Tasks," *Information Processing & Management*, vol. 45, n.º 4, págs. 427-437, 2009. DOI: 10.1016/j.ipm.2009.03.002

[64] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto y E. A. B. da Silva, "A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit," *Electronics*, vol. 10, n.º 3, pág. 279, 2021. DOI: 10.3390/electronics10030279

[65] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," en *European Conference on Computer Vision*, 2014, págs. 740-755. DOI: 10.1007/978-3-319-10602-1_48

[66] K. Bernardin y R. Stiefelhagen, "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, págs. 1-10, 2008. DOI: 10.1155/2008/246309

[67] E. Ristani, F. Solera, R. S. Zou, R. Cucchiara y C. Tomasi, "Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking," en *European Conference on Computer Vision*, 2016, págs. 17-35. DOI: 10.1007/978-3-319-48881-3_2

[68] ISO/IEC, *ISO/IEC 25010:2011 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, Geneva, Switzerland, 2011.

[69] M. Richards y N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly Media, 2020.

[70] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Tesis doct., University of California, Irvine, Irvine, CA, 2000.

[71] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, n.º 158, págs. 209-212, 1927. DOI: 10.1080/01621459.1927.10502953

[72] L. D. Brown, T. T. Cai y A. DasGupta, "Interval Estimation for a Binomial Proportion," *Statistical Science*, vol. 16, n.º 2, págs. 101-133, 2001. DOI: 10.1214/ss/1009213286

[73] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2.ª ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[74] D. Lakens, A. M. Scheel y P. M. Isager, "Equivalence Testing for Psychological Research: A Tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, n.º 2, págs. 259-269, 2018. DOI: 10.1177/2515245918770963

[75] J. K. Kruschke, "Rejecting or Accepting Parameter Values in Bayesian Estimation," *Advances in Methods and Practices in Psychological Science*, vol. 1, n.º 2, págs. 270-280, 2018. DOI: 10.1177/2515245918771304

[76] C. A. Field y A. H. Welsh, "Bootstrapping Clustered Data," *Journal of the Royal Statistical Society: Series B*, vol. 69, n.º 3, págs. 369-390, 2007. DOI: 10.1111/j.1467-9868.2007.00593.x

[77] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, n.º 2, págs. 153-157, 1947. DOI: 10.1007/BF02295996

[78] T. G. Dietterich, "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms," *Neural Computation*, vol. 10, n.º 7, págs. 1895-1923, 1998. DOI: 10.1162/089976698300017197

[79] A. J. Albrecht, "Measuring Application Development Productivity," en *Proceedings of the Joint SHARE/GUIDE/IBM Application Development Symposium*, Monterey, CA, 1979, págs. 83-92.

[80] International Function Point Users Group, *Function Point Counting Practices Manual*, Release 4.3.1, IFPUG, Westerville, OH, 2010.

[81] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall, 1981.

[82] K. R. Popper, *The Logic of Scientific Discovery*. London: Hutchinson, 1959.

[83] D. G. Altman y J. M. Bland, "Absence of Evidence Is Not Evidence of Absence," *BMJ*, vol. 311, n.º 7003, pág. 485, 1995. DOI: 10.1136/bmj.311.7003.485

Anexo A

Manual de usuario

Este anexo describe el uso operativo del sistema para el personal del parqueadero. Está dirigido a quien supervisa el acceso, no a quien administra la infraestructura: se supone que el sistema ya está desplegado y funcionando.

A.1. Requisitos y acceso

El único requisito del lado del usuario es un navegador actualizado con acceso a la red local donde se ejecuta la aplicación. En el entorno de desarrollo la interfaz se sirve en el puerto 5173 y la interfaz de programación en el 8000; el despliegue con contenedores publica la aplicación en el puerto 8080.

El acceso exige credenciales. El sistema rechaza las solicitudes de consulta que no presenten una sesión válida, de modo que la pantalla de acceso (Figura 3.4) es el punto de entrada obligatorio.

1. Abrir la dirección de la aplicación en el navegador.
2. Introducir usuario y contraseña provistos por el administrador.
3. Confirmar. La aplicación abre en el panel de monitoreo.

La sesión se conserva en una cookie que el código del navegador no puede leer, y se

cierra desde el botón de salida del encabezado. Tras varios intentos fallidos, el servicio aplica una espera antes de admitir un nuevo intento.

A.2. Panel de monitoreo

El panel (Figura 3.5) es la vista que el operador mantiene abierta durante el turno. La Tabla A.1 describe cada elemento y qué acción corresponde cuando el valor no es el esperado.

Cuadro A.1: Elementos del panel de monitoreo

Elemento	**Qué indica**	**Qué hacer**
Medidor de ocupación	Vehículos dentro del recinto sobre la capacidad total, y el porcentaje correspondiente.	Contrastar con lo que se observa. Si no coincide, revisar las sesiones activas y las sesiones huérfanas.
Ingresos del último día y estadía promedio	Cantidad de vehículos confirmados en las últimas 24 horas y duración media de las sesiones cerradas.	Sirve de control de tendencia; no requiere acción.
Latencia p95	Tiempo de inferencia por fotograma en el percentil 95.	Un aumento sostenido indica carga en el equipo o un modelo más pesado. Consultar la vista de desempeño del modelo.
Cámaras en línea	Cámaras que reportan latido, sobre el total configurado.	Si el número baja, verificar el proceso de captura del punto de acceso y la conectividad de la cámara.
Tarjetas de cámara	Puerta asignada, estado y hora del último latido.	Es el diagnóstico por cámara: identifica cuál dejó de reportar.

Continuación de la Tabla A.1

Elemento	Qué indica	Qué hacer
Vehículos activos	Matrícula, tipo, color, puerta de ingreso, hora de entrada y tiempo transcurrido de cada sesión abierta.	Es la lista sobre la que se opera: permite cerrar una sesión manualmente.
Marca de sesión huérfana	Sesión cuya permanencia excede el umbral configurado, probablemente sin egreso registrado.	Buscar el vehículo en el historial y cerrar la sesión con el motivo correspondiente.
Panel en vivo	Eventos de ingreso, salida y variaciones de ocupación recibidos por el canal persistente.	No requiere acción; confirma que el sistema está recibiendo eventos.
Ingresos de los últimos siete días	Vehículos confirmados por día.	Control de tendencia y de estacionalidad semanal.

Fuente: Elaboración propia

A.3. Búsqueda de incidentes

Es el procedimiento que se sigue cuando hay que averiguar qué vehículo pasó por el acceso en un momento determinado, por ejemplo ante un reclamo o un incidente.

1. Abrir la vista de incidentes (Figura 3.6).
2. Ajustar el intervalo *Desde* y *Hasta*. Por omisión comprende la hora anterior y la siguiente al instante actual; conviene centrarlo en el momento recordado.
3. Elegir un color si se conoce, o dejar *todos*. El filtro es blando a propósito: incluye los vehículos cuyo color estimado está próximo al pedido, porque el clasificador confunde tonos claros y oscuros y un filtro estricto ocultaría al candidato buscado.

4. Pulsar *Buscar*. Los candidatos aparecen ordenados por cercanía al instante consultado, con su recorte, matrícula, tipo, color y confianza.
5. Identificar el vehículo por la imagen. Los casos sin matrícula reconocida permanecen en la lista: son los que requieren revisión visual.
6. Seleccionar la tarjeta para abrir el detalle.

A.4. Detalle del vehículo y cierre manual

El detalle (Figura 3.7) presenta el recorte de mayor tamaño disponible, los atributos con su confianza, el identificador interno y la tabla de sesiones con ingreso, salida, duración, estado y responsable del cierre.

El cierre manual se usa cuando el sistema no pudo cerrar la sesión por sí mismo, situación habitual cuando el vehículo egresa sin que la matrícula se lea. El procedimiento es el siguiente:

1. Desde el panel, localizar la sesión en la lista de vehículos activos.
2. Pulsar *Cerrar*.
3. Escribir el motivo del cierre, o aceptar el texto sugerido. El motivo y el usuario quedan registrados junto con el instante de salida.
4. Confirmar. La ocupación se actualiza y el evento se difunde a los demás clientes conectados.

El motivo no es un trámite: es el dato que permite auditar después por qué una sesión se cerró sin egreso automático. Un cierre sin motivo convierte la ocupación en un número sin trazabilidad.

A.5. Historial de sesiones

La vista de historial presenta las sesiones registradas, de la más reciente a la más antigua, con matrícula, tipo, color, ingreso, salida, duración, estado y responsable del cierre.

El campo de búsqueda filtra por matrícula, tipo o color. La vista solicita las doscientas sesiones más recientes; para intervalos mayores se consulta el historial paginado del servicio.

A.6. Resolución de problemas frecuentes

Cuadro A.2: Situaciones frecuentes y acción recomendada

Situación	Causa probable	Acción
Una cámara figura fuera de línea	El proceso de captura se detuvo o la cámara no responde.	Verificar el proceso de captura del punto de acceso y la conectividad de la cámara; el panel toma el último latido recibido.
Vehículo registrado sin matrícula	Resolución insuficiente, placa obstruida o lectura por debajo del umbral.	Buscar por ventana horaria y color; la imagen del recorte es la evidencia disponible.
Sesión abierta sin egreso	El vehículo salió sin que se registrara la salida.	Verificar en el historial y cerrar manualmente con el motivo.
Color informado que no coincide	Confusión entre tonos claros y oscuros, frecuente de noche.	Priorizar la imagen sobre el atributo; el color es un auxiliar de búsqueda, no una prueba.
Latencia p95 elevada	Carga del equipo o ejecución sin motor compilado.	Consultar la vista de desempeño del modelo y verificar que el motor especializado esté disponible en el equipo.
La consulta responde con error de autorización	Sesión vencida o inexistente.	Volver a iniciar sesión.

Fuente: Elaboración propia

A.7. Límites del manual

Este anexo describe la operación, no la administración del sistema. La creación de usuarios y la configuración de las cámaras se documentan en la Sección 3.8.6; la calibración de zonas, el despliegue de los modelos y la política de retención de datos corresponden a tareas de administración que exceden el alcance de este documento.

Anexo B

Protocolo de anotación y verificación humana

Este anexo documenta el procedimiento con el que se construyeron las etiquetas del conjunto de datos. Se incluye porque los resultados del capítulo de implementación dependen de él: cuando el texto afirma que una muestra fue "verificada por un operador", el procedimiento que define qué significa eso es este.

B.1. Principios

El objetivo del protocolo no es que el anotador acierte, sino que dos anotadores —o la misma persona en días distintos— elijan la misma clase ante la misma imagen. La consistencia tiene prioridad sobre el criterio individual: un conjunto consistente vale más que uno perfecto pero ruidoso, porque el ruido de etiqueta impone un techo de desempeño que ningún modelo supera.

De ese principio se derivan tres reglas de aplicación obligatoria:

1. La etiqueta se asigna por la **silueta completa** del vehículo, no por una parte, por el color ni por la marca. Reconocer un modelo comercial no decide la clase.
2. Una muestra contiene un vehículo principal. Si hay varios, se etiqueta el que domina la imagen.

3. Ante la duda no se adivina: la muestra se **descarta**. Una imagen descartada no cuesta nada; una mal etiquetada enseña al modelo un error.

B.2. Participación de los conjuntos

Ninguna partición se realizó por imagen. Un mismo vehículo aparece en decenas de fotogramas consecutivos, y distribuirlos al azar entre entrenamiento y prueba haría que el modelo memorice el vehículo en lugar de generalizar. La partición se establece por **sesión de grabación** y por **bloque horario**, y se verificó que no exista ninguna sesión compartida entre entrenamiento y prueba: el número de sesiones compartidas reportado es cero.

Para el conjunto de placas se aplicó el mismo criterio sobre bloques de grabación, de modo que el conjunto de prueba contenga horas de captura no vistas durante el entrenamiento.

B.3. Etiquetado del tipo de vehículo

La taxonomía comprende cinco clases. La Tabla B.1 resume el criterio y la pista decisiva de cada una.

Cuadro B.1: Clases de tipo de vehículo y criterio de asignación

Clase	Qué incluye	Pista decisiva
moto	Motocicleta.	Dos ruedas.
automóvil	Sedán de tres volúmenes.	Capó, cabina y **baúl saliente**; techo bajo.
vagoneta	Todoterreno, *hatchback*, rural y van de carga o familiar.	Techo alto que llega hasta el portón trasero, **sin baúl saliente**.
camioneta	Camioneta de caja abierta.	**Caja de carga abierta** atrás, incluso en cabina doble.

Continuación de la Tabla B.1

Clase	Qué incluye	Pista decisiva
minibús	Van o minibús de pasajeros.	Van **con ventanas** laterales traseras y muchas plazas.

Fuente: Elaboración propia

B.3.1. Árbol de decisión

El orden de las preguntas es parte del protocolo: si se responden en otro orden, dos anotadores pueden llegar a clases distintas. Se aplica de arriba hacia abajo y gana la primera condición que se cumpla.

1. ¿Tiene dos ruedas? Es **moto**.
2. ¿Tiene caja de carga abierta atrás? Es **camioneta**.
3. ¿Es un camión o un bus grande, de chasis pesado y muchas ruedas? Se **descarta**: no pertenece a ninguna clase de la taxonomía.
4. ¿Es una van? Con ventanas y muchas plazas es **minibús**; cerrada o de uso particular es **vagoneta**.
5. ¿Es un auto pequeño? Con baúl saliente y techo bajo es **automóvil**; con el techo continuo hasta el portón, sin baúl, es **vagoneta**.

B.3.2. Las dos confusiones reales

El error del clasificador se concentra en dos pares, y ambos tienen una regla de corte explícita:

- **automóvil ↔ vagoneta.** ¿Hay un baúl que sobresale? Es automóvil. ¿No lo hay? Es vagoneta. Un *hatchback* pequeño no es automóvil: es vagoneta.

- **vagoneta ↔ minibús.** La vagoneta es un todoterreno o una van pequeña de uso particular; el minibús es una van de transporte de pasajeros, con fila de ventanas y muchas plazas.

B.3.3. La clase fusionada

Hasta el 23 de octubre de 2026 existió una sexta clase, *furgón*, destinada a la van de carga sin ventanas laterales traseras. Se fusionó con *vagoneta* porque su única pista distintiva —la ventana trasera— no es visible de noche, con el vehículo de cola o con el portón tapado; el F1 de prueba de esa clase era 0.16 frente a 0.66 de macro. Los 98 recortes etiquetados como furgón se reauditaron individualmente: 23 se reasignaron a camioneta, 20 a vagoneta, 6 a minibús y 49 se descartaron por no ser identificables. La taxonomía publicada tiene cinco clases.

B.4. Etiquetado del color

El color se etiqueta como tono predominante de la carrocería sobre once clases. El criterio es el tono percibido, no el nivel de brillo: por eso un vehículo gris claro bajo sol directo y el mismo vehículo a la sombra comparten etiqueta. Esa ambigüedad es la que explica la confusión medida entre las clases acromáticas y se declara en el análisis de resultados.

A diferencia del tipo, el aumento de datos del clasificador de color aplica únicamente variación de exposición y nunca de saturación ni de tono: el tono es la etiqueta que el modelo debe predecir, de modo que perturbarlo sería perturbar la respuesta correcta.

Se conservan las once clases, pero el informe distingue las que alcanzan soporte suficiente en prueba (al menos 40 ejemplares) de las que no. Promediar clases con una o dos imágenes produce un macro que no describe el desempeño del modelo.

B.5. Localización de placas

El etiquetado de placas se realizó en tres etapas encadenadas, con intervención humana en la segunda.

B.5.1. Propuesta automática

Un motor de reconocimiento de texto de terceros actuó como **pseudo-etiquetador** en el sentido estricto del término: propuso una caja delimitadora por recorte de vehículo, sin que ninguna de esas cajas ingresara al conjunto de entrenamiento sin revisión. De 4,006 recortes, 727 (18.1 %) contenían una región compatible con una placa; el resto corresponde en su mayoría a vistas laterales donde la placa no es visible, es decir, negativos reales y no fallos del etiquetador.

B.5.2. Revisión humana

Un operador revisó 1,127 recortes —todas las propuestas más una muestra aleatoria de 400 negativos— y aceptó, corrigió, redibujó o descartó cada caja. El resultado fueron 868 recortes con placa (77 %) y 259 sin placa (23 %). De las 700 propuestas que sí correspondían a una placa, solo 183 (26.1 %) resultaron geométricamente utilizables sin corrección: el etiquetador encontraba la placa, pero su caja recortaba los glifos.

Una caja rechazada no se descarta del todo: se conserva como **negativo**, que es lo que impide que el detector aprenda a alucinar una placa sobre cualquier vehículo. La participación resultante fue de 768 recortes de entrenamiento, 181 de validación y 178 de prueba.

B.6. Verificación del texto de matrícula

La revisión de cajas responde *dónde* está la placa; el reconocedor responde *qué* dice. Ninguna de las dos mide el sistema ensamblado sobre un recorte de vehículo real, porque para eso hace falta el texto de una placa que el sistema no haya visto y la revisión de cajas nunca lo pidió.

Por eso se agregó una tercera etapa: un operador escribe el texto de las placas ya verificadas, sobre una muestra acotada. Esa muestra no es un conjunto de entrenamiento, es la regla con la que se mide el sistema completo. De las 100 placas propuestas para verificación, 88 resultaron legibles y conformaron el conjunto de medición de extremo a extremo.

B.7. Registro y trazabilidad

Cada muestra se inscribe en un archivo de manifiesto que conserva su ruta, su etiqueta y, cuando corresponde, una marca que la identifica como sintética. Esa marca es la que permite separar en cualquier momento el material real del generado y auditar qué proporción del entrenamiento provino de cada origen.

Los conteos publicados de cada conjunto son verificables contra el manifiesto y contra los archivos de imagen, y se resumen en la Tabla B.2.

Cuadro B.2: Composición de los conjuntos construidos

Conjunto	**Muestras**	**Origen**
Pool de atributos	4,006	5 clases
Tipo: entrenamiento	2,783	Pool congelado
Tipo: validación	590	Pool congelado
Tipo: prueba	633	Pool congelado
Color: entrenamiento	2,482	Pool congelado
Color: validación	539	Pool congelado
Color: prueba	551	Pool congelado
Recortes de placa verificados	1,237	Revisión humana
Placas: entrenamiento	1,085	Texto confirmado
Placas: prueba	152	Texto confirmado
Placas sintéticas	360,000	Formato boliviano
Medición de extremo a extremo	99	Texto verificado

El pool de atributos (4,006 recortes) y la partición de color (3,572: $2{,}482 + 539 + 551$) difieren en 434 recortes. La diferencia se explica por dos filtros declarados en el protocolo de construcción del conjunto: 426 recortes corresponden a minibuses, excluidos porque su librea hace que el color dominante dependa del ángulo de captura, y 8 recortes estaban etiquetados con la clase residual *otro*, que no se adopta como clase de entrenamiento por falta de soporte y se descarta con el umbral de 10 muestras. Ambos conjuntos no son intercambiables y cualquier comparación entre sus métricas debe hacerse sobre el mismo subconjunto.

B.8. Reproductibilidad del etiquetado

El protocolo se apoya en dos decisiones que lo hacen repetible. La primera es que las herramientas de revisión son deterministas sobre la misma entrada: la propuesta automática, la lista de candidatos y la partición por sesión se regeneran igual mientras el material de origen no cambie. La segunda es que la partición de ablación queda fijada por una semilla declarada y un archivo de protocolo que registra semilla y tamaños, de modo que cualquier medición posterior pueda repetirse sobre exactamente los mismos subconjuntos.

Anexo C

Reproducibilidad

Este anexo reúne lo necesario para repetir la construcción de los conjuntos, el entrenamiento de los modelos y la verificación del sistema. No es un manual de instalación: es la lista de condiciones y comandos que determinan los resultados reportados.

C.1. Entorno de ejecución

El entorno se declara con archivo de bloqueo de dependencias, de modo que las versiones efectivamente usadas quedan registradas y no dependen de la fecha de instalación. La Tabla C.1 resume el entorno de entrenamiento y el de operación.

Cuadro C.1: Entorno de entrenamiento y de operación

Componente	**Versión y condición**	**Uso**
Python	3.12, con gestor de dependencias y archivo de bloqueo	Servicios, trabajador y herramientas
PyTorch y torchvision	≥ 2.4 y ≥ 0.19	Entrenamiento e inferencia
Ultralytics	≥ 8.4	Detección de vehículos y de placas

Continuación de la Tabla C.1

Componente	Versión y condición	Uso
OpenCV	≥ 4.10, sin interfaz gráfica	Captura, preprocesamiento y calibración
Acelerador	GPU de 4 GB de memoria de video	Entrenamiento y compilación de motores
Almacenamiento	NVMe para conjuntos, recortes y pesos	Fuente de verdad del material
Servicios de operación	PostgreSQL 16 y Redis 7 en contenedores	Persistencia y cola de tareas
Compilación de modelos	ONNX, ONNX Runtime y TensorRT	Motor especializado por dispositivo

Fuente: Elaboración propia

C.2. Semillas y determinismo

La partición de ablación del reconocimiento de matrículas se fija con semilla 7 y proporción de prueba del 20 %, y el archivo de protocolo generado registra la semilla y los tamaños resultantes. La generación de placas sintéticas usa la misma semilla, de modo que el conjunto sintético es reproducible byte a byte mientras no cambie el generador.

Los conjuntos de tipo y color no se remuestrean: se derivan del pool congelado de recortes, y la partición se establece por sesión de grabación. Cambiar la semilla en esos conjuntos no cambia los resultados, porque la partición no es aleatoria por imagen.

C.3. Comandos por etapa

Los comandos se ejecutan desde el directorio del software. Cada bloque produce un artefacto verificable.

C.3.1. Construcción y verificación de los conjuntos

```
# Reconstruye los conjuntos de tipo y color desde el pool.
FORCE=1 ./train_all.sh datasets

# Verifica clases y tamaños antes de entrenar.
for d in "type dataset v3" "color dataset v5"; do
  grep '^nc:' "$DATASETS/$d/data.yaml"
  for s in train valid test; do
    ls "$DATASETS/$d/$s/images" | wc -l
  done
done
```

El valor esperado es 5 clases con 2,783, 590 y 633 muestras para tipo, y 11 clases con 2,482, 539 y 551 para color. La verificación es obligatoria porque el script de construcción omite un directorio existente sin comprobar su integridad: un conjunto truncado por una copia fallida se saltearía en silencio.

C.3.2. Ablación y entrenamiento del reconocimiento

```
uv run python tools/prepare_ocr_ablation.py --dataset "$DATASETS"
uv run python tools/generate_synth_plates.py \
    --dataset "$DATASETS" --count 360000 --seed 7

uv run python -u tools/train_ocr_crnn.py \
    --dataset "$DATASETS" \
    --stage stage1_synth --epochs-synth 8 \
    --lr-schedule fixed --save-dir "$RUNS/ocr_v2"
```

```
uv run python -u tools/train_ocr_crnn.py \
    --dataset "$DATASETS" \
    --stage stage2_real --epochs-real 500 \
    --lr-real 1e-3 --lr-schedule fixed --synth-mix-ratio 0 \
    --save-dir "$RUNS/ocr_v2"
```

Las banderas no son arbitrarias: cada una reemplaza una configuración que se midió y se descartó. La tasa de aprendizaje fija evita el colapso del decodificador hacia la secuencia vacía que produjo el planificador por meseta, y la ausencia de mezcla sintética en la segunda etapa mantiene el ajuste fino sobre material real.

C.3.3. Clasificadores, detector de placas y evaluación

```
./train_all.sh type       # clasificador de tipo
./train_all.sh color      # clasificador de color
./train_all.sh platedet   # detector de placas
./train_all.sh all        # todas las etapas en orden

uv run python tools/eval_type_classifier.py \
    --dataset "$DATASETS/color dataset v5" \
    --checkpoint "$DATASETS/runs/resnet18_final_color_v5_best.pt" \
    --split test

uv run python tools/report_metrics.py \
    --metrics "$DATASETS"/runs/*_metrics.json \
    --title "Clasificadores" --out ../reports/clasificadores.md
```

Cada checkpoint se acompaña de su archivo de métricas. El archivo no es un adorno: el motor de inferencia lee de allí el orden de las clases, de modo que un peso sin su archivo de métricas no puede servirse y el trabajador se niega a arrancar.

C.3.4. Intervalos de confianza

Los intervalos del Capítulo III no usan la varianza binomial sobre recuentos de recortes: varias métricas son promedios macro y los recortes de una misma placa no son independientes. Se calculan con `tools/bootstrap_ci.py`, que remuestrea la unidad realmente muestreada.

```
# 1. Exportar predicciones por ítem (una fila por recorte)
uv run python tools/eval_type_classifier.py \
    --dataset "$DATASETS/color dataset v5" \
    --checkpoint "$DATASETS/runs/resnet18_final_color_v5_best.pt" \
    --dump-predictions /tmp/color_test.jsonl

uv run python tools/train_ocr_crnn.py --dataset ../datasets/ocr_real_box \
    --stage stage2_real --eval-only --save-dir ../runs/ocr_v3_plus \
    --dump-test-predictions /tmp/ocr_test.jsonl

# 2. Intervalo por placa y decisión frente al umbral
uv run python tools/bootstrap_ci.py items /tmp/ocr_test.jsonl \
    --task text --cluster-col plate --macro-by-cluster --threshold 0.90

# 3. F1 macro desde una matriz de confusión guardada
uv run python tools/bootstrap_ci.py confusion \
    "$DATASETS"/runs/resnet18_final_color_v5_metrics.json --threshold 0.85
```

`confusion` remuestrea cada clase verdadera desde su distribución observada; `items` remuestrea placas o sesiones completas y, con `--macro-by-cluster`, promedia primero dentro de cada placa para que un vehículo con varios recortes no pese más que otro con uno. `proportion` devuelve el intervalo de Wilson, el exacto de Clopper–Pearson y la prueba binomial unilateral. Ninguno de estos intervalos incorpora la varianza entre corridas de entrenamiento, que se reporta por separado.

C.3.5. Capacidad del trabajador de inferencia

```
uv run python tools/measure_worker_capacity.py \
    --image puerta_1_1.png --image puerta_2_1.png \
```

```
    --frames 60 --warmup 8 --sample-interval 0.5
```

La herramienta usa el detector y los clasificadores de la configuración de operación, de modo que el resultado corresponde a los pesos que el sistema sirve y no a un modelo de referencia. El rango reportado en la Subsección 3.7.9 proviene de tres corridas sobre dos accesos: el costo depende de cuántos vehículos haya en el fotograma, porque los clasificadores de atributos se ejecutan una vez por vehículo.

C.3.6. Conjunto de detección y evaluación de H1.1

```
# 1. Un fotograma por pasada, sólo si el vehículo alcanza la zona interior
uv run python tools/export_detection_frames.py \
    --list seleccion.txt \
    --output ~/Smart_Parking_dataset/detection --model yolo26m.pt

# 2. Anotación humana; una de cada cuatro imágenes sin propuestas
uv run python tools/label_boxes_gui.py \
    --root ~/Smart_Parking_dataset/detection --blind-every 4

# 3. Progreso y tasa de aceptación
uv run python tools/label_boxes_gui.py \
    --root ~/Smart_Parking_dataset/detection --blind-every 4 --check

# 4. Evaluación
uv run python tools/eval_vehicle_detection.py \
    --root ~/Smart_Parking_dataset/detection \
    --model yolo26n.pt --model yolo26s.pt \
    --rounds 2000 --json ap50.json

# 5. Contraste del sesgo de anotación
uv run python tools/eval_vehicle_detection.py \
    --root ~/Smart_Parking_dataset/detection \
    --subset blind --model yolo26n.pt
```

`seleccion.txt` contiene una ruta por línea y se construye tomando las horas completas de grabación a intervalos regulares sobre el orden cronológico, para cubrir los días y

las franjas horarias sin favorecer ninguna. El evaluador exige una etiqueta por imagen: una imagen sin revisar no se puntúa como "sin vehículos", porque eso premiaría al detector por trabajo que nadie hizo.

C.3.7. Determinación de eventos y evaluación de H1.5

```
# 1. Verdad de referencia: cada cruce, con su sentido, en segundos de video
uv run python tools/label_events_gui.py \
    --video "/mnt/nina/PARKING/Vistos/C.OCCIDENTE_ch16_main_20260415140012_2026041515
    --output ~/Smart_Parking_dataset/truth/ch16_2026041514.csv --speed 2

# 2. Reproducción de extremo a extremo de la misma hora
RTSP_CAMERAS="gate-1=rtsp://127.0.0.1:8554/gate-1,gate-2=rtsp://127.0.0.1:8554/gate-2"
timeout 3600 uv run python tools/simulate_cameras.py \
    --hour 2026041514 --with-capture --transcode \
    --start-file ~/Smart_Parking_dataset/truth/val_2026041514.start

# 3. Eventos registrados por el sistema, acotados a la ventana reproducida
START=$(cat ~/Smart_Parking_dataset/truth/val_2026041514.start)
END=$(date -u -d "$START + 3600 seconds" +%Y-%m-%dT%H:%M:%S)
uv run python tools/export_system_events.py \
    --output ~/Smart_Parking_dataset/truth/system_val.json \
    --since "$START" --until "$END"

# 4. Contraste
uv run python tools/compare_events.py \
    --truth ~/Smart_Parking_dataset/truth/truth_val.csv \
    --system ~/Smart_Parking_dataset/truth/system_val.json \
    --replay-start "$START" --tolerance 10
```

Dos condiciones del procedimiento afectan el resultado y se documentan aquí. La reproducción usa `-transcode`: copiar el flujo HEVC del grabador pierde fotogramas de referencia y el detector recibe imágenes dañadas, lo que degrada el resultado por una causa ajena al método. Y la ventana del paso 3 debe acotarse con `-since` y `-until`: sin el segundo, los eventos de corridas anteriores entran en la comparación.

C.4. Artefactos

La Tabla C.2 enumera los artefactos que un tercero necesita para auditar los resultados, con su ubicación relativa al directorio del material.

Cuadro C.2: Artefactos y contenido

Artefacto	**Contenido**
`pool_20261021`	Recortes de vehículo congelados, 5 clases
`type dataset v3`	Partición de tipo con archivo de clases
`color dataset v5`	Partición de color con archivo de clases
`plates/`	Placas reales y sintéticas, manifiesto
`plates_det/`	Cajas propuestas, verificadas y texto
`runs/`	Pesos de los clasificadores y métricas
`runs/ocr_v2/`	Etapas del reconocedor y métricas finales
`ablation/protocol.json`	Semilla y tamaños de la ablación

C.5. Verificación del sistema

El software del sistema de gestión se verifica con su propio banco de pruebas. La ejecución completa comprende las pruebas unitarias y las de integración; estas últimas requieren un servidor PostgreSQL accesible y se omiten cuando no lo hay, de modo que la suite puede correr en un equipo sin servicios levantados.

```
uv run pytest            # unitarias; omite integración sin base
uv run pytest -m integration       # requiere PostgreSQL accesible
uv run alembic -c backend/alembic.ini upgrade head
```

El resultado reportado en este trabajo es de 156 pruebas aprobadas —134 unitarias y 22 de integración—, con una cobertura del 83 % sobre `backend/app`. Las pruebas de integración aplican las migraciones sobre una base vacía, ejercitan el registro y el cierre de una sesión y verifican el efecto en la base de datos.

C.6. Condiciones que limitan la reproducción

- **Los motores compilados son específicos del dispositivo.** Un motor generado para un acelerador y un controlador determinados no es portable a otro equipo, por lo que cada destino requiere su propia compilación. Los resultados de latencia no son trasladables entre equipos.
- **El material de origen no se distribuye con el documento.** Las imágenes provienen de las cámaras del recinto y su publicación está sujeta a las condiciones de tratamiento de datos descritas en el capítulo de diseño. Lo reproducible sin ellas son los procedimientos, las particiones y las métricas.
- **Una corrida no es un resultado.** La varianza medida entre corridas idénticas es de 1 a 3.4 puntos de F1 macro, de modo que una diferencia menor a ese rango no debe interpretarse como mejora. Reproducir un valor puntual exige repetir la corrida, no solo el comando.
- **Integridad del almacenamiento.** El disco externo de trabajo devolvió errores de entrada y salida en una fracción de los recortes y truncó archivos durante la transferencia. El material se migró a almacenamiento NVMe, que pasó a ser la única fuente de verdad; cualquier reproducción debe partir de esa copia y no del disco original.